

PrimeSim™ Continuum Overview

Version W-2024.09, September 2024



Copyright and Proprietary Information Notice

© 2024 Synopsys, Inc. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at <https://www.synopsys.com/company/legal/trademarks-brands.html>.

All other product or company names may be trademarks of their respective owners.

Free and Open-Source Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

www.synopsys.com

Contents

New in This Release	9
Related Products, Publications, and Trademarks	9
Conventions	10
Customer Support	11
Statement on Inclusivity and Diversity	11

1. Introducing the PrimeSim Continuum	12
What is the PrimeSim Continuum?	13
Simulation Engines in the PrimeSim Continuum	14
Synopsys Products Integrated With the PrimeSim Continuum	16

2. Getting Started With the PrimeSim Continuum	18
Configuring PrimeSim Continuum Token-Based License Mode	18
PrimeSim Continuum Simulator Documentation	21

3. Parameters and Functions	22
Using Parameters in Simulation (.PARAM)	22
Defining Parameters	23
Assigning Parameters	24
Example: Modeling an eFuse	25
Inline Parameter Assignments	25
Parameters in Output	25
User-Defined Function Parameters	27
Using Parameter Functions to Evaluate Expressions Containing Dynamic Signals	28
Predefined Analysis Function	29
Measurement Parameters	29
.PRINT and .PROBE Parameters	29
Multiply Parameter	29
Using Algebraic Expressions	29
Built-In Functions and Variables	30

External Table Parameter File Interface	35
Parameter Scoping and Passing	38
Library Integrity	39
Reusing Cells	39
Creating Parameters in a Library	39
String Parameters	41
String Parameters in Passive and Active Component Keywords	43
String Parameters in Subcircuit-based Macro Model Selection	44
Parameter Defaults and Inheritance	44
Parameter Passing	46
Parameter Passing Solutions	47
4. Library and Data Encryption	48
Library Encryption	48
Encrypting a Model Library Using the metaencrypt Utility	49
Encryption Guidelines	50
Installing and Running metaencrypt	50
Installing metaencrypt	51
Running metaencrypt	51
metaencrypt Encryption Methods	52
Traditional Library Encryption	53
Creating Files Using Traditional Encryption	54
Example: Traditional (freelib) Encryption in a PrimeSim HSPICE Netlist ..	55
8-Byte Key Encryption	56
Creating 8-byte Key Encryption	57
Placing an 8-byte Key Encrypted File Into a PrimeSim HSPICE-Compatible	
Netlist	58
Triple DES (3DES) Public and Random Keys	58
Creating 3DES Encrypted Files	59
Placing 3DES Encryption Files into a PrimeSim HSPICE-Compatible	
Netlist	60
Encrypting by 3DES Partially	61
Advanced Encryption Standard Algorithm	61
Creating AES Key Encryption	61
New Encryption Algorithm	62
General Example	62
Using the -pipe Option	64
Troubleshooting	64

Warning Message	64
Encrypting S-Parameter Files	65
Freelib Issue With Equations	66
Freelib Issue With DOS-format Files for Simulation on UNIX/Linux Platforms	66
5. Using Verilog-A	67
Getting Started	68
Introduction to Verilog-A Data Types and System Tasks and I/O Functions	69
Data Types	69
String Operators and Semantics	69
System Tasks and I/O Functions	70
Simulation With Verilog-A Modules	73
Verilog-A Output Directory	73
Loading Verilog-A Modules	74
.hdl Command Module Selection	75
.hdl Command Module Alias	75
Verilog-A File Search Path	76
Verilog-A File Loading Considerations	78
Instantiating Verilog-A Devices	79
Using Model Cards With Verilog-A Modules	80
No Restrictions on Verilog-A Module Names	81
Overriding Subcircuits With Verilog-A Modules	81
Setting the -vamodel Command-Line Option	82
Setting .OPTION vamodel in the Netlist	83
Disabling .OPTION vamodel With .OPTION spmodel (PrimeSim HSPICE Only)	84
File Path Syntax on `include Statement	84
Using Vector Buses or “Ports”	85
Using Variable-Width Buses	85
Using Integer Parameters	86
Implicit Parameter M Support	86
Module and Parameter Name Case-Sensitivity	87
M Instance Using Verilog-A	87
Instantiating Primitive Elements inside Verilog-A Modules	87
Instantiating PrimeSim Continuum Subcircuits Inside Verilog-A Modules	89
Output Simulation Data	90
V() and I() Access Functions	90

Contents

Output Bus Signals	91
Output Internal Module Variables	92
Output Module Parameters	92
.PROBE/.PRINT Commands in the PrimeSim Continuum Solution	93
Case-Sensitivity in Simulation Data Output	93
Using Wildcards in Verilog-A (PrimeSim HSPICE Only)	94
Port Probing and Branch Current Reporting Conventions	95
Unsupported Output Function Features	95
SPICE Netlist and Verilog-A Interactions	95
Passing .OPTION EXPMAX = value	96
Passing the Parameter String	96
Passing PrimeSim HSPICE-Compatible Netlist to Verilog-A	96
Passing the M-Factor	97
Illegal Format for \$strobe	99
Illegal Format for \$fscanf	100
Encrypting Verilog-A Files	100
Encryption Examples	101
Compiling Verilog-A Files	105
Supported LRM 2.4 Syntax and Features	106
How pVA Handles Strings	106
String Data Type	109
Module Port With Discipline	109
\$simparam\$str Parameter System Function	110
File Input-Output System Tasks and Functions	111
cross, above, and timer With Support for enable	111
Parameter Arrays	111
Parameter Value Range	111
Passing Array to Analog Function	112
Analog Initial	112
\$arandom	113
\$simprobe	113
\$finish_current_analysis	113
Supporting Semicolon (;) Symbols at the End of Nature and Discipline Identifiers	113
Supporting All Types of Integer Constants as Defined by the VAMS LRM 2.4	114
Sharing of Precompiled Verilog-A Files by Multiple Users	114
Performance Considerations With the PrimeSim Continuum Solution	115

Contents

Environment Variables for Forcing Compilation of Verilog-A Modules	115
Reusing Precompiled Verilog-A Files in Simultaneous Multiple Simulations . .	115
pVA Compiles ALL Modules in an .hdl Statement	117
pVA Time Derivative Function Performance Tuning Tip	118
Known Limitations	118
C Language Keywords Limitation	119
GCC Extension Keywords Prohibited	119
analysis() Function Behavior	119
Unsupported Language Features	120
Array/Vector Parameter Support	121
Verilog-A (pVA) Messages	121
Troubleshooting Error Messages	122
Environment Variables and Command Options	122
Example: Verilog-A Frequency Divider Model	124
6. Multi-Technology Simulation of 3D Integrated Circuit	127
Overview of PrimeSim HSPICE-Compatible 3DIC Simulation Netlist	127
PrimeSim HSPICE-Compatible Netlist Definitions	128
IC Module Usage	128
3DIC Netlist Construct and Usage	129
Scope Reference Rules	131
Using Verilog-A Modules Within the .MODULE Scope	132
VA Reference Usage Rules	133
VA Reference Usage Examples	133
Top-Level IC Module Reference	135
Examples of Top-Level References	135
Intrinsic Model Card Reference From a Top-Level Instance	136
Over-writing Circuit Properties	136
Parameter Direct Access From Top-Level Netlist	139
Using Parameters Defined in Top-Level Netlist	140
Transient Analysis and Alters Simulation Features	140
Sweeps for Temperature and Parameters	141
.ALTER Features	142
Full Circuit Example	144
3DIC Hierarchical IC Module Support	146
Netlist Construct for 3DIC Hierarchical IC Module Support	146

Contents

Scope Reference Rules for 3DIC Hierarchical IC Module Support	147
Parameter-passing Rules for 3DIC Hierarchical Instantiation	147
Options scale, geoshrink Inside module and optparhier	148
Module-based .temp and .option tn timer	148
Module-based .ivth and .option ivth	148
Access Model Card and Parameter Inside Module	148
Hierarchical Instantiation With Different IC Modules and Global Node Reference	149
.modulevar Usage Under Hierarchical 3DIC Netlist Configuration	150
Output Syntax for 3DIC Hierarchical IC Module Support	151
3DIC Support for TMI, Custom CMI, and MOSRA	153

About the PrimeSim Continuum Overview

The *PrimeSim Continuum Overview* highlights the common use model and features of the Synopsys® PrimeSim Continuum technology. It provides an overview of the Synopsys products that run on the PrimeSim Continuum environment.

This preface includes the following sections:

- [New in This Release](#)
- [Related Products, Publications, and Trademarks](#)
- [Conventions](#)
- [Customer Support](#)
- [Statement on Inclusivity and Diversity](#)

New in This Release

Information about new features, enhancements, and changes, known limitations, and resolved Synopsys Technical Action Requests (STARs) is available in the PrimeSim Continuum Release Notes on the [SolvNetPlus site](#).

Related Products, Publications, and Trademarks

For additional information about the PrimeSim Continuum tool, see the documentation on the Synopsys SolvNetPlus support site at the following address:

<https://solvnetplus.synopsys.com>

You might also want to see the documentation for the following related Synopsys products:

- PrimeSim™ HSPICE® (formerly HSPICE®)
- PrimeSim™ (PrimeSim™ Pro and PrimeSim™ SPICE)
- PrimeSim™ XA (formerly CustomSim™)
- PrimeWave™ Design Environment
- PrimeSim™ Reliability Analysis
- PrimeSim™ AVA
- PrimeSim™ CCK

- PrimeSim™ EMIR
- PrimeSim™ MOSRA
- PrimeSim™ Custom Fault
- VCS® PrimeSim™ AMS

Conventions

The following conventions are used in Synopsys documentation.

Convention	Description
Courier	Indicates syntax, such as <code>write_file</code> .
<i>Courier italic</i>	Indicates a user-defined value in syntax, such as <code>write_file design_list</code>
Courier bold	Indicates user input—text you type verbatim—in examples, such as <code>prompt> write_file top</code>
Purple	• Within an example, indicates information of special interest. • Within a command-syntax section, indicates a default, such as <code>include_enclosing = true false</code>
[]	Denotes optional arguments in syntax, such as <code>write_file [-format fmt]</code>
...	Indicates that arguments can be repeated as many times as needed, such as <code>pin1 pin2 ... pinN</code> .
	Indicates a choice among alternatives, such as <code>low medium high</code>
\	Indicates a continuation of a command line.
/	Indicates levels of directory structure.
Bold	Indicates a graphical user interface (GUI) element that has an action associated with it.
Edit > Copy	Indicates a path to a menu command, such as opening the Edit menu and choosing Copy .
Ctrl+C	Indicates a keyboard combination, such as holding down the Ctrl key and pressing C.

Customer Support

Customer support is available through SolvNetPlus.

Accessing SolvNetPlus

The SolvNetPlus site includes a knowledge base of technical articles and answers to frequently asked questions about Synopsys tools. The SolvNetPlus site also gives you access to a wide range of Synopsys online services including software downloads, documentation, and technical support.

To access the SolvNetPlus site, go to the following address:

<https://solvnetplus.synopsys.com>

If prompted, enter your user name and password. If you do not have a Synopsys user name and password, follow the instructions to sign up for an account.

If you need help using the SolvNetPlus site, click REGISTRATION HELP in the top-right menu bar.

Contacting Customer Support

To contact Customer Support, go to <https://solvnetplus.synopsys.com>.

Statement on Inclusivity and Diversity

Synopsys is committed to creating an inclusive environment where every employee, customer, and partner feels welcomed. We are reviewing and removing exclusionary language from our products and supporting customer-facing collateral. Our effort also includes internal initiatives to remove biased language from our engineering and working environment, including terms that are embedded in our software and IPs. At the same time, we are working to ensure that our web content and software applications are usable to people of varying abilities. You may still find examples of non-inclusive language in our software or documentation as our IPs implement industry-standard specifications that are currently under review to remove exclusionary language.

1

Introducing the PrimeSim Continuum

Describes the PrimeSim Continuum, outlines its different simulation engines, and lists additional Synopsys products that integrate with it as part of Synopsys' complete custom design platform.

This chapter includes the following sections:

- [What is the PrimeSim Continuum?](#)
- [Simulation Engines in the PrimeSim Continuum](#)
- [Synopsys Products Integrated With the PrimeSim Continuum](#)

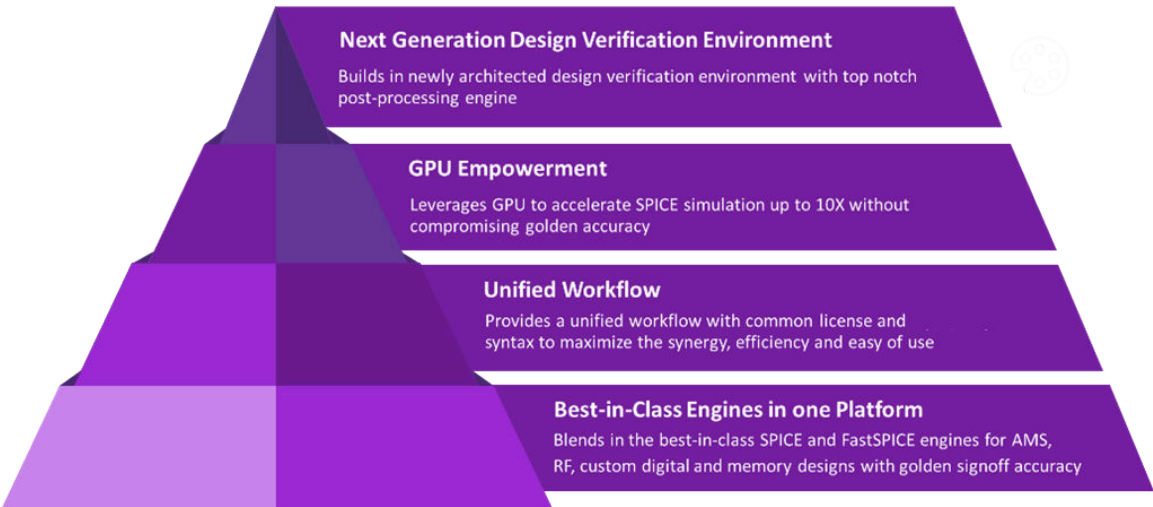
What is the PrimeSim Continuum?

The PrimeSim Continuum provides a unified workflow of best-in-class SPICE and FastSPICE engines—PrimeSim HSPICE, PrimeSim Pro, PrimeSim SPICE, and PrimeSim XA—with common licensing and common display to create, model, and analyze analog, RF, mixed signal, custom digital, and memory designs with golden signoff accuracy. The PrimeSim Continuum leverages GPU technology to accelerate SPICE simulation up to 10X and features an up-to-5X faster next-generation FastSPICE engine.

To push the productivity frontier even further, the PrimeSim Continuum blends in a new design verification environment, the PrimeWave Design Environment, and is ready to be used on the Cloud, where you can easily run, save, and restore your simulations, anywhere and anytime.

Figure 1 illustrates how the PrimeSim Continuum represents a breakthrough in SPICE simulation technology.

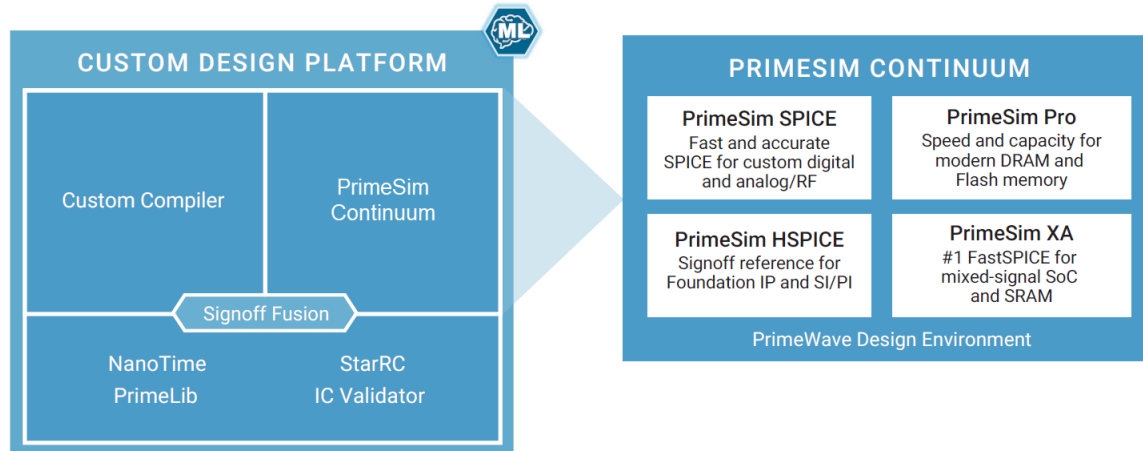
Figure 1 PrimeSim Continuum Overview



Simulation Engines in the PrimeSim Continuum

Figure 2 illustrates the components of the PrimeSim Continuum.

Figure 2 PrimeSim Continuum Components



The PrimeSim Continuum includes the following state-of-the-art SPICE and FastSPICE simulation engines, all available through a common token-based licensing model:

- [PrimeSim HSPICE](#)
- [PrimeSim Pro](#)
- [PrimeSim SPICE](#)
- [PrimeSim XA](#)

PrimeSim HSPICE

The PrimeSim HSPICE tool serves as the 'golden accuracy' cornerstone of the PrimeSim Continuum and is seamlessly integrated with and empowered by the other simulation engines in the PrimeSim Continuum. The PrimeSim HSPICE tool is the industry's 'gold standard' for accurate circuit simulation and offers foundries-certified MOS device models with state-of-the-art simulation and analysis algorithms. With extensive usage in chip, package, board, and backplane signal integrity simulation, cell and memory characterization, and analog mixed-signal IC design, the PrimeSim HSPICE tool is the industry's most popular, trusted, and comprehensive circuit simulator.

As chip and board speeds continue to increase, new design and verification challenges emerge. The PrimeSim HSPICE tool reveals signal integrity problems caused by jitter, crosstalk, ringing, ground bounce, and other noise sources. With extensive model and element support, the PrimeSim HSPICE tool is the ideal simulator that can satisfy your silicon-to-package-to-board-to-backplane SI simulation needs.

PrimeSim Pro

The PrimeSim Pro tool represents a next-generation FastSPICE architecture for fast and high-capacity analysis of modern DRAM and Flash memory designs. The new architecture is highly optimized to handle full-chip memory and image designs with high bandwidth, large power distributed networks and strict density requirements. Additionally, the PrimeSim Pro tool's unique heterogeneous architecture enables GPU simulation for even faster performance.

The FastSPICE engine in the PrimeSim Pro tool has several distinctive features that break down barriers commonly found in mixed-signal SoC simulations. The PrimeSim Pro tool takes advantage of improved circuit partitioning algorithms that solve complex topologies in all aspects of mixed-signal design, including custom, analog, and memory designs. The PrimeSim Pro tool provides improvements in simulation speed and design capacity by using multi-CPU and GPU technology for large custom blocks in FastSPICE.

PrimeSim SPICE

The PrimeSim SPICE tool is a high-performance SPICE circuit simulator for analog, RF, and mixed-signal applications. The PrimeSim SPICE tool offers a unique multicore and multimachine capability that delivers highly scalable performance with sign-off accuracy, as well as a state-of-the-art engine leveraging GPU technology to further increase performance. Additionally, the PrimeSim SPICE tool supports efficient high frequency noise analysis and S-Parameter handling, while offering advanced analysis capabilities for periodic and non-periodic time-domain and frequency-domain applications.

The PrimeSim SPICE tool is a true SPICE engine that has a single matrix solver and unique capability of solving the SPICE matrix over multiple CPUs or machines and GPU cards (using multi-CPU and GPU technology) with linear scaling and true SPICE accuracy. This SPICE technology is well suited for sensitive analog blocks such as PLLs, ADCs, charge pumps, and other traditionally difficult circuits for SPICE.

PrimeSim XA

The PrimeSim XA tool is the leading FastSPICE simulator for SRAM, custom digital, and mixed-signal verification. The PrimeSim XA tool's breakthrough patented technologies deliver superior performance and capacity while maintaining SPICE accuracy. Advanced simulation intelligence automatically recognizes devices, topologies, and hierarchies and employs the most efficient techniques to accurately simulate any design type.

The PrimeSim XA tool can accurately simulate many different circuit types, including highly sensitive analog designs; it supports traditional SPICE algorithms such as: Newton-Rapson iteration, second-order integration, local truncation error control, and charge conservation. The PrimeSim XA tool supports several technologies to improve simulation performance while maintaining accuracy, such as specialized sparse solvers, preprocessing of linear systems, partitioning, multi-rate time step control, and table models. The net effect of these technologies is that the CPU time required (per element) decreases as netlists become larger, so that the PrimeSim XA tool is an order of

magnitude faster than a SPICE tool for small netlists (less than one thousand elements) and many orders of magnitude faster for large netlists (more than one million elements).

Synopsys Products Integrated With the PrimeSim Continuum

The PrimeSim Continuum is designed to integrate seamlessly with the following Synopsys tools:

- [PrimeWave Design Environment](#)
- [PrimeSim Reliability Analysis](#)
- [VCS PrimeSim AMS](#)

PrimeWave Design Environment

The PrimeSim Continuum simulators are fully integrated with the PrimeWave Design Environment, a new design environment and waveform viewer that encapsulates analysis and display requirements and significantly improves productivity.

The PrimeWave Design Environment combines simulation setup, analysis, waveform viewing, and postprocessing onto one unified platform. It provides native integration to all PrimeSim Continuum simulation engines and all the advanced capabilities available within them, from analog and RF analysis, Advanced Variation Analysis (AVA), Custom Fault analysis, Device Aging, to Electromigration and IR Drop analysis. The integrated waveform viewer offers fast loading, display scrolling, and zooming of very large waveform files as well as support for multiple waveform file formats.

PrimeSim Reliability Analysis

The PrimeSim Continuum simulators are key engines inside the PrimeSim Reliability Analysis suite, a set of comprehensive solutions (fault simulation, static circuit check, MOS aging, IR/EM, and variation) that analyzes the full spectrum of a product life cycle, from early-life to end-of-life.

The PrimeSim Reliability Analysis suite is a set of comprehensive solutions that not only cover the verification of the entire product lifetime, early-life to end-of-life, but also span the full range of design validation from pre-layout phase to the post-layout stage. The solution suite includes PrimeSim CCK (static circuit check), PrimeSim Custom Fault (custom fault simulation and analysis), PrimeSim AVA (Advanced Variation Analysis), PrimeSim MOSRA (device aging and analysis), PrimeSim EMIR (Electromigration and IR Drop analysis), and SPRES (Static Power-net Resistance).

VCS PrimeSim AMS

The PrimeSim and PrimeSim XA tools are tightly integrated into Synopsys' VCS digital simulator through a direct-kernel integration (DKI) which enables fast development of a

rich feature set and continuous innovation. This mixed-signal solution delivers the highest throughput by combining the fastest transistor-level engine and the fastest digital engine.

The VCS PrimeSim AMS tool offers a flexible use model that allows for a broad mixture of mixed-signal modeling languages and abstractions levels and is integrated with both the PrimeWave Design Environment for design and verification, as well as the Verdi AMS Debug Environment for advanced digital and mixed-signal debug.

2

Getting Started With the PrimeSim Continuum

Describes how to set up token-based licensing for the PrimeSim Continuum and where to find documentation for each of the simulation engines.

This chapter includes the following sections:

- [Configuring PrimeSim Continuum Token-Based License Mode](#)
- [PrimeSim Continuum Simulator Documentation](#)

Configuring PrimeSim Continuum Token-Based License Mode

While each of the simulators in the PrimeSim Continuum can be run in standalone mode, its innovative token-based license mode allows you to install and run any of the simulators using a shared pool of license tokens with different simulation engines and features consuming an associated number of tokens.

Use one of the following methods to enable PrimeSim Continuum token-based license mode:

- [Define the Shell Environment Variable PRIMESIM to 1](#)
- [Use the -primesim 1 Command-Line Option](#)

Note:

If the shell environment variable is set and the `-primesim` command-line option is used, the command-line option takes precedence and overrides the shell environment variable setting.

Define the Shell Environment Variable PRIMESIM to 1

In C-Shell:

```
setenv PRIMESIM 1
```

In Bash shell:

```
export PRIMESIM=1
```

For example, for the PrimeSim HSPICE tool:

```
% setenv PRIMESIM 1  
% hspice netlist.sp
```

Or for the PrimeSim Pro tool:

```
% setenv PRIMESIM 1  
% primesim netlist.sp
```

Or for the PrimeSim SPICE tool:

```
% setenv PRIMESIM 1  
% primesim -spice netlist.sp
```

Or for the PrimeSim XA tool:

```
% setenv PRIMESIM 1  
% xa netlist.sp
```

Use the -primesim 1 Command-Line Option

For example, for the PrimeSim HSPICE tool:

```
% hspice netlist.sp -primesim 1
```

Or for the PrimeSim Pro tool:

```
% primesim netlist.sp -primesim 1
```

Or for the PrimeSim SPICE tool:

```
% primesim -spice netlist.sp -primesim 1
```

Or for the PrimeSim XA tool:

```
% xa netlist.sp -primesim 1
```

PrimeSim Continuum Token-Based License Mode Environment Variables

Table 1 lists the shell environment variables that control the token-based license mode and license queuing.

Table 1 Variables for Controlling Token-Based License Mode and License Queuing

Environment Variable	Description
<code>PRIMESIM 0 1 2</code>	This variable enables PrimeSim Continuum token-based license mode. Valid values: 0: Disables token-based license mode. 1: Enables token-based license mode. 2: (Default) Enables standalone and token-based license mode. By default, the standalone keys are attempted first (see <code>PRIMESIM_ORDER</code>). This variable can also be enabled via the <code>-primesim</code> command-line option.
<code>PRIMESIM_ORDER 0 1</code>	This variable controls whether standalone keys or token-based keys are attempted first when the <code>PRIMESIM</code> variable is set to 2. 0: (Default) Standalone keys are attempted first. 1: Token-based keys are attempted first.
<code>PRIMESIM_WAIT_LICENSE 0 1</code>	This variable enables and disables license queuing. 0: Disables license queuing. 1: (Default) Enables license queuing.
<code>PRIMESIM_WAIT_LICENSE_TIMEOUT value</code>	This variable controls the timeout period for license queuing. The value is in minutes and cannot exceed 43,200 minutes (30 days). The default period is 2 minutes. Non-integer values or negative integer values will result in an error. The following example sets the queue timeout period to 30 minutes: <pre>% setenv PRIMESIM_WAIT_LICENSE_TIMEOUT 30</pre>

PrimeSim Continuum Simulator Documentation

Detailed information about each simulator is beyond the scope of this *PrimeSim Continuum Overview* document. To open HTML-based versions of each simulator's complete documentation set, run each tool using the `-webhelp` option.

Note:

PDF versions of every manual are also included and accessible through the online Help.

Additionally, these documentation sets are available online on SolvNetPlus (<https://solvnetplus.synopsys.com>) and as downloadable documentation packages that can be accessed without an Internet connection after being installed locally.

Each simulator has its own documentation set, although many documents are common to all. For detailed information on how to get started with each simulator in the PrimeSim Continuum, see the following:

- PrimeSim HSPICE Documentation
 - *PrimeSim™ HSPICE® Quick Start Guide*
 - *PrimeSim™ HSPICE® User Guide: Basic Simulation and Analysis*
- PrimeSim Pro and PrimeSim SPICE Documentation
 - *PrimeSim™ User Guide: Pro and SPICE*
- PrimeSim XA Documentation
 - *PrimeSim™ XA User Guide*

3

Parameters and Functions

Describes how to use parameters within PrimeSim HSPICE-compatible netlists.

Parameters are similar to the variables used in most programming languages. Parameters hold a value that you assign when you create your circuit design or that the simulation calculates based on circuit solution values. Parameters can store static values for a variety of quantities (resistance, source voltage, rise time, and so on). You can also use them in sweep or statistical analysis.

The PrimeSim Continuum solution ships numerous examples for your use; see “Listing of Demonstration Input Files” in the *PrimeSim™ HSPICE® User Guide: Demonstration Netlists* for paths to the demo files.

For descriptions of individual commands referenced in this chapter, see “PrimeSim HSPICE Netlist Commands” in the *PrimeSim™ Continuum Reference Manual: Commands and Control Options*.

The following sections discuss these topics:

- [Using Parameters in Simulation \(.PARAM\)](#)
- [Using Algebraic Expressions](#)
- [Built-In Functions and Variables](#)
- [External Table Parameter File Interface](#)
- [Parameter Scoping and Passing](#)

Using Parameters in Simulation (.PARAM)

The following sections discuss these topics:

- [Defining Parameters](#)
- [Assigning Parameters](#)
- [User-Defined Function Parameters](#)
- [Predefined Analysis Function](#)

- [Measurement Parameters](#)
- [.PRINT and .PROBE Parameters](#)
- [Multiply Parameter](#)

Defining Parameters

Parameters in the PrimeSim Continuum solution are names that you associate with numeric values. (See [Assigning Parameters](#).) You can use any of the methods described in [Table 2](#) to define parameters.

Note:

A `.PARAM` statement with no definition is illegal.

Method	Description
Simple assignment	<code>.PARAM SimpleParam=1e-12</code>
Algebraic definition	<code>.PARAM AlgebraicParam='SimpleParam*8.2'</code> • <code>SimpleParam</code> excludes the output variable. • You can also use algebraic parameters in <code>.PRINT</code> and <code>.PROBE</code> statements. For example: <code>.PRINT AlgebraicParam=par('algebraic expression')</code> • You can use the same syntax for <code>.PROBE</code> statements. See Using Algebraic Expressions.
User-defined function	<code>.PARAM MyFunc(x, y)='Sqrt((x*x)+(y*y))'</code>
Character string definition	<code>.PARAM paramname=str('string')</code>
Subcircuit default	<code>.SUBCKT SubName ParamDefName=Value str('string')</code> <code>.MACRO SubName ParamDefName=Value str('string')</code>
Subcircuit cell instance	<code>XYZ NodeName1 ... NodeNamen</code> <code>+ SubName</code> <code>+ ParamDefName=Value str('string')</code>
Predefined analysis function	<code>.PARAM mcVar=AGAUSS(1.0,0.1)</code>
.MEASURE statement	<code>.MEASURE [DC AC TRAN] result TRIG ...</code> <code>+ TARG ... [GOAL=val] [MINVAL=val]</code> <code>+ [WEIGHT=val] [MeasType] [MeasParam]</code> See “Specifying User-Defined Analysis (.MEASURE)” in the <i>PrimeSim™ HSPICE® User Guide: Basic Simulation and Analysis</i> for more information.
.PRINT .PROBE	<code>.PRINT .PROBE</code> <code>+ outParam=Par_Expression</code>

A parameter definition in the PrimeSim Continuum solution always uses the last value found in the input netlist (subject to local versus global parameter rules). These definitions assign a value of 3 to the `DupParam` parameter.

```
.PARAM DupParam=1
...
.PARAM DupParam=3
```

The PrimeSim Continuum solution assigns 3 as the value for all instances of `DupParam`, including instances that are earlier in the input than the `.PARAM DupParam=3` statement.

All parameter values in the PrimeSim Continuum solution are IEEE double floating point numbers. The parameter resolution order is:

1. Resolve all literal assignments.
2. Resolve all expressions.
3. Resolve all function calls.

Table 2 shows the parameter passing order, from highest to lowest.

Table 2 *Parameter Passing Order*

.OPTION PARHIER=GLOBAL	.OPTION PARHIER=LOCAL
Analysis sweep parameters	Analysis sweep parameters
.PARAM statement (library)	.SUBCKT call (instance)
.SUBCKT call (instance)	.SUBCKT definition (symbol)
.SUBCKT definition (symbol)	.PARAM statement (library)

Assigning Parameters

You can assign the following types of values to parameters:

- Constant real number
- Algebraic expression of real values
- Predefined function
- User-defined function
- Circuit value
- Model value

To invoke the algebraic processor, enclose a complex expression in single quotation marks. A simple expression consists of one parameter name.

The parameter keeps the assigned value, unless:

- A later definition changes its value, or
- An algebraic expression assigns a new value during simulation. The PrimeSim Continuum solution does not warn you if it reassigns a parameter.

Note:

Parameters you define using `.PARAM` statements are constants; they cannot vary as a function of simulation time. However, you can use elements such as resistors and capacitors whose values can vary as a function of time by using the keyword `TIME`, that is,

```
Ctime 1 2 q='1p*time'
```

Example: Modeling an eFuse

You can model an electrically programmable eFUSE device as follows. Instantiate an eFUSE as a subcircuit and pass a parameter that determines whether the eFUSE is “blown” or intact:

```
.subckt efuse in out blown=0
Rfuse in out r='2*(1-blown)+100e6*blown'
.ends efuse
```

If `blown=0`, then the fuse is intact (2 ohms). If `blown=1` then the fuse is blown and you get the much higher resistance of 100 meg. To use the eFUSE, instantiate it with a subcircuit call:

```
xefuse1 in out efuse blown=0
```

Alternatively, you can control the eFUSE with a parameter setting:

```
.param blown=1
x1 in out efuse
```

Inline Parameter Assignments

To define circuit values, use a direct algebraic evaluation:

```
r1 n1 0 R='1k/sqrt(HERTZ)' $ Resistance for frequency
```

Parameters in Output

To use an algebraic expression as an output variable in a `.PRINT`, `.PROBE` or `.MEASURE` statement, use the `PAR` keyword. In addition to using quotation marks, you must define the parameter inside the `PAR('...')` statement for output. The PrimeSim Continuum simulator

issues an error message if the parentheses do not surround the quotation marks or if you use the keyword *incorrectly* as in `PAR=` instead of `PAR('...')`.

Example

```
.PRINT DC v(3) gain=PAR('v(3)/v(2)') PAR('v(4)/v(2)')
```

Reusing the PAR(...) Output as Input to Other Elements

You can use the `par(...)` output as the input voltage to another elements. For example:

```
.print tran v(5) par('5*cos(6.28*v(10)*v(5)*k/360)')
```

You can use the signal either as the input to another element in the same simulation (see Example 1), or you can save the signal and use it as input in another simulation (see Examples 2 and 3). You can use the definitions in print or probe output statements only in output statements. They cannot be referred by any other definitions. For example, you can use the E-element:

Example 1:

```
e1 in 0 vol='5*cos(6.28*v(10)*v(5)*k/360)'
```

Example 2: Then, you can use the node `in` as the input to the other element in the same netlist, as shown in this example.

```
M1 dr in src subr pch w=2u l=1u
.subckt inv vdd 0 A B
  M1 A B vdd vdd pch w=6u l=1u
  M2 A B 0 0 nch w=3u l=1u
.ends inv
x1 vdd 0 in out inv
```

Example 3: You can use `.STIM` statements to save the signal from the first simulation to create a PWL source.

```
.stim tran pw1 filename=test1 vsrc[0]=v(in) node1=A node2=B from=0.0ns
to=10ns
+ npoints=100
.stim tran pw1 filename=test2 vsrc[1]=v(in) node1=C node2=D from=0.0ns
to=10ns
+ npoints=50
```

In this example, the `.STIM` command creates two stimulus files named `test1.pw10_tr0` and `test2.pw12_tr0`. Each has a voltage source: one named `vsrc[0]`, applied between nodes A and B, and one named `vsrc[1]`, applied between nodes C and D. The stimulus files have a PWL source function based on the voltage of node `in` during the time `0.0ns`

to 10ns with 50 points (for `vsrc[0]`) and 100 points (for `vsrc[1]`). Contents of the `test1.pwl0_tr0` file:

```
vsrc[0] A B PWL
+ 0.          5.0000
+ 200.00000p  2.2114
+ 400.00000p  2.4666
+ 600.00000p -362.1421m
.....
```

Contents of the `test2.pwl1_tr0` file:

```
vsrc[1] C D PWL
+ 0.          5.0000
+ 100.00000p -1.8008
+ 200.00000p -3.2748
+ 300.00000p -1.3264
.....
```

You can use the PWL files generated from the `.STIM` commands as inputs to another simulation.

Load the signal in WaveView and then export the (x,y) data of the signal in text format by selecting **File > Save > Plotfiles**. You must edit the data so it looks like a PWL source by adding a source definition and line continuation characters.

User-Defined Function Parameters

You can define a function that is similar to the parameter assignment, but you cannot nest the functions more than two deep.

- An expression can contain parameters that you did not define.
- A function must have at least one argument, and can have up to 20 (and in many cases, more than 20) arguments.
- You can redefine functions.

The format of a function is:

```
funcname1(arg1[,arg2...])=expression1
+ [funcname2(arg1[,arg2...])=expression2] off
```

Parameter	Description
<i>funcname</i>	Specifies the function name. This parameter must be distinct from array names and built-in functions. In subsequently defined functions, you must have previously defined all embedded functions.
<i>arg1, arg2</i>	Specifies variables used in the expression.

Parameter	Description
off	Voids all user-defined functions.

Example

```
.PARAM f(a,b)=POW(a,2)+a*b g(d)=SQRT(d)
+ h(e)=e*f(1,2)-g(3)
```

Using Parameter Functions to Evaluate Expressions Containing Dynamic Signals

When you use a parameter function to evaluate expression that contain dynamic signals, you must provide a user-defined function. Otherwise, you get unexpected results and the tool generates a warning message.

For example, in the following netlist, a node-voltage (dynamic signal) is evaluated one time at $t=0$ and keeps this value for the entire simulation. Because the value is only evaluated at the beginning of the simulation and not at each time step, the results are not as expected.

```
* Test Case
VIN D 0 pwl 0 1 10n 5
.param aaa='v(D)-v(0)'
.tran 0.1n 20n
vdd vdd 0 5
R1 vdd 0 'v(D) - v(0)'
R2 vdd 0 'aaa'
.options post
.probe i(*)
.tran 0.1n 20n
.end
```

If you replace the constant parameter assignment with user-defined parameter functions, the function `.param aaa(x)='x'` causes the PrimeSim Continuum simulator to evaluate the parameter at each time point during the simulation to give you the expected results.

```
* Test Case
VIN D 0 pwl 0 1 10n 5
.param aaa(x)='x'
.tran 0.1n 20n
vdd vdd 0 5
R1 vdd 0 'V(D) - V(0)'
R2 vdd 0 'aaa(V(D,0))'
.options post
.probe i(*)
.end
```

Predefined Analysis Function

PrimeSim Continuum includes specialized analysis types, such as Optimization and Monte Carlo, that require a way to control the analysis.

Measurement Parameters

`.MEASURE` statements produce a *measurement* parameter. The rules for measurement parameters are the same as for standard parameters, except that a `.MEASURE` statement defines measurement parameters, but a `.PARAM` statement does not define them.

For a description of the `.MEASURE` statement, see “Specifying User-Defined Analysis (.MEASURE)” in the *PrimeSim™ HSPICE® User Guide: Basic Simulation and Analysis*.

.PRINT and .PROBE Parameters

`.PRINT` and `.PROBE` statements in the PrimeSim Continuum solution produce a *print* parameter. The rules for print parameters are the same as the rules for standard parameters, except that you define the parameter directly in a `.PRINT` or `.PROBE` statement, not in a `.PARAM` statement.

For more information about the `.PRINT` or `.PROBE` statements, see “Displaying Simulation Results” in the *PrimeSim™ HSPICE® User Guide: Basic Simulation and Analysis*.

Multiply Parameter

The most basic subcircuit parameter in PrimeSim Continuum is the M (multiply) parameter.

For a description of this parameter, see “M (Multiply) Parameter” in the *PrimeSim™ HSPICE® User Guide: Basic Simulation and Analysis*.

Using Algebraic Expressions

Note:

The PrimeSim Continuum solution uses double-precision numbers (15 digits) for expressions, user-defined parameters, and sweep variables.

In the PrimeSim Continuum solution, an algebraic expression with quoted strings can replace any parameter in the netlist. You can then use these expressions as output variables in `.PRINT` statements. Algebraic expressions can expand your options in an input netlist file.

Some uses of algebraic expressions are:

- Parameters:

```
.PARAM x='y+3'
```

- Functions:

```
.PARAM rho(leff,weff)='2+*leff*weff-2u'
```

- Algebra in elements:

```
R1 1 0 r='ABS(v(1)/i(m1))+10'
```

- Algebra in .MEASURE statements:

```
.MEAS vmax MAX V(1)
.MEAS imax MAX I(q2)
.MEAS ivmax PARAM='vmax*imax'
```

- Algebra in output statements:

```
.PRINT conductance=PAR('i(m1)/v(22)')
```

Output expressions require the following basic syntax for algebraic expressions:

```
PAR('algebraic expression')
```

In addition to using quotation marks, you must define the expression inside the `PAR('...')` statement for output. The PrimeSim Continuum simulator issues an error message if the parentheses do not surround the quotation marks. The continuation character for quoted parameter strings is a double backslash (`\\`).

Built-In Functions and Variables

In addition to simple arithmetic operations (+, -, *, and /), use the built-in functions listed in [Table 3](#) and the variables listed in [Table 4](#) in PrimeSim Continuum expressions.

Table 3 PrimeSim Continuum Built-in Functions

PrimeSim Continuum Form	Function	Class	Description
<code>sin(x)</code>	sine	trig	Returns the sine of x (radians)
<code>cos(x)</code>	cosine	trig	Returns the cosine of x (radians)
<code>tan(x)</code>	tangent	trig	Returns the tangent of x (radians)
<code>asin(x)</code>	arc sine	trig	Returns the inverse sine of x (radians)
<code>acos(x)</code>	arc cosine	trig	Returns the inverse cosine of x (radians)

Table 3 PrimeSim Continuum Built-in Functions (Continued)

PrimeSim Continuum Form	Function	Class	Description
<code>atan(x)</code>	arc tangent	trig	Returns the inverse tangent of x (radians)
<code>sinh(x)</code>	hyperbolic sine	trig	Returns the hyperbolic sine of x (radians)
<code>cosh(x)</code>	hyperbolic cosine	trig	Returns the hyperbolic cosine of x (radians)
<code>tanh(x)</code>	hyperbolic tangent	trig	Returns the hyperbolic tangent of x (radians)
<code>abs(x)</code>	absolute value	math	Returns the absolute value of x: $ x $
<code>sqrt(x)</code>	square root	math	Returns the square root of the absolute value of x: $\text{sqrt}(-x) = -\text{sqrt}(x)$
<code>pow(x, y)</code>	absolute power	math	Returns the value of x raised to the integer part of y: $x^{(\text{integer part of } y)}$
<code>pwr(x, y)</code>	signed power	math	Returns the absolute value of x, raised to the y power, with the sign of x: $(\text{sign of } x) x ^y$
<code>x**y</code>	power		- If $x < 0$, returns the value of x raised to the integer part of y. - If $x = 0$, returns 0. - If $x > 0$, returns the value of x raised to the y power.
<code>log(x)</code> <code>ln(x)</code>	natural logarithm	math	Returns the natural logarithm of the absolute value of x, with the sign of x: $(\text{sign of } x) \log(x)$
<code>log10(x)</code>	base 10 logarithm	math	Returns the base 10 logarithm of the absolute value of x, with the sign of x: $(\text{sign of } x) \log_{10}(x)$
<code>exp(x)</code>	exponential	math	Returns e, raised to the power x: e^x
<code>db(x)</code>	decibels	math	Returns the base 10 logarithm of the absolute value of x, multiplied by 20, with the sign of x: $(\text{sign of } x) 20 \log_{10}(x)$
<code>int(x)</code>	integer	math	Returns the integer portion of x (which ignores the fractional portion of the number).
<code>nint(x)</code>	integer	math	Rounds x up or down, to the nearest integer.

Table 3 *PrimeSim Continuum Built-in Functions (Continued)*

PrimeSim Continuum Form	Function	Class	Description
<code>sgn(x)</code> <code>sign(x)</code>	return sign	math	• Returns -1 if x is less than 0 • Returns 0 if x is equal to 0 • Returns 1 if x is greater than 0
<code>sign(x, y)</code>	transfer sign	math	Returns the absolute value of x, with the sign of y: (sign of y) x
<code>floor(x)</code>	integer	math	Rounds down to the nearest integer (ignores the fractional part of the number)
<code>ceil(x)</code>	integer	math	Rounds up to the nearest integer (ignores the fractional part of the number)
<code>def(x)</code>	parameter defined	control	• Returns 1 if parameter x is defined • Returns 0 if parameter x is not defined
<code>min(x, y)</code>	smaller of two args	control	Returns the numeric minimum of x and y
<code>mmin(x1, x2, x3, ..., xn)</code>	smallest of n args	control	Returns the numeric minimum of x1, x2, x3, ..., xn
<code>max(x, y)</code>	larger of two args	control	Returns the numeric maximum of x and y
<code>mmax(x1, x2, x3, ..., xn)</code>	largest of n args	control	Returns the numeric maximum of x1, x2, x3, ..., xn
<code>fmod(x, y)</code>	floating-point remainder	math	Returns the floating-point remainder of x/y
<code>val(element)</code>	get value	various	Returns a parameter value for a specified element For example, <code>val(r1)</code> returns the resistance value of the <i>r1</i> resistor.
<code>val(element.parameter)</code>	get value	various	Returns a value for a specified parameter of a specified element For example, <code>val(rlod.temp)</code> returns the value of the <i>temp</i> (temperature) parameter for the <i>rlod</i> element.

Table 3 PrimeSim Continuum Built-in Functions (Continued)

PrimeSim Continuum Form	Function	Class	Description
<code>val(model_type: model_name. model_param)</code>	get value	various	Returns a value for a specified parameter of a specified model of a specific type For example, <code>val(nmos:mos1.rs)</code> returns the value of the <code>rs</code> parameter for the <code>mos1</code> model, which is an NMOS model type. CML models (level 54 and greater) <i>do not</i> support this form. See “Measuring the Value of MOSFET Model Card Parameters” in the <i>PrimeSim™ Continuum Reference Manual: MOSFET Models</i> for an example and details.
<code>valm(elem_name. model_param)</code>	get value		Returns a value for a specified model parameter of a specified element For example, <code>valm(m1.vth0)</code> returns the value of <code>vth0</code> parameter of the model card that uses <code>m1</code> . <code>valm()</code> supports only <code>vth0</code> , <code>lmin</code> , <code>lmax</code> , <code>wmin</code> , <code>wmax</code> , <code>lref</code> , <code>wref</code> , <code>xl</code> , <code>dl</code> , <code>dell</code> , <code>xw</code> , <code>dw</code> , <code>delw</code> , <code>scalw</code> , <code>lmlt</code> , <code>wmlt</code> and level 54, level 57, and level 70. See “Measuring the Value of MOSFET Model Card Parameters” for an example and details.
<code>valp(parameter)</code>	get value		Returns a value for a specified parameter or measure result The parameter can only be a named parameter or measure result. An expression is not permitted. Note: You must use <code>valp()</code> when the parameter or measure result name contains "/" and they are used in expressions. See Examples of valp() .
<code>lv(Element) or lx(Element)</code>	element templates	various	Returns various element values during simulation See “Element Template Output (PrimeSim HSPICE Only)” in the <i>PrimeSim™ HSPICE® User Guide: Basic Simulation and Analysis</i> for more information.
<code>v(Node) , i(Element)...</code>	circuit output variables	various	Returns various circuit values during simulation See “DC and Transient Output Variables” in the <i>PrimeSim™ HSPICE® User Guide: Basic Simulation and Analysis</i> for more information.
<code>cond ? x : y</code>	ternary operator		Returns <code>x</code> if <code>cond</code> is not zero. Otherwise, returns <code>y</code> <code>.param z= 'condition ? x:y'</code>
<code><</code>	relational operator (less than)		Returns 1 if the left operand is less than the right operand. Otherwise, returns 0 <code>.para x=y<z (y less than z)</code>

Table 3 *PrimeSim Continuum Built-in Functions (Continued)*

PrimeSim Continuum Form	Function	Class	Description
<=	relational operator (less than or equal)		Returns 1 if the left operand is less than or equal to the right operand. Otherwise, returns 0 .para x=y<=z (y less than or equal to z)
>	relational operator (greater than)		Returns 1 if the left operand is greater than the right operand. Otherwise, returns 0 .para x=y>z (y greater than z)
>=	relational operator (greater than or equal)		Returns 1 if the left operand is greater than or equal to the right operand. Otherwise, returns 0 .para x=y>=z (y greater than or equal to z)
==	equality		Returns 1 if the operands are equal. Otherwise, returns 0 .para x=y==z (y equal to z)
!=	inequality		Returns 1 if the operands are not equal. Otherwise, returns 0 .para x=y!=z (y not equal to z)
&&	logical AND		Returns 1 if neither operand is zero. Otherwise, returns 0 .para x=y&&z (y AND z)
	logical OR		Returns 1 if either or both operands are not zero. Returns 0 only if both operands are zero .para x=y z (y OR z)
&	bitwise AND		Returns a number whose binary representation has a 1 in each bit position for which the corresponding bits of both operands are 1.
	bitwise OR		Returns a number whose binary representation has a 1 in each bit position for which the corresponding bits of either or both operands are 1.

Examples of valp()

The following example gets the parameter value defined in a subcircuit:

```
.meas tran asdf param='valp(x1/zzz.p1)'
```

The following example gets the measure result when the measure name contains "/" and is used in an expression:

```
.MEASURE TRAN
+ DELAY_RISE_X1/X2/X3/X4/X5/A
+ TRIG V(in) VAL='pwr_vdd * 0.5' RISE=LAST
```

```
+ TARG V(out) VAL ='pwr_vdd * 0.5' RISE=LAST

.MEASURE TRAN
+ DELAY_FALL_X1/X2/X3/X4/X5/A
+ TRIG V(in) VAL='pwr_vdd * 0.5' FALL=LAST
+ TARG V(out) VAL ='pwr_vdd * 0.5' FALL=LAST

.MEASURE TRAN
+ DCD_DELAY_X1/X2/X3/X4/X5/A
+ param = "valp(DELAY_RISE_X1/X2/X3/X4/X5/A) -
+ valp(DELAY_FALL_X1/X2/X3/X4/X5/A) "
```

Reserved Variable Names

The PrimeSim Continuum solution reserves the variable names listed in [Table 4](#) for use in elements, such as E, G, R, C, and L. You can use them in expressions, but you cannot redefine them; for example, this statement would be illegal:

```
.param temper=100
```

Table 4 PrimeSim Continuum Special Variables

PrimeSim Continuum Form	Function	Class	Description
time	current simulation time	control	Uses parameters to define the current simulation time, during transient analysis.
temper	current circuit temperature	control	Uses parameters to define the current simulation temperature, during transient/temperature analysis. You can use the PrimeSim Continuum simulation temperature in an equation by using the temper variable parameter. For example: .temp 20 50 100 .par x="temper/2" v0 1 0 1 r0 1 0 r=x
hertz	current simulation frequency	control	Uses parameters to define the frequency, during AC analysis.

External Table Parameter File Interface

Description

You can use the `table_param()` built-in function to specify a file that contains a parameter values table and the PrimeSim Continuum solution returns the linear interpolated value from that table.

Syntax

```
table_param(str('table_file_name'), disc_num, nd1, nd2,
..., cont_num, nc1, nc2, ..., value_col)
```

Arguments

Argument	Description
<code>str('table_file_name')</code>	Name of file containing a parameter values table Note: Specify the file name using the <code>str()</code> function.
<code>disc_num</code>	Total number of discrete from the first column
<code>nd1, nd2, ...</code>	Discrete values
<code>cont_num</code>	Number of continuous index to be considered for interpolation
<code>nc1, nc2, ...</code>	Continuous values
<code>value_col</code>	Continuous column value used in linear interpolation

Examples

The following example uses the `table_param()` built-in function to specify the `cap.table` file that contains a parameter values table defined as follows:

```
Corner    layer    width    space    Cbot    Ctop    Ccpl
1         3        0.0450   0.030    0.078901 0.06789 0.03456
1         3        0.0450   0.032    0.089012 0.05678 0.02109
1         3        0.0450   0.037    0.090123 0.04567 0.01098
1         3        0.0470   0.030    0.079     0.06789 0.03456
1         3        0.0470   0.037    0.091     0.04567 0.01098
1         3        0.0470   0.032    0.085     0.05678 0.02109
```

```
.param cbot = table_param(str("cap.table"), 2, 1, 3, 2, 0.0465, 0.0307,
1)
```

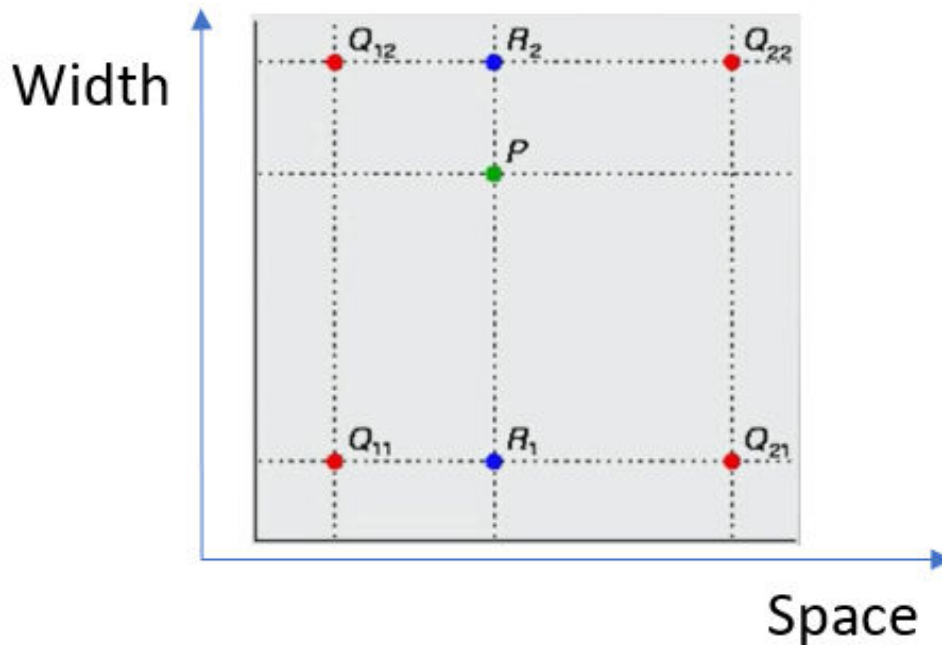
In this example, the `disc_num` function is 2. This means that the first two columns of the table, `Corner` and `layer`, are the integer index which should not be interpolated. They are considered as a discrete number and 1, 3 is the value of input.

The `cont_num` function is 2. This means that the next two columns of the table, `width` and `space`, must be considered for interpolation with the next two arguments, 0.0465 and 0.0307 (which is two-dimensional).

The `value_col` is 1. This means that the fifth column of the table, `Cbot`, is the final value column.

An example of return bilinear interpolation is shown in [Figure 3](#).

Figure 3 Return Bilinear Interpolation



The value of `Cbot` is 0.0814349625, which is calculated as follows.

1. Interpolate for space:

- Q11: (0.030, 0.045), Q21 (0.032, 0.045)
- R1: $0.078901 + (0.089012 - 0.078901) * (0.0307 - 0.030) / (0.032 - 0.030) = 0.08243985$
- Q12(0.030, 0.047), Q22(0.032, 0.047)
- R2: $0.079 + (0.085 - 0.079) * (0.0307 - 0.030) / (0.032 - 0.030) = 0.0811$

2. Interpolate for width

- P: $0.08243985 + (0.0811 - 0.08243985) * (0.0465 - 0.045) / (0.047 - 0.045) = 0.0814349625$

Note:

The `table_param()` function always chooses two points, which are the closest to your input value for each column to calculate the interpolated value.

Parameter Scoping and Passing

If you use parameters to define values in subcircuits, you need to create fewer similar cells, to provide enough functionality in your library. You can pass circuit parameters into hierarchical designs, and assign different values to the same parameter within individual cells, when you run simulation.

For example, if you use parameters to set the initial state of a latch in its subcircuit definition, then you can override this initial default in the instance call. You need to create only one cell, to handle both initial state versions of the latch.

You can also use parameters to define the cell layout. For example, you can use parameters in a MOS inverter, to simulate a range of inverter sizes, with only one cell definition. Local instances of the cell can assign different values to the size parameter for the inverter.

In the PrimeSim Continuum solution, you can also perform Monte Carlo analysis or optimization on a cell that uses parameters.

How you handle hierarchical parameters depends on how you construct and analyze your cells. You can construct a design in which information flows from the top of the design, down into the lowest hierarchical levels.

- To centralize the control at the top of the design hierarchy, set *global* parameters.
- To construct a library of small cells that are individually controlled from within, set *local* parameters and build up to the block level.

This section describes the scope of parameter names and how the PrimeSim Continuum solution resolves naming conflicts between levels of hierarchy, in the following topics:

- [Library Integrity](#)
- [Reusing Cells](#)
- [Creating Parameters in a Library](#)
- [String Parameters](#)
- [String Parameters in Passive and Active Component Keywords](#)
- [String Parameters in Subcircuit-based Macro Model Selection](#)
- [Parameter Defaults and Inheritance](#)
- [Parameter Passing Solutions](#)

Library Integrity

Integrity is a fundamental requirement for any symbol library. Library integrity can be as simple as a consistent, intuitive name scheme, or as complex as libraries with built-in range checking.

Library integrity might be poor if you use libraries from different vendors in a circuit design. Because vendors do not standardize names of circuit parameters, two components can include the same parameter name for different functions. For example, one vendor might build a library that uses the name `Tau` as a parameter to control one or more subcircuits in their library. Another vendor might use `Tau` to control a different aspect of their library. If you set a global parameter named `Tau` to control one library, you also modify the behavior of the second library, which might not be the intent.

If the scope of a higher-level parameter is global to all subcircuits at lower levels of the design hierarchy, higher-level definitions override lower-level parameter values with the same names. The scope of a lower-level parameter is local to the subcircuit where you define the parameter (but global to all subcircuits that are even lower in the design hierarchy). Local scoping rules in the PrimeSim Continuum solution prevent higher-level parameters from overriding lower-level parameters of the same name.

Reusing Cells

Parameter name problems also occur if different groups collaborate on a design. Global parameters prevail over local parameters, so all circuit designers must learn the names of all parameters, even those used in sections of the design for which they are not responsible. This can lead to a large investment in standard libraries. To avoid this situation, use local parameter scoping, to encapsulate all information about a section of a design, within that section.

Creating Parameters in a Library

To ensure that the input netlist includes critical, user-supplied parameters when you run simulation, you can use “illegal defaults”—that is, defaults that cause the simulator to end if you do not supply overrides for the defaults.

If a library cell includes illegal defaults, you must provide a value for each instance of those cells. If you do not, the simulation ends.

For example, you might define a default MOSFET width of 0.0. The PrimeSim Continuum simulation ends because MOSFET models require this parameter.

Example 1

In this example, the simulation ends on the `x1` subcircuit instance because you never set the required `wid` parameter on the subcircuit instance line. The `x2` subcircuit simulates

correctly. Additionally, the instances of the `Inv` cell are subject to accidental interference because of exposure of the `Wid` global parameter outside the domain of the library. Anyone can specify an alternative value for the parameter, in another section of the library or the circuit design. This might prevent the simulation from catching the condition on `x1`.

```
* Subcircuit default definition
.SUBCKT Inv A Y Wid=0 $ Inherit illegal values by default
mp1 NodeList Model L=1u W='Wid*2'
mn1 NodeList Model L=1u W=Wid
.ENDS

* Invoke symbols in a design
x1 A Y1 Inv          $ Bad! No widths specified
x2 A Y2 Inv Wid=1u   $ Overrides illegal value for Width
```

Example 2

In this example, the name of a global parameter conflicts with the internal library parameter named `Wid`. Another user might specify such a global parameter, in a different library. In this example, the user of the library has specified a different meaning for the `Wid` parameter, to define an independent source.

```
.Param Wid=5u          $ Default Pulse Width for source
v1 Pulsed 0 Pulse ( 0v 5v 0u 0.1u 0.1u Wid 10u )
...
* Subcircuit default definition
.SUBCKT Inv A Y Wid=0    $ Inherit illegals by default
mp1 NodeList Model L=1u W='Wid*2'
mn1 NodeList Model L=1u W=Wid
.ENDS
* Invoke symbols in a design
x1 A Y1 Inv            $ Incorrect width!
x2 A Y2 Inv Wid=1u     $ Incorrect! Both x1 and x2
$ simulate with mp1=10u and
$ mn1=5u instead of 2u and 1u.
```

Under global parameter scoping rules, simulation succeeds, but incorrectly. The PrimeSim Continuum solution does not warn that the `x1` inverter has no assigned width because the global parameter definition for `Wid` overrides the subcircuit default.

Note:

Similarly, sweeping with different values of `Wid` dynamically changes both the `Wid` library internal parameter value, and the pulse width value to the `Wid` value of the current sweep.

In global scoping, the highest-level name prevails, when resolving name conflicts. Local scoping uses the lowest-level name.

When you use the parameter inheritance method, you can specify to use local scoping rules.

When you use local scoping rules, the Example 2 netlist correctly ends in x1 for W=0. (The default Wid=0, in the `.SUBCKT` definition, has higher precedence, than the `.PARAM` statement.) This results in the correct device sizes for x2. This change can affect your simulation results, if you intentionally or accidentally create a circuit such as the second one shown previously.

You can use an alternative to the test of the width in the Example 2 netlist: Use `.OPTION DEFW` to achieve a limited version of library integrity. This option sets the default width for all MOS devices during a simulation. Part of the definition is still in the top-level circuit, so this method can still make unwanted changes to library values, without notification from the PrimeSim Continuum simulator.

Table 5 compares the three primary methods to configure libraries, to achieve required parameter checks for default MOS transistor widths.

Table 5 *Methods for Configuring Libraries*

Method	Parameter Location	Pros	Cons
Local	On a <code>.SUBCKT</code> definition line	Protects library from global circuit parameter definitions, unless you override it. Single location for default values.	
Global	At the global level and on <code>.SUBCKT</code> definition lines	Works with all PrimeSim Continuum versions	An indiscreet user, another vendor assignment, or the intervening hierarchy can change the library. Cannot override a global value at a lower level.
Special	<code>.OPTION DEFW</code> statement	Simple to do	Third-party libraries, or other sections of the design, might depend on <code>.OPTION DEFW</code>

String Parameters

The PrimeSim Continuum solution uses a special delimiter `str()` to identify string and double parameter types. The single quotation marks (`'`), double quotation marks (`"`), or curly brackets (`{ }`) do not work for these parameter types.

String Parameter Definition

To define a string parameter, use the following syntax:

```
string_param_name=str('string')
```

The string within `str()` has quotation marks and that means that the statement is a string parameter definition.

For example:

```
rft1 1 0 pdmi_model1 l=15u w=0.1u devtype=str('xyz1')
cft2 1 0 pdmi_model2 l=16u w=0.14u devtype=str('xyz2')
```

Reference to a String Parameter

Use the following syntax:

```
new_parameter_name=str(string_parameter_name)
```

In this case, the string within `str()` has no quotation marks and this means that the statement refers to another previously-defined string parameter.

For example:

```
.param rdevtype=str('xyz1') cdevtype=str('xyz2')
rft1 1 0 pdmi_model1 l=15u w=0.1u devtype=str(rdevtype)
cft2 1 0 pdmi_model2 l=16u w=0.14u devtype=str(cdevtype)
```

Netlist Example

The following example netlist shows how you can use these definitions for various commands, keywords, parameters, and elements:

```
xibis1 vccq vss out in IBIS
+ IBIS_FILE=str('file1.ibs') IBIS_MODEL=str('model1')
xibis2 vccq vss out in IBIS
+ IBIS_FILE=str('file2.ibs') IBIS_MODEL=str('model2')

.subckt IBIS vccq vss out in
+ IBIS_FILE=str('file.ibs')
+ IBIS_MODEL=str('ibis_model')
ven en 0 vcc
BMCH vccq vss out in en v0dq0 vccq vss buffer=3
+ file= str(IBIS_FILE) model=str(IBIS_MODEL)
+ typ=typ ramp_rwf=2 ramp_fwf=2 power=on
.ends
```

The PrimeSim Continuum solution supports the following definitions and instances with their respective netlist components:

- **.PARAM statements**
- **.SUBCKT statements**
- **S-parameter FQMODEL** in both the S-parameter instance and S-parameter model
- **S-element keyword:** TSTONEFILE
- **E-element and G-element keyword:** NOISEFILE
- **V-element and I-element keyword:** PWLFILE

- `FILE` and `MODEL` keywords
- `B`-elements
- `W`-element keywords: `RLGCFEIL`, `RLGCMODEL`, `UMODEL`, `FSMODEL`, `TABLEMODEL`, and `SMODEL`
- `.IF` and `.ELSEIF` conditions
- `.LIN` command keywords: `FILENAME` and `MODELNAME`
- `.IBIS` statement keyword: `MOD_SEL`

String Parameters in Passive and Active Component Keywords

You can include string parameters in all PrimeSim Continuum solution passive and active component model name keywords. When you define a parameter that is a character string, use the keyword `str('string')`. When you use an instance of the parameter, the parameter name is `str(parameter_name)`.

Syntax

For passive elements:

```
Rxxx n1 n2 [mname [str(mname)]] Rval [TC1 [TC2][TC]] [SCALE=val]
+ [M=val] [AC=val] [DTEMP=val] [L=val] [W=val] [C=val]
+ [NOISE = val]
Cxxx n1 n2 [mname [str(mname)]] [C = ]capacitance [[TC1 = ]val]
+ [[TC2 = ]val] [SCALE = val] [IC = val] [M = val]
+ [W = val] [L = val] [DTEMP = val]
Lxxx n1 n2 [L = ]inductance [mname [str(mname)]] [[TC1 = ]val]
+ [[TC2 = ]val] [SCALE = val] [IC = val] [M = val]
+ [DTEMP = val] [R = val]
```

For active elements, define the model name by using the original syntax, or string parameter model-name syntax. (See the *PrimeSim™ Continuum Reference Manual: MOSFET Models* for full listing of transistor parameters that can be overridden in transistor definitions.)

```
Dxxx nplus nminus str(mname) [[AREA = ]area] [[PJ = ]val]
+ [WP = val] [LP = val] [WM = val] [LM = val] [OFF]
+ [IC = vd] [M = val] [DTEMP = val]
Qxxx nc nb ne [ns] str(mname) [area] [OFF]
+ [IC = vbeval,vceval] [M = val] [DTEMP = val]
Jxxx nd ng ns [nb] str(mname) [[[[AREA] = area | [W = val]
+ [L = val]]] [OFF] [IC = vdsval,vgsval] [M = val]
+ [DTEMP = val]
Mxxx nd ng ns [nb] str(mname) [[L = ]length] [[W = ]width]
+ [AD = val] AS = val] [PD = val] [PS = val]
+ [NRD = val] [NRS = val] [RDC = val] [RSC = val] [OFF]
```

```
+ [IC = vds,vgs,vbs] [M = val] [DTEMP = val]
+ [GEO = val] [DELVTO = val]
```

Example

```
.param mypmos=str('p')
.param mynmos=str('n')
.lib 'ltst.lib' TT

.subckt circuit vout vin vdd nmod=str('nch')pmod=str('pch')
m1 vout vin vdd vdd str(pmod) w=4u l=5u
m2 vout vin 0 0 str(nmod) w=2u l=5u
.ends circuit

x1 vout vin vdd circuit dtemp=11 nmod=str(mynmos)pmod=str(mypmos)
```

String Parameters in Subcircuit-based Macro Model Selection

You can include string parameters in a PrimeSim Continuum solution subcircuit-based macro model to select a different model name. When you use an instance of the macro model name, the name is `str(model_name)`.

Syntax

For subcircuit-based macro model instances:

```
XMxxx nd ng ns [nb] str(mname) [[L = ]length] [[W = ]width]
+ [AD = val] AS = val] [PD = val] [PS = val]
+ [NRD = val] [NRS = val] [RDC = val] [RSC = val] [OFF]
+ [IC = vds,vgs,vbs] [M = val] [DTEMP = val]
+ [GEO = val] [DELVTO = val]
```

Example

To select the HV macro MOSFET models of `nch_mac_hv` and `pch_mac_hv`, instead of the normal macro MOSFET models of `nch_mac` and `pch_mac`:

```
.lib 'ltst.lib' TT_mac
.subckt circuit vout vin vdd
+ nmod=str('nch_mac')pmod=str('pch_mac')
xm1 vout vin vdd vdd str(pmod) w=4u l=5u
xm2 vout vin 0 0 str(nmod) w=2u l=5u
.ends circuit
x1 vout vin vdd circuit dtemp=11
+ nmod=str('nch_mac_hv')pmod=str('pch_mac_hv')
```

Parameter Defaults and Inheritance

Use the `.OPTION PARHIER` parameter to specify scoping rules.

Syntax

```
.OPTION PARHIER=[GLOBAL | LOCAL]
```

The default setting is GLOBAL.

Example

This example explicitly shows the difference between local and global scoping for using parameters in subcircuits.

The input netlist includes the following:

```
.OPTION parhier=[global | local]
.PARAM DefPwid=1u
.SUBCKT Inv a y DefPwid=2u DefNwid=1u
Mpl MosPinList pMosMod L=1.2u W=DefPwid
Mnl MosPinList nMosMod L=1.2u W=DefNwid
.ENDS
```

Set the .OPTION PARHIER=parameter scoping option to GLOBAL. The netlist also includes the following input statements:

```
xInv0 a y0 Inv          $ override DefPwid default,
$ xInv0.Mpl width=1u
xInv1 a y1 Inv DefPwid=5u $ override DefPwid=5u,
$ xInv1.Mpl width=1u

.measure tran Wid0 param='lv2(xInv0.Mpl)' $ lv2 is the
           $ template for
.measure tran Wid1 param='lv2(xInv1.Mpl)' $ the channel
           $ width
           $ 'lv2(xInv1.Mpl)'
.ENDS
```

Simulating this netlist produces the following results in the listing file:

```
wid0=1.0000E-06
wid1=1.0000E-06
```

If you change the .OPTION PARHIER=parameter scoping option to LOCAL:

```
xInv0 a y0 Inv          $ not override .param
           $ DefPwid=2u,
           $ xInv0.Mpl width=2u
xInv1 a y1 Inv DefPwid=5u      $ override .param
           $ DefPwid=2u,
           $ xInv1.Mpl width=5u:
.measure tran Wid0 param='lv2(xInv0.Mpl) '$ override the
.measure tran Wid1 param='lv2(xInv1.Mpl) '$ global .PARAM
...
```

Simulation produces the following results in the listing file:

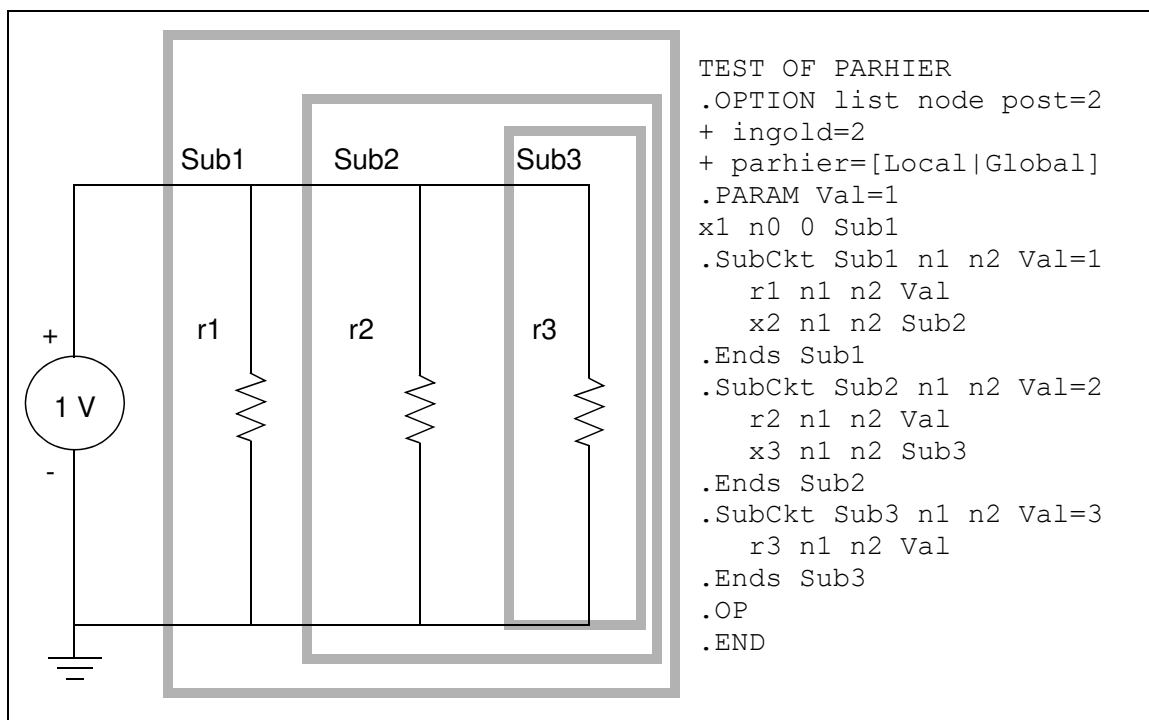
```
wid0=2.0000E-06
wid1=5.0000E-06
```

Parameter Passing

Figure 4 shows a flat representation of a hierarchical circuit, which contains three resistors.

Each of the three resistors obtains its simulation time resistance from the *Val* parameter. The netlist defines the *Val* parameter in four places, with three different values.

Figure 4 Hierarchical Parameter Passing Problem



The total resistance of the chain has two possible solutions: 0.3333Ω and 0.5455Ω .

You can use `.OPTION PARHIER` to specify which parameter value prevails, when you define parameters with the same name at different levels of the design hierarchy.

Under global scoping rules, if names conflict, the top-level assignment `.PARAM Val=1` overrides the subcircuit defaults, and the total is 0.3333Ω . Under local scoping rules, the lower level assignments prevail, and the total is 0.5455Ω (one, two, and three ohms in parallel).

The example in [Figure 4](#) produces the results in [Table 6](#), based on how you set `.OPTION PARHIER` to local or global:

Table 6 *PARHIER=LOCAL vs. PARHIER=GLOBAL Results*

Element	PARHIER=Local	PARHIER=Global
r1	1.0	1.0
r2	2.0	1.0
r3	3.0	1.0

Parameter Passing Solutions

The following checklist determines whether you see simulation differences when you use the default scoping rules. These checks are especially important if your netlists contain devices from multiple vendor libraries.

- Check your subcircuits for parameter defaults, on the `.SUBCKT` or `.MACRO` line.
- Check your subcircuits for a `.PARAM` statement, within a `.SUBCKT` definition.
- To check your circuits for global parameter definitions, use the `.PARAM` statement.
- If any of the names from the first three checks are identical, set up two PrimeSim Continuum solution simulation jobs: one with `.OPTION PARHIER=GLOBAL`, and one with `.OPTION PARHIER=LOCAL`. Then look for differences in the output.

4

Library and Data Encryption

Describes the Synopsys library encryption methods and their use to protect your intellectual property.

PrimeSim Continuum ships several suites of examples for your use; see “Listing of Demonstration Input Files” in the *PrimeSim™ HSPICE® User Guide: Demonstration Netlists* for paths to the demo files. For encryption demo files, see “Encryption Examples”.

This chapter presents the PrimeSim Continuum solution encryption methods in the following topics:

- [Library Encryption](#)
- [Encryption Guidelines](#)
- [Installing and Running metaencrypt](#)
- [metaencrypt Encryption Methods](#)
- [General Example](#)
- [Troubleshooting](#)

Library Encryption

You can encrypt your own proprietary PrimeSim Continuum solution custom models, parameters, and circuits and distribute to others without revealing your company’s sensitive information. Recipients of an encrypted library can run PrimeSim Continuum simulations and your libraries, so that encrypted parameters, encrypted circuit netlists, and internal node voltages do not appear in output files. Your library user sees the devices and circuits as black boxes that provide terminal functions only.

The following topic discusses the `metaencrypt` utility:

- [Encrypting a Model Library Using the metaencrypt Utility](#)

Encrypting a Model Library Using the metaencrypt Utility

A typical model library from a foundry has the following structure:

```
* model library mylib.lib
.lib tt
.param toxn=
...
.inc mymodel.mdl
.endl
```

If you encrypt both `mylib.lib` and `mymodel.mdl`, then you generate the error message: Command exited with non-zero status 1 during the PrimeSim Continuum solution simulation. This is because PrimeSim Continuum solution does not support the nesting of encrypted files.

To correctly encrypt the model file, you must change the library structure. The model parameters and the models need encryption separately as shown in the following steps:

1. The modified structure should be as follows:

```
. * model library mylib.lib
.lib tt
.inc myparam.par    $ put parameter definitions into myparam.par
.inc mymodel.mdl    $ original model file
.endl
```

2. Encrypt the parameter file, model file, and netlist as follows:

```
metaencrypt -i myparam.par -o myparam.par.enc -t randkey
metaencrypt -i mymodel.mdl -o mymodel.mdl.enc -t randkey
metaencrypt -i mynetlist.sp -o mynetlist.sp.enc -t randkey
```

3. To simulate the circuit, include the encrypted files and call the library file, `mylib.lib`, in the top-level netlist:

```
* top level netlist
.inc mynetlist.sp.enc
.lib mylib.lib tt
...
.end

* modified library mylib.lib
.lib tt
.inc myparam.par.enc
.inc mymodel.mdl.enc
.end
```

Encryption Guidelines

Before encrypting, you must test out any circuits and device parameters, as you are not able to see what is wrong after encryption because the PrimeSim Continuum solution does not let you read the encrypted data.

You can use any legal PrimeSim Continuum command inside subcircuits that you encrypt. Refer to “Using Subcircuits” in the *PrimeSim™ HSPICE® User Guide: Basic Simulation and Analysis* for more information about how to construct subcircuits. The structure of your libraries can affect how you encrypt them. If your library requires that you change the name of a subcircuit, you must encrypt that circuit again.

To encrypt more than one file in a directory, use the following shell script, which encrypts the files as a group. In this example, the script uses the traditional encryption method. The script produces an `.inc` encrypted file, for each `.dat` file in the current directory. The `metaencrypt` command assumes that unencrypted files have a `.dat` suffix.

```
#!/bin/sh
for i in *.dat
do
Base=`basename $i .dat`
metaencrypt -i $Base.dat -o $Base.inc -t Freelib
done
```

Use an encrypted file much the same way as you do before encryption. The name of the file may be different, so you may have to update the `.include` and `.lib` commands.

Note:

Verilog-A supports module files for encryption only when using 8-byte key and Triple DES private or random key. The Freelib (traditional encryption method) does not support Verilog-A due to limitations in the `metaencrypt` utility.

You can probe any specified encrypted nodes using `.OPTION PROBE`.

Installing and Running metaencrypt

This section describes how to install and run `metaencrypt`.

The following sections discuss these topics:

- [Installing metaencrypt](#)
- [Running metaencrypt](#)

Installing metaencrypt

The `metaencrypt` utility is part of the general PrimeSim HSPICE, PrimeSim XA, and PrimeSim distribution and found in the `$installdir/bin` directory.

If you have not installed any of these simulators on your system, first install them according to the *Installation Guide* and the *Release Notes*. Verify that the license file contains the license token `encrypt`.

Running metaencrypt

Use the following syntax to run `metaencrypt`:

```
metaencrypt -i input_file|-pipe -o encrypted_output_file
+ -i input_file -o encrypted_output_file
+ -t encrypt_type [-p|-d encrypt_dir] [-expire_date YYYYMMDD] [-z]]
+ [-r -r synopsys_tool[:access_control]]
+ [-r -r synopsys_tool[:access_control]]...
```

Argument	Description
<code>-i inptfileName</code>	Unencrypted input filename. Use the <code>-i input_file</code> argument in the <code>metaencrypt</code> command line to specify the file to be encrypted.
<code>-pipe</code>	Reads in data from a UNIX pipe.
<code>-o outfileName</code>	Encrypted output filename. Use the <code>-o output_file</code> argument in the <code>metaencrypt</code> command line to specify the file to which the encrypted content is output.
<code>-t encrypt_type</code>	Encryption methods: • Freelib: Weak (low security) <code>-t [ddl1 ddl2 custom freelib]</code> See Traditional Library Encryption. • 8-byte Data Encryption Standard (DES): Strong (medium security) <code>-t 8-byte-string</code> See 8-Byte Key Encryption. • Triple DES: Stronger (high security) <code>-t [privkey [key or file] randkey]</code> See Triple DES (3DES) Public and Random Keys. • AES 256 bits: Strongest (highest security) <code>-t [aes256]</code> See Advanced Encryption Standard Algorithm. • ENC 256 bits: Strongest (enhanced security) <code>-t [enc2023]</code> See Advanced Encryption Standard Algorithm.

Argument	Description
<code>-p</code>	Valid when using DES or Triple DES to encrypt a file. Data between the <code>.PROT</code> and <code>.UNPROT</code> commands is merged into a single file. For more information, see Encrypting by 3DES Partially .
<code>-d encrypt_dir</code>	Valid when using DES or Triple DES to encrypt a file. Data between the <code>.PROT</code> and <code>.UNPROT</code> commands undergoes encryption and outputs to an encrypted output file in directory <code>encrypt_dir</code> .
<code>-expire_date</code>	When using DES or 3DES encryption methods, the encrypted data expires and cannot be read by <i>Synopsys tools</i> again after the specified date.
<code>-z</code>	When using DES or 3DES encryption methods, the <code>-z</code> command line option enables compression first, then encrypts the data to get smaller encrypted files.
<code>-r synopsys_tool.access_control</code>	Valid values for <code>synopsys_tool</code> are: <code>hspice</code> , <code>primesim</code> , <code>nanosim</code> , <code>hsim</code> , <code>xa</code> , <code>finessim</code> , <code>autochar</code> , <code>nanotime</code> , or <code>synopsys</code> . The default is <code>synopsys</code> , meaning the encrypted file can be read by all Synopsys tools even if the <code>-r</code> argument is not specified. Valid values for <code>access_control</code> : • 0: Simulator suppresses any warning-related information to the encrypted block, such as warning or error messages, operating points, waveform results, and so on. • 1: Simulator outputs operating point information related to the encrypted block. Currently, this is supported only in the PrimeSim HSPICE tool, not in other PrimeSim Continuum simulators. Note: The PrimeSim HSPICE tool can always read an encrypted file, even if you have not specified <code>-r hspice</code>. If there are multiple settings for the same tool, the PrimeSim HSPICE tool uses the last setting. Example 1: In this invocation, the <code>access_control</code> for NanoSim is 0, and the <code>access_control</code> for PrimeSim HSPICE is 1: <pre>metaencrypt -i test.sp -o test.spe -t randkey + -r hspice:1 -r nanosim:0</pre> Example 2: Limit the parsing of the encrypted files to PrimeSim HSPICE. Other simulators cannot parse the encrypted file: <pre>metaencrypt -i test.sp -o test.spe -t randkey + -r hspice</pre>

metaencrypt Encryption Methods

The PrimeSim Continuum solution supports the following types of encryption through the `metaencrypt` utility:

- [Traditional Library Encryption](#)
- [8-Byte Key Encryption](#)

- [Triple DES \(3DES\) Public and Random Keys](#)
- [Advanced Encryption Standard Algorithm](#)

The `metaencrypt` utility can encrypt files with lines up to 254 characters or shorter. You can include multiple types of encrypted files in a PrimeSim Continuum solution simulation.

Traditional Library Encryption

A 5-rotor Enigma machine is the basis for the traditional library encryption algorithm (Freelib). You can specify which portions of subcircuits to encrypt. The `.PROT`/`.UNPROT` commands designate encrypted portions. The PrimeSim Continuum solution does not encrypt any other netlist entries. Library encryption uses a key value, which the PrimeSim Continuum solution reconstructs for decryption.

Note:

If you divide a data line into more than one line, and use the line continuation character (+) to link the lines, you cannot add `.PROT` or `.UNPROT` commands among these lines. The following example fails:

```
.prot
.Model N1 NMOS Level= 57
+TNOM = 27 TOX = 4.5E-09 TSI = .0000001 TBOX = 8E-08
+MOBMOD = 0 CAPMOD = 2 SHMOD =0
.unprot
+PARAMCHK=0 WINT = 0 LINT = -2E-08
```

When `metaencrypt` reads the input file, it looks for `.PROT` and `.UNPROT` pairs, and encrypts the text between them. You can encrypt only one file at a time.

Example

```
metaencrypt -i newmos.lib -o newmos.inc -t freelib
```

Note:

The netlist must be “complete”, that is, have an `.end` statement.

The following sections describe these topics:

- [Creating Files Using Traditional Encryption](#)
- [Example: Traditional \(freelib\) Encryption in a PrimeSim HSPICE Netlist](#)

Creating Files Using Traditional Encryption

The following sections describe:

- [Non-Library Encrypted Portions](#)
- [*.lib File Encryption](#)

Non-Library Encrypted Portions

You can encrypt the data between `.PROT` and `.UNPROT` commands in an `.sp` file, so that the PrimeSim Continuum solution can recognize it.

Note:

If you use `.sp` encryption, the encrypted data must not use `.INC`, `.LIB`, or `.LOAD` to include another file.

The following is an example of an `.sp` file:

```
*sample.sp*
.....

.lib 'cmos.lib' TT
.prot
.... $ data to be encrypted
.... $ do not include .inc .lib .load in encrypted data
.unpr
.inc sample2.inc
.....
.end
```

*.lib File Encryption

You can place any important information into a `.lib` file and encrypt it.

You can place parallel `.lib` commands into one library file and encrypt each `*.lib` separately. However, you must place `.PROT` and `.UNPROT` commands between each pair of `.lib` and `.endl` commands.

Note:

You must place `.PROT` and `.UNPROT` commands between `.lib` and `.endl` commands. To find the library name, the PrimeSim Continuum solution searches the `*.lib` file first.

The following is an example of a `*.lib` file:

```
*sample.lib*
.lib test1 $ .prot , .unpr should be put between
* .lib and .endl
.prot
..... $ data to be encrypted
```

```
.unpr
..... $ data not to be encrypted
.endl test1

.lib test2 $ .prot , .unpr should be put between
* .lib and .endl
.prot
..... $ data to be encrypted
.unpr
.endl test2

.lib test3
..... $ data
.endl test3
```

You use the preceding encrypted .lib file as you would any unencrypted one.

```
.lib './sample.lib' test1
.lib './sample.lib' test2
.lib './sample.lib' test3
.....
.end
```

Example: Traditional (freelib) Encryption in a PrimeSim HSPICE Netlist

The following complete example illustrates the metaencrypt encryption structure. This example enc.sp netlist has three encrypted files: mm.spe, xx.ic, and kk.lib.

```
file enc.sp:
*test .inc .lib .load encryption
.inc "mm.spe"
.load "xx.ic"
.lib 'kk.lib' pch
.OPTION post list
.tran 2ns 400ns
.end

file mm.spe:
.prot CUSTOM
-hs#y1B]*7[
+t'Y=0$S[t0]ajL
+C :Nx:$. $=<*X:$<#pP=020#ZWP=020x\K:[1:898
y[-x:$-#tRr0($x#4:/[U$<\K:I[U$<J <9 :P#ZQ
6%P2V7D6:]41/0#+:IXj0#ZWP=020#ZWP/[U$=J++bZ
3[7D6:BxHpg8
/C902P73+26
mh$y#D:bX/$\KwI)U-0R#=-ib+\[
a$0) :P.#$<) :P.#to)V:\7*K-I1M$#';-[Xz:9qpy
eMDv0%wUoxZ>mzwF*-(3_;W6x.*P!uW.]a+P0.h:n=O>1q+H(J0
o.H#-/B+($;W Me*0x<6#9[U-qpH/2h97%;-/B+T35Q
$m;['_-he[uE$%H) 5a:ZxRW9x=*77w$2]=*P!RW%.ahT3VQ
```

```
H0[I:[

file xx.ic:

.prot FREELIB
59yUH\$='x.3k77*<]8AT]8
<:7-(:9CV+7x15Xj+h'x=5Xj+(2 +4]8
<:7 D:\[2x9Y>/.7q
59y3\#D$ *y2k=u]PIq:97jH=u1w5Xj+x6
92k#<2FW0'k772<xBU677Q
59y3\#s# r21$],29b72[4'/RW72wd#$:O.U
+ 0sW%5$;[4sv;9=zV7[WFw[(g8#/']=AH%T5:7Z
[$%C999A2P!8
<:X2o60'$ 06($ #upe1:pX8
<5ax/toC n90;<0dw0]23G%C z9$Dh#Sw5a90
ZM*2!M[0
o729!=PAy73x(/1:6[
+ 0%2UT%8
_: -x*$X+q
$9P2y73x(/1:L
T#;*9A27!j+(/z
$$o#(:/b0
o7ZW-9 -PxJ+y
a9[$0\;n90;<0dw0]23G%C z9$Dh#Sw5a90
Zr ;6

file kk.lib:

.LIB NCH
.prot FREELIB
HO. T,# %fXz>MZWf*-(3_;w6X.*p!Uw.]A+p0.H:N=o>1Q+h(j0
o.H#-/B+($;W Me*0x<6#9[U-qpH/2h97%;-/B+T35Q
$m;'_-he[uE$%H) 5a:ZxRW9x=*77w$2]=*P!RW%.ahT3VQ
H0[I:[
.ENDL

.LIB PCH
.prot FREELIB
HO. T,#t%fXz>MZWf*-(3_;w6X.*p!Uw.]A+p0.H:N=o>1Q+h(j0
o.H#-/B+($;W Me*0x<6#9[U-qpH/2h97%;-/B+T35Q
$m;'_-he[uE$%H) 5a:ZxRW9x=*77w$2]=*P!RW%.ahT3VQH0[I:[
.ENDL
```

8-Byte Key Encryption

An 8-byte key encryption feature, based on a 56-bit DES, is available in `metaencrypt`. When using 8-byte encryption, you can encrypt files with line lengths of 254 characters or shorter. The encrypted data is in binary format.

To encrypt a file, you provide a *keyname* that can contain alphabetic characters and numbers, and which is no longer than 8 bytes. To use the encrypted file, you must use the `.inc` command in the main netlist.

The PrimeSim Continuum solution supports include file (`.inc`) encryption, when you use 8-byte key encryption. To use this encryption:

1. Insert the data to encrypt into an include file.
2. Encrypt this file.

Follow these rules when you use 8-byte key encryption:

- 8-byte key encryption supports only `.inc` encryption.
- 8-byte encryption does NOT support `.LIB`, `.LOAD`, or `.OPTION SEARCH` encryption. Choose another form of encryption for these types of files.
- If *keyname* is an 8-byte string (combination of characters and numbers), then `metaencrypt` performs the 8-byte key encryption.
- In an `.sp` file, you cannot encrypt the first line because it is the title. You also cannot encrypt the last line because it marks the end of the file.

The following sections discuss these topics:

- [Creating 8-byte Key Encryption](#)
- [Placing an 8-byte Key Encrypted File Into a PrimeSim HSPICE-Compatible Netlist](#)

Creating 8-byte Key Encryption

Use the following syntax to create 8-byte key encryption:

```
metaencrypt -i example.dat -o example.inc -t fGi85H9b
```

In the following example, `example.dat` contains the data to encrypt.

```
* DFF subckt netlist
$notice no .prot or .unprot used for this method
.subckt XGATE control in n_control out
m0 in n_control out vdd pmos l=0.90u w=3.4u
m1 in control out gnd nmos l=0.90u w=3.4u .ends
.....
v14 vdd gnd dc=5
Xi3 net25 net31 net27 dff_nq DFF l=1u wn=3.8u wp=10u
Xi2 dff_nq d_output INV wp=26.4u wn=10.6u
.ends XGATE
```

Placing an 8-byte Key Encrypted File Into a PrimeSim HSPICE-Compatible Netlist

The following fragment is an example of placing an 8-byte key encrypted file into a PrimeSim HSPICE-compatible netlist:

```
* example.sp file using encrypted example.dat
.Options Post NoMod
.Global vdd gnd
.lib 'demo.lib' TT
.inc 'example.inc' $ this is the encrypted file
...
*
.Tran 1n 8n Sweep Optimize = Opt1
+           Result      = MaxVout      $ Look at measure
+           Model       = OptMod
.end
```

Triple DES (3DES) Public and Random Keys

The PrimeSim Continuum solution Triple DES (3DES) encryption uses a 192-bit key to achieve a maximum level of security. You can generate the encryption keys for a new algorithm with one of the following options:

- 256-bit public key

With this option, the PrimeSim Continuum solution generates a 256-bit public key during the encryption process. You must distribute this key to the customer in order to run a simulation. Every time the 256-bit public key performs the encryption, the encrypted file and the generated public key are different, even with the same private key and input file. This allows you to generate different encrypted file and public key combinations for different customers.

Your customers cannot access the key used for encryption, but they can run a simulation on a circuit by putting this public key file in the same run directory.

The 256-bit public key supports multiple encrypted files. You have to put all relevant public key files in the directory from which the simulation originates. You can also generate these encrypted files with the same private key or different private keys. The actual “key” needs to be a user-generated 192-bit string. For example, the following two files receive encryption with the same key:

```
metaencrypt -i a.dat -o a.inc -t privkey
```

```
metaencrypt -i b.dat -o b.inc -t privkey
```

The encryption creates two public key files: a.inc.key, and b.inc.key. These files are different even when the same private key generates the encryption. A simulation run

with a netlist file containing `.include a.inc` and `.include b.inc` commands requires that both key files be in the simulation directory.

- 192-bit random key

With this option, the PrimeSim Continuum solution does not need an additional public key. The encrypted file is different every time you run an encryption.

Caution:

Files encrypted with 3DES encryption, both 256-bit public key and 192-bit random key encryption, are not backward compatible. That is, for enhanced security, the 3DES encryption methods are designed to be upgraded in each PrimeSim Continuum simulator feature release. Therefore, it is expected that the newly encrypted IP files cannot be read by older PrimeSim Continuum simulator releases. If cross-version compatibility and data security are important, older versions of `metaencrypt` can be used to encrypt files using 3DES encryption methods.

Encrypted IP files produced by other encryption methods such as 8-byte key are consistent to be used between PrimeSim Continuum simulator releases.

The following sections discuss these topics:

- [Creating 3DES Encrypted Files](#)
- [Placing 3DES Encryption Files into a PrimeSim HSPICE-Compatible Netlist](#)
- [Encrypting by 3DES Partially](#)

Creating 3DES Encrypted Files

Observe these rules when creating Triple DES encrypted files:

- You can use embedded `.lib` encryption only if you set it up using `.PROT` and `.UNPROT` commands inside the `.lib` file, plus use the `metaencrypt` utility `-d` option.
- Do not use `.OPTION SEARCH`, when you encrypt models and subcircuits. (The old `metaencrypt` functionality supported this method.) To directly encrypt subcircuits and model libraries, use the traditional `.inc` and `.lib` encryption methods.

Random Key Example

For files *without* embedded `.lib`, `.inc`, or `.load` commands:

```
metaencrypt -i dff.sp -o dff_rand.spe -t randkey
```

Note:

This netlist file has no `.PROT` or `.UNPROT` commands, similar to 8-byte encryption.

For files *with* embedded `.lib` commands:

```
metaencrypt -i demo.lib -o demo_rand.lib -t randkey + -d ./lib_rand
```

Note:

The `demo.lib` file for this example has the same `.PROT/.UNPROT` setup as for traditional `freelib`.

Public Key Example

For files without embedded `.lib`, `.inc`, or `.load` commands:

```
metaencrypt -i dff.sp -o dff_priv.spe -t privkey  
0123456789ABCDEF9876543210FEDCBA1357924680ACEBDF
```

Note:

This netlist file has no `.PROT` or `.UNPROT` commands, similar to 8-byte encryption.

For a file that has an embedded `.lib` command:

```
metaencrypt -i demo.lib -o demo_priv.lib -t privkey  
0123456789ABCDEF9876543210FEDCBA1357924680ACEBDF -d ./lib_priv
```

Note:

The `demo.lib` file for this example has the same `.PROT/.UNPROT` setup as for traditional `freelib`.

Placing 3DES Encryption Files into a PrimeSim HSPICE-Compatible Netlist

While using a random or private key method for Triple DES may look similar, the private one requires that the key resides in the same directory.

Example 1: Random Triple DES

```
* Example netlist for including random Triple DES  
.Options Post NoMod  
.Global vdd gnd  
.lib 'demo_rand.lib' TT    $ bring in the random 3DES lib file,  *that  
    looks at the ./lib_rand directory for files  
.inc 'dff_rand.spe'        $ bring in random 3DES encrypted design *file  
...  
.end
```

Example 2: Private Triple DES

```
* Example netlist for including private Triple DES  
.Options Post NoMod  
.Global vdd gnd
```

```
.lib 'demo_priv.lib' TT    $ bring in private key lib file; the *keys are
in the ./lib_priv directory along with the files
.inc 'dff_priv.spe'      $ bring in private key encrypted file; *key
must be in this same directory (dff_priv.spe.key)
...
.end
```

Encrypting by 3DES Partially

When using embedded libraries and 3DES encryption, the PrimeSim Continuum solution can now merge the encrypted and non-encrypted portions into a single file. The PrimeSim Continuum solution uses 3DES to encrypt the part specified in `.PROT/.UNPROT` to the same encrypted output file. This called encrypting by DES partially or PDES.

Use the following syntax for PDES:

```
metaencrypt -i input_netlist -t 8_byte_string -p
+ -o output_file_name
```

Advanced Encryption Standard Algorithm

The Advanced Encryption Standard (AES) algorithm is based on a design principle known as a substitution–permutation network, and is efficient in both software and hardware. Unlike its predecessor DES, AES does not use a Feistel network. AES is a variant of Rijndael, with a fixed block size of 128 bits, and a key size of 128, 192, or 256 bits.

AES operates on a 4×4 column-major order array of bytes, termed the state. Most AES calculations are done in a particular finite field.

For instance, 16 bytes are represented as the following two-dimensional array:

```
b0 b4 b8  b12
b1 b5 b9  b13
b2 b6 b10 b14
b3 b7 b11 b15
```

Creating AES Key Encryption

Use the following syntax to create AES key encryption:

```
metaencrypt -i input_file -o encrypted_output_file -t aes256
```

Input File

Use `-i input_file` in the `metaencrypt` command line to specify the file to encrypt. The input file is encrypted by the AES algorithm with `-t aes256` in the `metaencrypt` command line.

Output File

Use `-o output_file` in the `metaencrypt` command line to specify the file to which the encrypted content is displayed as output. The first line of the file is `.PROT aes256`.

New Encryption Algorithm

A new encryption algorithm for enhanced security.

Creating New Encryption Algorithm Encryption

Use the following syntax to create the New Encryption Algorithm encryption:

```
metaencrypt -i input_file -o encrypted_output_file -t enc2023
```

Input File

Use `-i input_file` in the `metaencrypt` command line to specify the file to encrypt. The input file is encrypted by the new algorithm with `-t enc2023` in the `metaencrypt` command line.

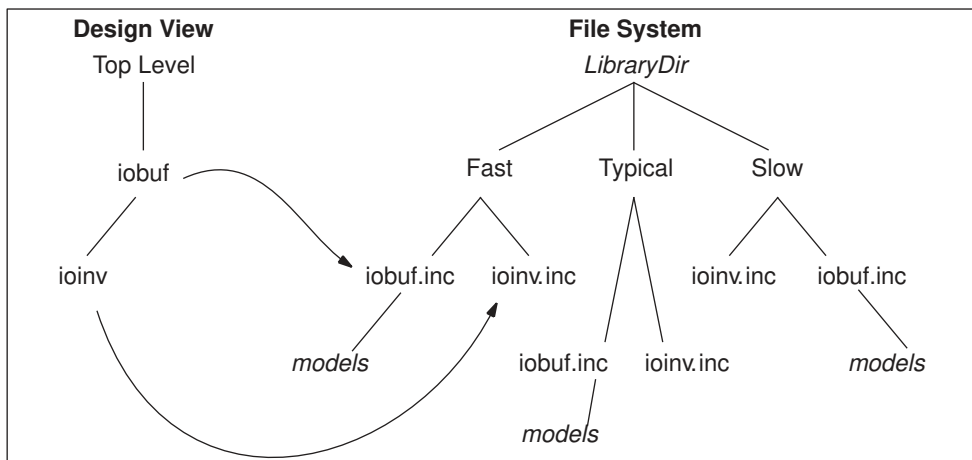
Output File

Use `-o output_file` in the `metaencrypt` command line to specify the file to which the encrypted content is displayed as output. The first line of the file is `.PROT ENC2023`.

General Example

The requirements for encrypted libraries of subcircuits are the same as the requirements for regular subcircuit libraries, as described in this guide. To refer to an encrypted subcircuit, use its subcircuit name in a subcircuit element line of a PrimeSim HSPICE netlist.

Figure 5 Encrypted Library Structure



The following example describes an encrypted I/O buffer library subcircuit. This subcircuit consists of several subcircuits and model commands that you need to protect with encryption. [Figure 5](#) shows the organization of subcircuits and models, in the libraries used in this example.

The following input file fragment from the main circuit level selects the Fast library. It also creates two instances of the iobuf circuit.

```
...
.Option Search='LibraryDir/Fast'    $ Corner Spec
x1 drvin drvout iobuf    Cload=2pF    $ Driver
u1 drvout 0 recvin 0 PCBModel    ...    $ Trace
x2 recvin recvout iobuf    $ Receiver
...
```

The *LibraryDir/Fast/iobuf.inc* file contains:

```
.Subckt iobuf Pin1 Pin2 Cload=1pF
*
* iobuf.inc - model 2001 improved iobuf
*
cPin1 Pin1 0 1pF    $ Users cannot change this!
x1 Pin1 Pin2 ioinv
.Model pMod pmos Level=28 Vto=...    $ <FastModels>
.Model nMod nmos Level=28 Vto=...    $ <FastModels>
cPin2 Pin2 0 Cload    $ gives you some control
.Ends
```

The *LibraryDir/Fast/ioinv.inc* file contains:

```
.Subckt ioinv Pin1 Pin2
mp1 Vcc    Pin1 Pin2 Vcc pMod...
mn1 Pin2    Pin1 Gnd Gnd
nMod...
.Ends
```

The encrypted file looks similar to the following:

```
.SUBCKT ioinv Pin1 Pin2
.PROT FREELIB    $ Encryption starts here ...
X34%43*27@#^3rx*34&%^#1
^(*^!^HJHD(*@H$!:&*$
dFE2341&*&) (@@3    $ ... and stops here

.ENDS
```

Note:

The `.UNPROTECT` statement also becomes encrypted during the encryption process.

After encryption, the basic layout of the subcircuits is the same. However, you cannot read the file. All the PrimeSim simulators can read this file.

Encryption also suppresses printouts of encrypted model information from the PrimeSim HSICE tool. All the PrimeSim simulators can decrypt the model.

Using the -pipe Option

A command series can be read in, using the input from a UNIX pipe.

```
FILE *pipe;
char buf[100];
pipe = popen("metaencrypt -o outfile ....", "w");
strcpy(buf, ".....");
fprintf(pipe, buf);
strcpy(buf, ".....");
fprintf(pipe, buf);
pclose(pipe);
```

After the pipe closes, all the PrimeSim simulators can generate the encrypted file.

Troubleshooting

The following sections discuss these issues:

- [Warning Message](#)
- [Encrypting S-Parameter Files](#)
- [Freelib Issue With Equations](#)
- [Freelib Issue With DOS-format Files for Simulation on UNIX/Linux Platforms](#)

Warning Message

The following warning occurs even when there are no explicit encrypted blocks in the netlist:

```
**warning** parameters... as an expression containing output signals
```

There are two reasons for this warning message:

- `.PROT` and `.UNPROT` commands are in the netlist.
- The results of parameter expressions which contain output signals are not correct. For example:

```
.param myfunc (one,two)='abs (one - two) '
.param test=myfunc(v(1),v(2))
.protect
.if ( test <= 1 )
.param k='2*1p'
```

```
.elseif ( test <= 4 )
.param k='8*1p'
.else
.param k='1*test*1p'
.endif
.unprotect
c1 2 0 c=k
```

Note:

If you use `.PROT/.UNPROT` in a library or file that is unencrypted, you get warnings that the file is an encrypted file and the file or library is a “black box.”

Encrypting S-Parameter Files

While S-parameters do not convey any IP information and encryption, you do not need to add encryption. You can encrypt S-parameters and use them in PrimeSim Continuum simulations if the file is in the SELEM format (`*.sc0`).

To encrypt a SELEM-formatted S-parameter file and use it in the PrimeSim Continuum solution, follow this procedure:

1. Generate an `.sc0` file. Use the `.LIN` command to extract the S-parameters from a circuit. The `.LIN` command creates an `.sc0` file. If you already have a TouchStone 1.0/2.0 or CITI formatted file, you can also use the `.LIN` command to convert the file to an `.sc0` formatted file.
2. Edit the `.sc0` file so that it is a `.lib` file. For example,

```
.lib s
.protect*| NPort=2 DATA=1000 COMPLEX_DATAFORMAT=RI NOISE=0
GROUPDELAY=0*| NumOfBlock=1 NumOfParam=0
*
.MODEL filter S
+ N=2 FQMODEL=SFQMODEL TYPE=S Zo= 50.0000 50.0000
* + FBASE= FMAX=
.MODEL SFQMODEL SP N=2 SPACING=POI INTERPOLATION=LINEAR
MATRIX=NONSYMMETRIC
+ DATA=1000
+ ...
<S-parameter data>
.unprotect
.endl
```

3. Encrypt the file using either 8-bit encryption or 3DES encryption.
4. To use the encrypted file in the netlist, call the S-parameter data as a `.lib` in addition to defining the S-element.

```
* Encrypted S-parameter Example
...
```

```
.lib 'filter.sdt' s          $ encrypted S-parameter library file
                           $ contains the model 'filter'
S1 in out 0 mname=filter    $ S-element with model name 'filter'
...
.end
```

Freelib Issue With Equations

When you run encrypted files using the freelib method, you may see error messages such as:

```
**error** Left parenthesis improperly placed in column 90
**error** Characters after end of expression in column 148
```

The error messages only occur when using freelib encryption because this method does not support the mathematical operator caret (^) used for power in equations. If you are using freelib encryption, use two asterisks (**) as an operator for power in equations.

Note:

It is recommended that you use either 8-byte or Triple DES encryption as both the 8-byte and 3DES encryption are much stronger encryption methods than the freelib encryption method.

Freelib Issue With DOS-format Files for Simulation on UNIX/Linux Platforms

When a DOS (Windows) format file that is encrypted using the freelib encryption method, is simulated on a Linux machine, you may see the following error message:

```
**error** Error in encrypted blocks, please contact encrypted file
provider.
```

To avoid this error, you should use either the 8-byte or the 3DES encryption method for encryption.

Note:

The 8-byte and 3DES methods are stronger encryption methods, that is, they provide more secure encrypted output than freelib.

5

Using Verilog-A

Describes how to use Verilog-A in PrimeSim Continuum simulations.

Verilog-A derives from the IEEE 1364 Verilog Hardware Description Language (HDL) specification for describing behavior in analog systems. The Verilog-A language that the PrimeSim Continuum simulators support is compliant with *Verilog-AMS Language Reference Manual, Version 2.3.1*. The [Unsupported Language Features](#) section lists limitations.

The Verilog-A implementation in the PrimeSim Continuum solution supports a mixed design of Verilog-A descriptions and transistor-level SPICE netlists with a simple use model. Verilog-A supports most analysis features available in the PrimeSim Continuum simulators for Verilog-A based devices, including AC, DC, transient analysis, statistical analysis, and optimization.

For information on Verilog-A integrated with 3DIC modularization, see [Using Verilog-A Modules Within the .MODULE Scope](#).

For Verilog-A demo netlists, see “Verilog-A Examples” in the *PrimeSim™ HSPICE® User Guide: Demonstration Netlists*.

These sections discuss the following topics:

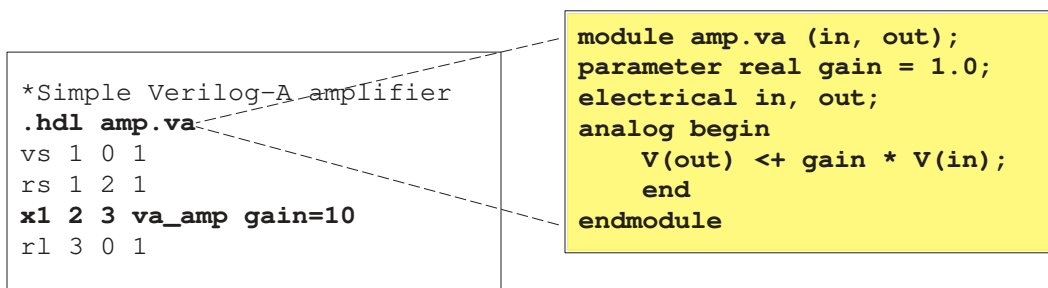
- [Getting Started](#)
- [Introduction to Verilog-A Data Types and System Tasks and I/O Functions](#)
- [Simulation With Verilog-A Modules](#)
- [Loading Verilog-A Modules](#)
- [Instantiating Verilog-A Devices](#)
- [Instantiating Primitive Elements inside Verilog-A Modules](#)
- [Instantiating PrimeSim Continuum Subcircuits Inside Verilog-A Modules](#)
- [Output Simulation Data](#)
- [SPICE Netlist and Verilog-A Interactions](#)

- [Encrypting Verilog-A Files](#)
- [Compiling Verilog-A Files](#)
- [Supported LRM 2.4 Syntax and Features](#)
- [Sharing of Precompiled Verilog-A Files by Multiple Users](#)
- [Performance Considerations With the PrimeSim Continuum Solution](#)
- [Known Limitations](#)
- [Verilog-A \(pVA\) Messages](#)
- [Environment Variables and Command Options](#)
- [Example: Verilog-A Frequency Divider Model](#)

Getting Started

This section explains how to get started with Verilog-A in the PrimeSim Continuum environment.

Figure 6 *PrimeSim Continuum and Verilog-A*



Verilog-A devices use the following conventions:

- Load modules into a PrimeSim Continuum simulator with either the `.HDL` netlist command or the `-hdl` command-line option (not supported in advanced analog analyses).
- Use "X" as the first character for the name of instance. The PrimeSim Continuum simulator instantiates modules in the same manner as PrimeSim Continuum subcircuits.
- Modify instance and model parameters the same as you do with any other PrimeSim Continuum simulator subcircuit instances.

- Use the `vamodel` and `spmodel` commands to pick the subcircuit or Verilog-A module to use, as module names may conflict with any built-in device keywords in PrimeSim Continuum simulators.
- Output node voltages and branch currents by using conventional output commands.

To run a PrimeSim Continuum solution Verilog-A simulation using the PrimeSim HSPICE tool, locate and run the `hspice` script in the `$installdir/hspice/bin/hspice` directory.

Introduction to Verilog-A Data Types and System Tasks and I/O Functions

This section provides an overview of the Verilog-A language, as described in the following topics:

- [Data Types](#)
- [System Tasks and I/O Functions](#)

See the Verilog-AMS LRM 2.3.1 from Accellera for syntax and usage details.

Data Types

Several Verilog-A data types are available. The parameter type passes values from the netlist to the Verilog-A module.

String Operators and Semantics

[Table 7](#) describes the string operators and their meanings.

Table 7 Verilog-A String Operators

Operators	Semantics
<code>Str1 == Str2</code>	Equality. Checks whether the two strings are equal. Result is 1 if they are equal and 0 if they are not.
<code>Str1 != Str2</code>	Inequality. Logical negation of <code>==</code>
<code>Str1 < Str2</code> <code>Str1 <= Str2</code> <code>Str1 > Str2</code> <code>Str1 >= Str2</code>	Comparison. Relational operators return 1 if the corresponding condition is true using the lexicographical ordering of the two strings <code>Str1</code> and <code>Str2</code> .
<code>{Str1, Str2, ..., StrN}</code>	Concatenation.
<code>{M{Str}}</code>	Replication. Multiplier <code>M</code> must be of integral type and can be non-constant.

System Tasks and I/O Functions

System functions provide access to system-level tasks as well as access to simulator information.

Table 8 Verilog-A System Tasks and I/O Functions

Function	Description
<code>\$fopen()</code>	Opens a file for writing and assigns it to an associated multichannel descriptor or a file descriptor, determined by the absence or presence of the <i>type</i> argument: <pre>multi_channel_desc = \$fopen("file"); file_desc = \$fopen("file", "type");</pre> Unlike multichannel descriptors, you cannot combine file descriptors with the bitwise OR operator to direct output to multiple files. Instead, the file descriptor opens files for input, output, and both input and output, as well as for append operations, based on the value of argument <i>type</i>, according to the following table: Arguments • <code>r</code> or <code>rb</code>: Open for reading • <code>w</code> or <code>wb</code>: Truncate to zero length or create for writing • <code>a</code> or <code>ab</code>: Append; open for writing at end of file; or create for writing • <code>r+</code> or <code>r+b</code> or <code>rb+</code>: Open for update (reading or writing) • <code>w+</code> or <code>w+b</code> or <code>wb+</code>: Truncate or create for update • <code>a+</code> or <code>a+b</code> or <code>ab+</code>: Append; open for writing or create for update at end of file For example: <pre>hdl1 = \$fopen("append_mode.txt", "a"); hdl2 = \$fopen("overwrite_mode.txt", "w");</pre> The following expansions are not part of the LRM 2.3.1: • Use an escape sequence <code>"%I"</code> to represent the name of the instance in the file name. For example, when the following is specified and called from instance <code>x2</code>: <pre>hdl1=\$fopen("%I_data.txt")</pre> The tool looks for a file called <code>x2_data.txt</code>. • Use a tilde <code>"~"</code> to reference the value of the environment variable <code>\$HOME</code> (or <code>%HOMEDRIVE%%HOMEPATH%</code> on Windows). The tilde expands to the full path of the home directory. For example, if the following is specified and called from instance <code>x1</code>: <pre>\$HOME=/user/default hdl1=\$fopen("~/datafiles/%I_data.txt")</pre> The tool looks for a path and file called <code>/user/default/datafiles/x1_data.txt</code>.

Table 8 Verilog-A System Tasks and I/O Functions (Continued)

Function	Description
<code>\$fopen()</code> (cont'd)	The following predefined percent codes are supported by <code>\$fopen()</code>. • <code>%C</code> – Design file name. Example: <code>/tmp/works/input.spi</code> • <code>%D</code> – Date <code>yyyy-mm-dd</code>. Example: <code>94-02-28</code> • <code>%H</code> – Host name. Example: <code>snps</code> • <code>%S</code> – Simulator type. Example: <code>finestim</code> • <code>%P</code> – Unit process id#. Example: <code>3641</code> • <code>%T</code> – Time (<code>24hh:mm:ss</code>). Example: <code>15:19:25</code> • <code>%I</code> – Instance name. Example: <code>opamp3</code> • <code>%A</code> – Analysis name. Example: <code>dc0p</code>, <code>timeDomain</code>, <code>acSup</code> Syntax <pre>x1 a b va_module filename=str("%C.%I.%T.txt")</pre> The special output formatting commands can be followed by one or more modifiers, which extract information from UNIX file names. Any other character after a colon (<code>:</code>) signals the end of modifications. That character is copied with the previous colon. • <code>::</code> – The colon (<code>:</code>) character itself • <code>:r</code> – Root (base name) of the path for the file. Example: <code>%C:r /tmp/works/input</code> • <code>:e</code> – Extension of the path for the file. Example: <code>%C:e spi</code> • <code>:h</code> – Head of the path for any portion of the file before the last slash (<code>/</code>). Example: <code>%C:h /tmp/works</code> • <code>:t</code> – Tail of the path for any portion of the file after the last slash (<code>/</code>). Example: <code>%C:t input.spi</code> The modifiers are typically used with the <code>%C</code> command although they can be used with any of the commands. However, when the output of a formatting command does not contain a slash (<code>/</code>) and a period (<code>.</code>), the modifiers <code>:t</code> and <code>:r</code> return the whole name and the <code>:e</code> and <code>:h</code> modifiers return a period (<code>.</code>). As a result, using modifiers with formatting commands other than <code>%C</code> might not produce the results you expect.
<code>\$fclose()</code>	Closes a file from a previously-opened channel. <pre><code>\$fclose(multi_channel_descriptor);</code></pre>

Table 8 Verilog-A System Tasks and I/O Functions (Continued)

Function	Description
<code>\$fscanf()</code>	Non-LRM 2.3.1 function. Provides a means to read data from files. Syntax <pre>[integer_return_value =] \$fscanf (multi_channel_descriptor, format_string [, list_of_arguments]);</pre> where: <code>multi_channel_descriptor</code> is the multichannel descriptor returned by the <code>\$fopen()</code> command at the opening of the file. <code>format_string</code> is a string describing how the data is to be matched. <code>list_of_arguments</code> is optional and comma-separated, and stores the read data that matches the list of arguments. The allowed commands for the <code>format_string</code> are the same as those available for the <code>\$strobe()</code> function argument. Each data value read is sequentially matched to the corresponding argument in the <code>list_of_arguments</code>. The <code>list_of_arguments</code> must have the correct number of variables to match the data value types in the <code>format_string</code>. The optional return value of the function is set to the number of valid arguments read during the operation; if the return value is not used, a warning is issued. The channel specified in the <code>multi_channel_descriptor</code> must be assigned to an open file by using the <code>\$fopen()</code> function. Example The following example reads an integer, real, and character value from the file <code>data.txt</code> and puts the values in <code>int_value</code>, <code>real_value</code>, and <code>char_value</code>, respectively. The integer valid is set to the number of valid reads, in this case, 3. <pre>integer multi_ch_desc, valid, int_value, char_value; real real_value; @(initial_step) multi_ch_desc = \$fopen ("data.txt", "r"); valid = \$fscanf (multi_ch_desc, "%d %e %c", int_value, real_value, char_value);</pre>
<code>\$fstrobe()</code> <code>\$fdisplay()</code> <code>\$fwrite()</code>	Writes simulation data to an opened channel when the simulator has converged. Follows the format for <code>\$strobe()</code>: <pre>\$fstrobe(multi_channel_descriptor, "information to be written");</pre>
<code>\$fflush()</code>	Non-LRM 2.3.1 function. Writes any buffered output to the files specified by the file descriptor. Syntax <pre>\$fflush (multi_channel_descriptor);</pre> where: <code>multi_channel_descriptor</code> is an integer that represents one or more opened files. The following example illustrates typical uses for the <code>\$fflush()</code> command. <pre>\$fflush (multi_ch_desc);</pre>

Simulation With Verilog-A Modules

When simulating with Verilog-A in the PrimeSim Continuum solution, you must have the following basic input files:

- PrimeSim Continuum simulator netlist/model card (Mandatory)
- Verilog-A model file (for example, *.va or *.vams file) or encrypted Verilog-A file
- If a Verilog-A model file is provided without an extension, the PrimeSim Continuum simulator automatically appends the extension .va to the file name.

For example, if the command is entered as `.hdl "xyz"` then the PrimeSim Continuum simulator opens the file as `./xyz.va`.

- PrimeSim Continuum solution Verilog-A feature setup options (Optional, but mandatory under certain conditions)

Basic output files:

- PrimeSim Continuum standard output files
- The Verilog-A log file, *.valog, which contains Verilog-A specific messages from compiling and simulating phases. The content of the *.valog file is also echoed to the *.lis file.

Verilog-A Output Directory

The Verilog-A output directory, <-o>.pvadir/, contains the following intermediate files on UNIX (and *.dll files on the Windows platform):

File Type	Contains
pvaRTL.log	Log file output from gcc compilation
pvaRTL_linux64.so	Runtime library input to PrimeSim Continuum simulators
pvaEnv	Storage information for incrementally compiling user Verilog-A files

For example, for the PrimeSim HSPICE tool:

Command-line statement	Creates
<code>hspice test.sp -o output</code>	<code>output.pvadir</code>
<code>hspice test.sp</code>	<code>test.pvadir</code>
<code>hspice test.sp -o test</code>	<code>test.pvadir</code>

Command-line statement	Creates
<code>hspice test.sp -o /remote/hspice/benchmark</code>	<code>/remote/hspice/benchmark.pvadir</code>

The Verilog-A output directory, `<-o>.pvadir/`, can be overridden by

```
setenv PVA_DIR $absolute|relative_path
```

The preceding environment variable creates a new runtime library path, shares the existing path, or reuses precompiled Verilog-A code in the existing runtime library. In this example, the following intermediate directories on UNIX (and `*.dll` files on the Windows platform) are created:

Command-line statement	Creates
<code>hspice test.sp -o output</code>	<code>\$absolute relative_path.pvadir</code>
<code>hspice test.sp</code>	<code>\$absolute relative_path.pvadir</code>
<code>hspice test.sp -o test</code>	<code>\$absolute relative_path.pvadir</code>
<code>hspice test.sp -o /remote/hspice/benchmark</code>	<code>\$absolute relative_path.pvadir</code>

Loading Verilog-A Modules

This section describes how to load Verilog-A modules into a PrimeSim Continuum simulator and how to specify cell names for Verilog-A definitions. A module must be loaded before it can be instantiated.

Verilog modules are loaded into a PrimeSim Continuum simulator by:

- Including an `.HDL` statement in a PrimeSim HSPICE-compatible netlist
- Using the `-hdl` command-line option (not supported in advanced analog analyses)

Files can be in the current directory or specified via an absolute or relative path. The Verilog-A file is assumed to have a `*.va` extension when only a prefix is provided. For example, `.hdl "model"` looks for a file named “model” first and then `model.va` file if “model” is not found.

Use the `-vamodel` command-line option to specify cell names for Verilog-A definitions (not supported in advanced analog analyses).

For a description of the `.hdl` statement, see the `.HDL` command in the *PrimeSim™ Continuum Reference Manual: Commands and Control Options*.

For a description of the `-hdl` and `-vamodel` command-line options, see “Syntax to Invoke PrimeSim HSPICE” in the *PrimeSim™ HSPICE® User Guide: Basic Simulation and Analysis*.

The following sections discuss these topics:

- [.hdl Command Module Selection](#)
- [.hdl Command Module Alias](#)
- [Verilog-A File Search Path](#)
- [Verilog-A File Loading Considerations](#)

.hdl Command Module Selection

The `.hdl` command module selection uses the following syntax:

```
.hdl use_va {module_selection}
```

For more information, see [pVA Compiles ALL Modules in an .hdl Statement](#).

.hdl Command Module Alias

The `.hdl` command module alias uses the following syntax:

```
.hdl filename {module_name module_alias}
```

Note:

The following conditions apply while defining a module alias:

- The `module_name` and `module_alias` parameters must be specified in pairs.
- The PrimeSim Continuum solution supports multiple pair specifications.

The optional `module_name` and `module_alias` parameters do not apply to PrimeSim Continuum advanced analog analyses.

Example 1

```
.hdl 'fast.va' 'RES' 'FAST_RES'  
...  
x1 1 2 fast_res r=1  
...
```

The case of the `module_name` and `module_alias` parameters is defined by single or double quotation marks, because Verilog-A is case-sensitive.

In the preceding example, Verilog-A uses the case-sensitive `RES` to find the module in the `fast.va` file. When Verilog-A finds `RES`, it changes the name internally to `FAST_RES` for the PrimeSim Continuum simulator to use.

Then the PrimeSim Continuum simulator converts `FAST_RES` to `fast_res` (lowercase) and uses it to find the `x1` instance in the SPICE netlist.

Example 2

```
.hdl 'fast.va' RES FAST_RES
...
x1 1 2 fast_res r=1
...
```

Without single or double quotation marks, `RES` and `FAST_RES` are converted to lowercase by the PrimeSim Continuum simulator. Verilog-A uses the lowercase `res` and returns `fast_res (d,e)` to the PrimeSim Continuum simulator for the `x1 1 2 fast_res r=1` subcircuit instance.

Note:

In the following example, the names of the modules are treated as two different modules:

```
module RES (A,B)

module res (d,e)
```

As a result, there are no error messages generated during Verilog-A compilation.

Verilog-A File Search Path

During a simulation, the PrimeSim Continuum simulator searches for Verilog-A files in the current directory. You can also provide a search path via the `-hdlpath` command-line option (not supported in advanced analog analyses) or the `HSP_HDL_PATH` environment variable to have the PrimeSim Continuum simulator search other directories for the files. The `-hdlpath` command-line option is provided for PrimeSim Continuum solution Verilog-A use only, which defines the search path specifically for Verilog-A files.

For a description of the `-hdlpath` command-line option, see “Syntax to Invoke PrimeSim HSPICE” in the *PrimeSim™ HSPICE® User Guide: Basic Simulation and Analysis*.

When a Verilog-A file cannot be found in the current working directory or the directory defined by `-hdlpath`, or there is no `-hdlpath` defined, the PrimeSim Continuum simulator searches for the Verilog-A file in a directory defined by `HSP_HDL_PATH`.

The directory search order for Verilog-A files is:

1. Current working directory; for example (for the PrimeSim HSPICE tool):

- Command line: `hspice res.sp -o res`
- In SPICE netlist: `.hdl "res.va"`

The PrimeSim HSPICE tool opens the `./res.va` file.

2. Path defined by `-hdlpath` on the PrimeSim HSPICE command line or `-I` on the PrimeSim XA command line
3. Path defined by `setenv HSP_HDL_PATH`
4. Path defined by `.OPTION SEARCH` (see the `.OPTION SEARCH` command in the *PrimeSim™ Continuum Reference Manual: Commands and Control Options*)

The path defined by `-hdlpath` or `HSP_HDL_PATH` can consist of a set of directory names. The path separator must follow PrimeSim HSPICE conventions or platform conventions (that is, ":" on UNIX or ";" on Windows). Path entries that do not exist are ignored and no error or warning message is issued.

On Windows, the name of the directory can have white spaces between words; but UNIX cannot have white spaces. *A file name cannot have a white space on either Windows or UNIX.*

Examples

The following examples assume the C-shell is used.

UNIX platform:

```
setenv HSP_HDL_PATH /lib_path/veriloga:/lib_path/compact_model
```

Windows platform:

```
setenv HSP_HDL_PATH d:\lib_path\veriloga;d:\lib_path\compact_model
```

The following example has a white space in the folder name:

```
setenv HSP_HDL_PATH d:\Documents and Settings\  
My Documents\lib_path\veriloga;d:\lib_path\compact_model
```

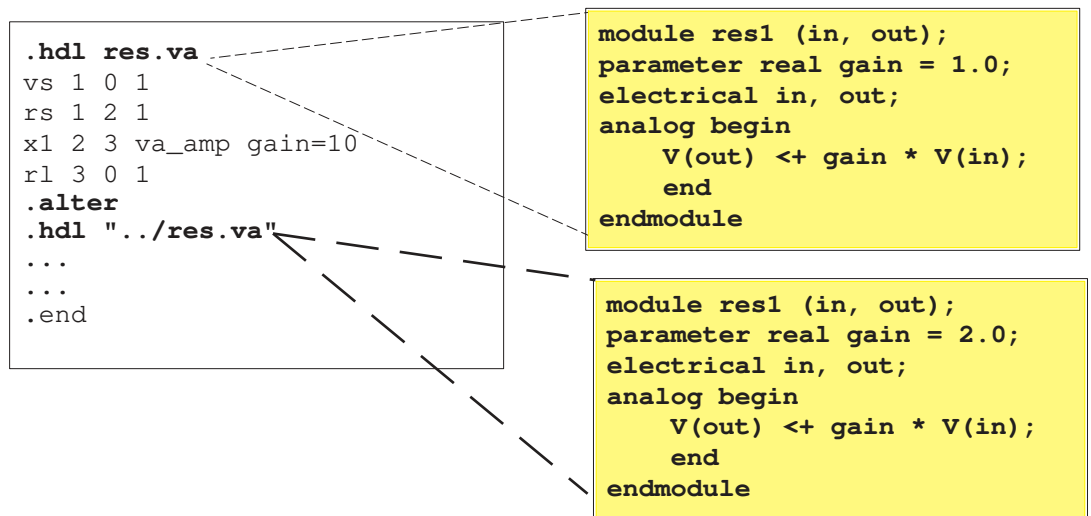
Verilog-A File Loading Considerations

Several restrictions and issues must be considered when loading Verilog-A modules:

- You can place an `.hdl` statement anywhere in the top-level circuit. All Verilog-A modules are loaded into the system before any device instantiation.
- An `.hdl` statement is not allowed inside a `.subckt` or `IF-ELSEIF-ELSE` block; otherwise, the simulation exits with an error message.
- When a module to be loaded has the same name as a previously-loaded module, or the names differ in case only, the first module is used.

In a SPICE netlist that has an `.ALTER` block, the simulator selects the current module name from the current `.ALTER` block if there is a duplicate module name (`res1`). For example:

Figure 7 Module name duplication in an `.ALTER` statement



- The `.hdl` command can be followed by variable statements surrounded by double quotation marks. For example:

```

.hdl "/tmp/design01/lib/INV.VA"
.hdl "${VA_PATH}/${DEVICE_NAME}.VA"
.hdl "../../lib/${VA_LIBRARY}.VA"

```

Where: `${VA_PATH}`, `${DEVICE_NAME}`, and `${VA_LIBRARY}` can be defined by the `setenv` UNIX command.

- The `-Ddefine=value` and `-Iinclude-path` options are allowed in the `.hdl` command in the PrimeSim HSPICE-compatible netlist. If there are two `.hdl` commands in a

netlist, the second `.hdl` command cannot accept the `-D` or `-I` option from the first `.hdl` command. For example:

```
.hdl "res.va -I/remote/hspice/compact -DXYZ=1"  
.hdl "cap.va"
```

Instantiating Verilog-A Devices

Verilog-A devices are X elements. A Verilog-A device can have zero or more nodes and can accept zero or more parameter assignments. Verilog-A devices also support the concept of a model card.

Syntax

```
X<inst> <nodes> moduleName | modelName param=value |  
  param=str('strvalue') | param=str(strparm)
```

Where, `strparm` can be defined by a `.PARAM` command.

Verilog-A module definitions are unique in each PrimeSim Continuum solution simulation. A Verilog-A module that matches the name, or differs only in case of a previously loaded module is ignored. A Verilog-A module definition is ignored if its name conflicts with a PrimeSim Continuum built-in model. Verilog-A devices parameters can be reassigned in either instance statements or model card statements (see [Using Model Cards With Verilog-A Modules](#)). Invalid parameters that are not predefined in the Verilog-A module file are ignored. The PrimeSim Continuum simulator issues warning messages on those invalid parameters.

Parameters that are defined in a Verilog-A module file are treated as device parameters to that particular module. They are not netlist parameters and thus cannot be overridden by the `.PARAM` command directly. For example:

```
*  
.param w=1u  
.hdl "nmos_va.va"  
** nmos_va is the Verilog-A module name  
** 'w', 'l' are nmos_va module parameters.  
  
x1 d g s b nmos_va w=0.35u l=0.35u  
x2 d g s b nmos_va w=w l=0.35u  
...
```

The 'w' for x1 is 0.35u and it is not overridden by the `.param` command. Only 'w' of x2 is reassigned to '1u' because its device parameter 'w' has been preassigned to a netlist parameter 'w', and thus can be overridden by a `.param` command.

The following sections discuss these topics:

- [Using Model Cards With Verilog-A Modules](#)
- [No Restrictions on Verilog-A Module Names](#)
- [Overriding Subcircuits With Verilog-A Modules](#)

Using Model Cards With Verilog-A Modules

Verilog-A modules may use a model card with similar usage to PrimeSim Continuum solution standard built-in devices. Any parameter may be specified on that card. When an instance uses a model card, it inherits the parameter values on the card and overrides those with any that may be specified directly on the instance itself. The Verilog-A module name is allowed to conflict with the following built-in device keywords:

AMP, C, CORE, D, L, NJF, NMOS, NPN, OPT, PJF, PLOT, PMOS, PNP, R, U, W, SP

The model card type should be the same as the Verilog-A module name. Every Verilog-A module can have one or more associated model cards.

Unlike built-in device model cards and instances, you can specify any module parameter in Verilog-A model cards, instance statements, or inherited parameter values from module definitions. Instance parameters always override model parameters. If the model card includes parameters that are not predefined in its associated module file, the PrimeSim Continuum simulator issues a warning message, ignores the definition, and continues with the simulation.

Syntax

```
.model mname type pname1= pname2= pname3= ...
```

Argument	Description
<i>mname</i>	User-defined model name reference. Elements must use this name to refer to this model card.
<i>type</i>	Model type. This must be the same as Verilog-A module name.
<i>pname#</i>	Parameter name. Every parameter must be predefined in its associated Verilog-A module with default parameter value set. For legibility, use either blanks or commas to separate each assignment. Use a plus sign (+) to start a continuation line.

Example

For the following examples, assume the following Verilog-A module is used:

```
module va_amp(in, out);  
    electrical in, out;  
    input in;
```

```
output out;
parameter real gain=1.0;
parameter real fc=100e6;
...
analog begin
```

Its associated model cards can then be:

```
.model myamp va_amp gain=2 fc=200e6
.model myamp2 va_amp gain=10
```

The instantiations of Verilog-A module `va_amp` are:

```
x1 n1 n2 myamp
x2 n3 n4 myamp gain=3.0
x3 n5 n6 myamp gain=2.0 fc=150e6
x4 n7 n8 myamp2 fc=300e6
x5 n9 n10 va_amp
```

- Instance `x1` inherits model `myamp` parameters (that is, `gain=2`, `fc=200e6`).
- Instance `x2` inherits `fc=200e6` from model `myamp`, but overrides `gain` with the value `3.0`.
- Instance `x3` overrides all model `myamp` parameters.
- Instance `x4` inherits parameter `gain=10` from model `myamp2`, and overrides parameter `fc`, which is an implicit parameter in `myamp2`.
- Instance `x5` does not use a model card and directly instantiates the Verilog-A module `va_amp` and inherits all module `va_amp` default parameters, which are `gain=1.0` and `fc=100e6`.

For any X element, the default search order to find the cell definition is:

1. PrimeSim Continuum solution subcircuit definition
2. PrimeSim Continuum solution model card
3. Verilog-A module definition

No Restrictions on Verilog-A Module Names

There are no restrictions on Verilog-A module names. You can use `vamodel` or `spmodel` to select the model you want.

Overriding Subcircuits With Verilog-A Modules

If both a subcircuit and a Verilog-A module have the same case-insensitive name, by default, the PrimeSim Continuum simulator uses the subcircuit definition. This

behavior can be changed by setting the `vamodel` option, either at the command line or in an `.OPTION` statement. The `vamodel` option is not supported in PrimeSim Continuum solution advanced analog analyses.

The `VAMODEL` option works on cell-based and model card definitions. Instance-based overriding is not supported.

The following sections discuss these topics:

- [Setting the -vamodel Command-Line Option](#)
- [Setting .OPTION vamodel in the Netlist](#)
- [Disabling .OPTION vamodel With .OPTION spmodel \(PrimeSim HSPICE Only\)](#)
- [File Path Syntax on `include Statement](#)
- [Using Vector Buses or “Ports”](#)
- [Using Variable-Width Buses](#)
- [Using Integer Parameters](#)
- [Implicit Parameter M Support](#)
- [Module and Parameter Name Case-Sensitivity](#)
- [M Instance Using Verilog-A](#)

Setting the -vamodel Command-Line Option

Syntax

```
-vamodel name -vamodel name2 ...
```

This command-line option is not supported in PrimeSim Continuum solution advanced analog analyses. The *name* parameter is the cell name that uses a Verilog-A definition rather than subcircuit when both are exist. Each `-vamodel` command-line option can take no more than one cell name. Repeat `-vamodel` if multiple Verilog-A modules are defined.

If no *name* parameter after `-vamodel` is supplied, then in any case the Verilog-A definition, whenever it is available, overrides the subcircuit.

The following examples show various ways to set the option and the resulting PrimeSim Continuum behavior.

Example 1

This example instructs the PrimeSim Continuum simulator to use Verilog-A definition for all instantiations of cell `vco`.

```
hspice pll.sp -vamodel vco
```

Example 2

This example instructs the PrimeSim Continuum simulator to use Verilog-A definition for all instantiations of cell `vco` and cell `chargepump`.

```
hspice pll.sp -vamodel vco -vamodel chargepump
```

Example 3

This example instructs the PrimeSim Continuum simulator to always use a Verilog-A definition whenever it is available.

```
hspice pll.sp -vamodel
```

Setting .OPTION vamodel in the Netlist

Syntax

```
.OPTION vamodel[=name]
```

This option is not supported in PrimeSim Continuum solution advanced analog analyses. The *name* parameter is the cell name that uses a Verilog-A definition rather than the subcircuit when both exist. Each `vamodel` option can take no more than one cell name. Multiple cell names require multiple `vamodel` options.

If no *name* parameter is provided for the `vamodel` option, the PrimeSim Continuum simulator uses the Verilog-A definition whenever it is available.

Example 1

```
.option vamodel=vco
```

This example instructs the PrimeSim Continuum simulator to use Verilog-A definition for all instantiations of cell `vco`.

Example 2

```
.option vamodel=vco vamodel=chargepump
```

This example instructs the PrimeSim Continuum simulator to use Verilog-A definition for all instantiations of cell `vco` and cell `chargepump`.

Example 3

```
.option vamodel
```

This example instructs the PrimeSim Continuum simulator to always use the Verilog-A definition whenever it is available.

Disabling .OPTION vamodel With .OPTION spmodel (PrimeSim HSPICE Only)

Use the `.OPTION spmodel` netlist option to switch back to the PrimeSim HSPICE definition. For example, if you override the PrimeSim HSPICE definition with the Verilog-A definition using `.OPTION vamodel`, use `.OPTION spmodel` during `.ALTER` analysis to revert to the PrimeSim HSPICE definition, which is the same as the `VAMODEL` option. The `SPMODEL` option works on cell-based definitions only. Instance-based overriding is not supported.

Syntax

```
.OPTION spmodel [=name]
```

The *name* parameter is the cell name that uses a SPICE definition. Each `spmodel` option can take no more than one cell name; multiple cell names require multiple `spmodel` options.

Example 1

This example disables the previous `.OPTION vamodel`, but has no effect on the other `vamodel` options if they are specified for the individual cells. For example, if `.option vamodel=vco` is set, the cell of `vco` uses the Verilog-A definition whenever it is available.

```
.option spmodel
```

Example 2

This example disables the previous `.option vamodel=chargepump`, which causes all instantiations of `chargepump` to reuse the subcircuit definition.

```
.option spmodel=chargepump
```

File Path Syntax on ``include` Statement

The ``include` syntax can be followed by variable statements enclosed with double quotation marks.

Example

```
`include "/tmp/design01/discipline.h"  
`include "${VA_PATH}/${DEVICE_NAME}/discipline.h"  
`include "../..lib/${VA_LIBRARY}.inc"
```

Where: `${VA_PATH}`, `${DEVICE_NAME}`, and `${VA_LIBRARY}` can be defined by the `setenv` UNIX command.

Using Vector Buses or “Ports”

The Verilog-A language supports the concept of buses (vector ports), whereas the PrimeSim Continuum solution does not. If you instantiate a module that has a vector port, the connections to individual bus signals in the PrimeSim HSPICE-compatible netlist must be specified. The Verilog-A module internally expands the vector port and connects them to the signals inside the Verilog-A module.

Example

Given a Verilog-A module with a vector port defined:

```
module d2a(in, out);  
    electrical [1:4] in;  
    electrical out;  
    analog ...  
endmodule
```

Its instantiation in the PrimeSim Continuum solution can be:

```
x1 in1 in2 in3 in4 o1 d2a
```

In this case, the nodes `in1` through `in4` are mapped to ports `in[1]` through `in[4]`, respectively. If the bus in the Verilog-A module is specified as `electrical [4:1]`, then the signals are connected as `in1` through `in4` mapped to ports `in[4]` through `in[1]`, respectively.

Using Variable-Width Buses

Verilog-A supports a variable-width bus, which is a port or a node array whose size is variable from instance to instance.

Example 1

This example defines a Verilog-A module port with a variable-width bus:

```
module varray (p, n);  
    parameter integer w=4;  
    inout [w-1:0] p;  
    inout [0:w-1] n;  
    electrical [w-1:0] p;  
    electrical [0:w-1] n;  
    parameter real I0 = 1e-6;  
    genvar i;  
    analog begin  
        for (i = 0; i < w; i = i+1) begin  
            I(p[i],n[w-1-i]) <+ I0 * (exp(V(p[i],n[w-1-i])/0.0257) - 1.0);  
        end  
    end  
endmodule
```

Its instantiation in the PrimeSim Continuum solution can be:

```
x1 1 2      0 0      varray w=2
x2 1 2 3    0 0 0    varray w=3
x3 1 2 3 4  0 0 0 0  varray w=4
```

Example 2

This example defines a Verilog-A internal node with a variable-width array:

```
module res(p,n);
    inout p, n;
    electrical p, n;
    parameter integer width=2;
    electrical [width:1] node;
    parameter real R = 1k;
    integer i;
    analog begin
        for (i = 1; i <= width; i = i + 1) begin
            I(p, node[i]) <+ V(p, node[i])/R;
            I(node[i], n) <+ V(node[i], n)/R;
        end
    end
endmodule
```

Its instantiation in the PrimeSim Continuum solution can be:

```
r1 3 2 1k
x1 2 0 res width=2
r2 3 4 1k
x2 4 0 res width=4
```

This is equivalent to two resistors connected in series with `width` and a number of branches in parallel.

Using Integer Parameters

PrimeSim HSPICE-compatible netlist parameters are all of type real. When an integer Verilog-A parameter is assigned a real value, it is rounded to the nearest integer value.

Implicit Parameter M Support

Verilog-A supports the multiplicity factor. A Verilog-A device can have parameter that is not device specific:

```
M          Multiplicity factor
```

If a loaded Verilog-A module has parameter named either “M” or “m”, then that module parameter cannot be set in the instance line. The “M” or “m” parameter in the instance line always means the “Multiplicity factor” parameter and the appropriate multiplicity factor is applied to the Verilog-A device during the simulation. The implicit device parameter scaling

factor *S* and the temperature difference between the element and circuit, *DTEMP*, are not supported.

Module and Parameter Name Case-Sensitivity

Verilog-A is case-sensitive, while the PrimeSim Continuum solution is case-insensitive. This places certain restrictions on use in terms of module and parameter names and output control.

Module Names

When you attempt to load a second module into the system with a module name that differs from a previously loaded module by case only, then the second module is ignored and a warning message is issued.

Module Parameters

Parameters in the same module with names that only differ by case cannot be redefined in either the Verilog-A instance line or Verilog-A `.MODEL` cards. The PrimeSim Continuum simulator issues an error message and exits the simulation.

Example

In this example, a simple amplifier accepts two parameters, *gain* and *Gain*, as input to the module.

```
module my_amp(in, out);
    electrical in, out;
    parameter real gain = 1.0;
    parameter real Gain = 1.0;
    analog V(out) <+ (Gain+gain)*V(in);
endmodule
```

If you instantiate this module as:

```
x1 n1 n2 my_amp Gain=1
```

The PrimeSim Continuum simulator cannot uniquely define the *Gain* parameter, so a warning message is issued and the definition of *Gain* is ignored. This module can be instantiated as is, provided neither the *Gain* nor *gain* parameter is assigned in the netlist.

M Instance Using Verilog-A

If you set `.OPTION MACMOD=1`, the M instance can be used to call Verilog-A modules.

Instantiating Primitive Elements inside Verilog-A Modules

PrimeSim Continuum Verilog-A supports the instantiation of PrimeSim Continuum primitive devices inside Verilog-A modules. You can instantiate primitive devices with the same feature set as Verilog-A modules. For example, the following is part of a PrimeSim

HSPICE-compatible netlist, which defines one `pmos` model and one `nmos` model, and calls a Verilog-A module, `inv.va`.

```
*
.param ww=50u
.hdl 'inv.va'
.model n nmos level=49 version = 3.0
.model p pmos level=49 version = 3.0
.vvdd vdd 0 3.3
.vgnd gnd 0 0
.vin in 0 pulse 0 3.3 0 1p 1p 100n 200n
.x1 in out vdd gnd inva parm_w=ww
...
```

The following is the Verilog-A module file, `inv.va`, which instantiates a PrimeSim Continuum solution primitive `nmos` device and a `pmos` device, whose model definitions are in the `.model` statements of the preceding PrimeSim HSPICE-compatible netlist. Note that the Verilog-A function `$mfactor` is used to define the M (multiplicity) factor in the hierarchical instance of the PrimeSim Continuum solution primitive MOSFET device.

```
//
`include "discipline.h"
module inva(in, out,vdd, gnd);
    input in,vdd, gnd;
    output out;
    electrical in, vdd, gnd, out;
    parameter parm_w=5.0u, parm_l=0.35u;
    n # (.w(parm_w), .l(parm_l), .$mfactor(30), .dtemp(100)) m1(out,in, gnd, gnd);
    p # (.w(10u),.l(parm_l), .$mfactor(30), .dtemp(100)) p1(out,in, vdd, vdd);
endmodule
```

Devices	PrimeSim Continuum Solution Model Levels and Comments
MOSFET	1,2,3,4,5,49,50,53,54,55,57,58,59,61,62,63,64,68,72,73
BJT	1
JFET	1
Diode	1, 2, 3 (See the following example)
Resistor Capacitor	The resistor and capacitor can be instantiated directly or via a model card. For example: <code>resistor #(.r(r)) my_dev2(p,n);</code>
Inductor	The inductor must be instantiated directly. For example: <code>inductor #(.l(l)) my_dev1(p,n);</code>

Instantiating PrimeSim Continuum Subcircuits Inside Verilog-A Modules

PrimeSim Continuum solution Verilog-A supports instantiation of PrimeSim Continuum solution subcircuits from within Verilog-A modules. Subcircuits are instantiated using the format of a Verilog-A hierarchical instantiation, using the subcircuit and parameter names defined in the PrimeSim HSPICE-compatible netlist.

For example, the following is part of a PrimeSim HSPICE-compatible netlist, which defines a subcircuit named `my_rcnetwork` with one node and two parameters. The netlist instantiates a Verilog-A module called `va_amp`. This module in turn instantiates the `my_rcnetwork` subcircuit.

```
.hdl 'modules.va'
.param local_r=50 local_c=1p
.subckt my_rcnetwork(n1) r=local_r p=local_c
R1 n1 0 r
C1 n1 0 c
.ends
vin in 0 pulse 0 3.3 0 1p 1p 100n 200n
x1 in out va_amp gain=2 r_filter=25 c_filter=2p
...
```

The following is the Verilog-A module file, `modules.va`, which defines the module `va_amp`.

```
`include "discipline.h"
module va_amp(in, out);
input in;
output out;
electrical in, out;
parameter real gain=1.0;
parameter real r_filter=50;
parameter real c_filter=1p;
// Instantiate the HSPICE subcircuit:
my_rcnetwork #(.r(r_filter), .c(c_filter))
  my_filter1(in);
my_rcnetwork #(.r(r_filter), .c(c_filter)) my_filter2(in);
analog
  V(out) <+ gain * V(in);
endmodule
```

The subcircuit `my_rcnetwork` is instantiated twice, at node `in` and node `out`. The input parameters to the Verilog-A module, `r_filter` and `c_filter`, are passed to the subcircuit `my_rcnetwork` parameters `r` and `c` using the Verilog-A parameter passing syntax.

Output Simulation Data

Verilog-A devices support the same output capabilities as built-in devices. Using PrimeSim Continuum solution output statements such as `.PRINT`, `.PROBE`, and `.DOUT`, you can access the following Verilog-A device quantities:

- Port current
- Port voltage
- Internal node voltage (PrimeSim HSPICE only)
- Internal named branch current (PrimeSim HSPICE only)
- Internal module variables (PrimeSim HSPICE only)
- Module parameters (PrimeSim HSPICE only)

The following sections discuss these topics:

- [V\(\) and I\(\) Access Functions](#)
- [Output Bus Signals](#)
- [Output Internal Module Variables](#)
- [Output Module Parameters](#)
- [Case-Sensitivity in Simulation Data Output](#)
- [Using Wildcards in Verilog-A \(PrimeSim HSPICE Only\)](#)
- [Port Probing and Branch Current Reporting Conventions](#)
- [Unsupported Output Function Features](#)

V() and I() Access Functions

You can access port voltage and internal node voltage of Verilog-A devices via the `V()` function. Port current and internal branch currents can be accessed via the `I()` function.

The internal nodes of Verilog-A devices are accessible via the `V()` function when the full hierarchical name is provided. The port current and named branches (on the instance base only) can be accessible via the `I()` function.

Examples

In the following examples, assume the Verilog-A module definition fragment is:

```
module va_fnc(plus, minus);  
  inout plus, minus;
```

```
electrical plus, minus;  
electrical int1, int2;  
branch (int1, int2) br1;  
    //creates an internal branch br1 between internal  
    //nodes int1 and int2;  
  
analog begin
```

The Verilog-A module may be instantiated in the netlist as:

```
x1 1 2 va_fnc
```

To print the current on Verilog-A device port name `plus` for the instance `x1`:

```
.print I(x1.plus)
```

The `plus` is the port name defined in Verilog-A module, not the netlist node name.

To print the Verilog-A module internal node named `int1` for the instance `x1`:

```
.print V(x1.int1)
```

If the `va_fnc` module is hierarchical and has a child instance called `c1` with an internal node `int1`, then the node `int1` can be output as:

```
.print V(x1.c1.int1)
```

That is, the full PrimeSim Continuum instance name is concatenated with the full internal Verilog-A instance name to form the complete name.

You can probe branch current with PrimeSim Continuum output commands. In the previous Verilog-A module, there is an internal branch name `br1` declared. To probe the branch current:

```
.print I(x1.br1)
```

Output Bus Signals

Verilog-A bus signals can be accessed with PrimeSim Continuum output commands using the Verilog-A naming and accessing conventions.

Example

Given an example Verilog-A module:

```
module my_bus(in, out);  
    electrical in;  
    electrical [1:4] out;  
    ...  
endmodule
```

And instantiated in the netlist as:

```
x1 1 2 3 4 5 my_bus
```

Then the values at vector port out can be output by explicitly listing each position:

```
.print v(x1.out[1]), v(x1.out[2]), v(x1.out[3]), v(x1.out[4])
```

Bus elements can also be specified using wildcards, as described in [Using Wildcards in Verilog-A \(PrimeSim HSPICE Only\)](#).

Output Internal Module Variables

Note:

This section applies only to: PrimeSim HSPICE, PrimeSim Pro, and PrimeSim SPICE.

Verilog-A internal variables, by default, are hidden from output. However, module variables with a description or units attribute, or both, are known as output variables, and the PrimeSim HSPICE tool provides access to their values. For example, suppose a module for a MOS transistor with the following declaration at module scope provides the output variable `cgs`:

```
(* desc="gate-source capacitance", units="F" *) real cgs;
```

The `cgs` module variable can be printed just like a normal parameter variable.

Syntax

Instance: `internal_variable`

Example

```
.print xva_vco:freq
```

This example outputs internal variable frequency value of Verilog-A instance `xva_vco`.

Output Module Parameters

Note:

This section applies only to: PrimeSim HSPICE, PrimeSim Pro, and PrimeSim SPICE.

You can apply PrimeSim HSPICE element templates to PrimeSim Continuum primitives that are instantiated in Verilog-A modules. The element templates are found in “Element Template Listings” in the *PrimeSim™ HSPICE® User Guide: Basic Simulation and Analysis*.

Example

For a netlist instantiation of a Verilog-A device:

```
. model mod1 npn beta=222
X1 c b 0 0 va_wrapper area=0.25 m=2
...
.PRINT TRAN X1.Q1:BETA LV2(X1.Q1)
```

where the Verilog-A source code definition of a module name `va_wrapper`:

```
`include "disciplines.vams"
module va_wrapper(c,b,e,s);
electrical c,b,e,s;
parameter real area=1.0;
mod1 #(.area(area)) q1(c,b,e,s);
endmodule
```

This example outputs the `BETA` value and the `icvbe` value (template element alias `lv2`) for the Verilog-A instance `x1` that in turn instantiated a primitive BJT of the model `mod1` in an instance `Q1`.

.PROBE/.PRINT Commands in the PrimeSim Continuum Solution

The following table shows the PrimeSim Continuum command syntax for the `.PROBE/.PRINT` Verilog-A values.

Values	PrimeSim Continuum Identifier
Port/node voltage	<code>V(inst_hierarchy.name)</code>
Port current	<code>I(inst_hierarchy.name)</code>
Branch voltage	<code>V(inst_hierarchy.bname)</code>
Branch current	<code>I(inst_hierarchy.bname)</code>
Variable	<code>inst_hierarchy:var_name</code> (Note colon separator) Examples: <code>x1.x2:signal</code> <code>x1.x2.x3:signal</code>
Parameter	<code>inst_hierarchy:par_name</code> (Note colon separator)

Case-Sensitivity in Simulation Data Output

When Verilog-A information is output via the PrimeSim Continuum output commands, the case of the node names associated with the quantities to be output is ignored. Contributions from the Verilog-A noise sources that have the same name when case is ignored are combined.

Example

```
I(d,s) <+ white_noise(4*k*T/R1, "thermalnoise");  
I(d2,s2) <+ white_noise(4*k*T/R2, "ThermalNoise");
```

The two noise contributions are combined into one contribution called `thermalnoise` in the output files.

Using Wildcards in Verilog-A (PrimeSim HSPICE Only)

Verilog-A names support the use of wildcards to simplify the use of the output commands.

Examples

Given the Verilog-A module:

```
module test(p,n);  
  electrical p,n;  
  electrical int1, int2;  
  ...  
endmodule
```

Instantiated as:

```
x1 1 2 test
```

Then all of the internal nodes (in this case `int1` and `int2`) can be printed using the command:

```
.print v(x1.*)
```

All indexes of a bus in the module:

```
module my_bus(in, out);  
  electrical in;  
  electrical [1:4] out;  
  ...  
endmodule
```

Can be specified as:

```
x1 1 2 3 4 5 my_bus  
.print v(x1.out[*])  
.print v(x1.*)
```

Both of the internal nodes, `int1` and `int2`, for the child `ch1` in the instance `x_par1` can be specified using:

```
.print v(x_par1.ch1.int*)
```

The PrimeSim Continuum `.OPTION POST` command does not output internal nodes from Verilog-A modules. Use the wildcard feature to specify a Verilog-A instance if you have to output all internal nodes.

Port Probing and Branch Current Reporting Conventions

When printing and reporting currents for Verilog-A devices, the PrimeSim Continuum solution follows the same conventions when specifying the direction of current flow as in built-in devices. A positive branch current implies that current is flowing into the device terminal or internal branch.

Unsupported Output Function Features

The following output functions are not supported:

- Port probing: `In()`, where *n* is the node number. Instead, you can use `I(instance.port_name_in_module)`.
- `Iall()`: Instead, you can output all the terminal currents using a wildcard.
- `P()` and `Power`: Instead, you can use the `$strobe` Verilog-A function. That is, using `$strobe` is the Verilog-A equivalent of using `.PROBE TRAN POWER` for the total dissipated power.
- Nodal capacitance
- Group delay
- Direct current probing on MOSFET elements that are instantiated inside Verilog-A modules
- Element template output on MOSFET elements that are instantiated inside Verilog-A modules

SPICE Netlist and Verilog-A Interactions

The following sections discuss these topics:

- [Passing .OPTION EXPMAX = value](#)
- [Passing the Parameter String](#)
- [Passing PrimeSim HSPICE-Compatible Netlist to Verilog-A](#)
- [Passing the M-Factor](#)
- [Illegal Format for \\$strobe](#)
- [Illegal Format for \\$fscanf](#)

Passing .OPTION EXPMAX = value

Set the option `EXPMAX` to increase the linearization of the Verilog-A `exp()` function to start after the limit (value) is set with that option. In the example, `.OPTION EXPMAX=200`, the value is 200.

```
analog begin
    I(d,s) <+ exp(300)
end
```

In other simulators, this option is equivalent to the option `VAMAXEXP` used in Verilog-A only. But in the PrimeSim Continuum solution, the `.OPTION EXPMAX=value` works in both the PrimeSim HSPICE-compatible netlist and Verilog-A.

Passing the Parameter String

You can pass the parameter string from the PrimeSim HSPICE-compatible netlist to Verilog-A through the SPICE instance. But the variable (`file1`, 2) with the declared string should be preceded by the `str` function in the PrimeSim Continuum instance.

```
.hdl "vpwlf_param.va"
.PARAM
+     file1=str('input1.pw1')
+     file2=str('input2.pw1')

xv0 vout0 0 vpwlf_param file_name=str(file1) DELAYT=0 VOFFSET=0
xv1 vout1 0 vpwlf_param file_name=str(file1) DELAYT=0 VOFFSET=0
....
....
xv2 vout2 0 vpwlf_param file_name=str(file2) DELAYT=0 VOFFSET=0
xv3 vout3 0 vpwlf_param file_name=str(file2) DELAYT=0 VOFFSET=0
....
```

In the Verilog-A module:

```
module vpwlf_param (a, b);
```

To receive the string from the PrimeSim Continuum instance, you must declare the parameter string:

```
parameter string file_name="input.pw1";
```

Passing PrimeSim HSPICE-Compatible Netlist to Verilog-A

You can pass the array parameter from the PrimeSim HSPICE-compatible netlist to Verilog-A.

Syntax

```
.param param_name = array(val1,val2, ...)
```

Use either one of the syntax in the following example:

```
Xname n1 n2 psrc pattern_array = param_name
```

or

```
xname n1 n2 psrc pattern_array = array(val1, val2...)
```

In the Verilog-A file:

```
parameter integer pattern_array[3:0] = {1, 2, 3, 4}
```

The array size of *param_name* in SPICE must be the same as that (*pattern_array*) defined in Verilog-A.

Passing the M-Factor

Portable Verilog-A (pVA) handles the issue of passing from the M-factor in a SPICE netlist to the Verilog-A parameter real (**inherited_mfactor**) *m=2*, parameter real *m*, and *\$mfactor* as shown in the following example:

```
.option accurate post=1  
.hdl "cap.va"  
r1 1 2 1k
```

```
x1 2 0 cap m=4 ---- netlist parameter m
```

```
v1 1 0 pw1 0 0 10n 1 20n 1 30n 0  
.tran 1p 30n  
.probe v(*)  
.end
```

```
module cap(vp, vn); --- cap.va  
  inout vp, vn;  
  electrical vp, vn;  
  parameter real (* integer inherited_mfactor; *) m=2 ---- case 2  
  or  
  parameter real m=2 ---- case 3  
  parameter real c = 1e-12;  
  real qocm;  
  analog begin  
    qocm = c*m; I(vp, vn) <+ ddt(qocm*V(vp,vn));  
    $strobe("XYZ qocm=%g $mfactor=%g m=%g", qocm, $mfactor,m );  
  end  
endmodule
```

Following the preceding example, to enable the multiplicity factor, use the UNIX system command `setenv` with one of the following methods:

- [Method 1: `setenv PVA\_INHERIT\_MFACTOR 0`](#)
- [Method 2: Default](#)
- [Method 3: `setenv PVA\_MULTIPLIER 1`](#)

Method 1: `setenv PVA_INHERIT_MFACTOR 0`

1. Enable the simulator multiplicity factor.
2. Enable the Verilog-A multiplicity factor, if `m` is used in contribution.
3. Verilog-A Parameter `m` is treated as a local parameter in cases 2 and 3.
4. pVA outputs the result if `setenv PVA_INHERIT_MFACTOR 0`.
5. Netlist instance parameter `m` is passed to `$mfactor`.
6. Both parameters, `M` and `m`, are treated as the same parameter `m`.

					Results in Verilog-A PrimeSim Continuum
Case #	Netlist Instance <code>m=</code>	Verilog-A Parameter <code>m=</code>	Parameter <code>m</code> in Verilog-A	<code>m</code>	<code>\$mfactor</code>
2	4	(*inherited_mfactor *) <code>m=2</code>	Verilog-A parameter <code>m</code> is treated as a local parameter	2	4
3	4	2	Verilog-A parameter <code>m</code> is treated as a local parameter	2	4

Method 2: Default

1. Enable the simulator multiplicity factor.
2. Enable the Verilog-A multiplicity factor, if `m` is used in contribution.
3. Netlist instance parameter `m` overrides Verilog-A parameter `m` in cases 2 and 3.
4. Verilog-A parameter `m` is treated as a local parameter in cases 2 and 3.
5. Netlist instance parameter `m` is passed to `$mfactor`.
6. Both parameters, `M` and `m`, are treated as the same parameter `m`.

Case #	Netlist Instance m=	Verilog-A Parameter m=	Parameter m in Verilog-A	Results in Verilog-A PrimeSim Continuum	
				m	\$mfactor
2	4	(*inherited_mfactor *) m=2	Netlist instance parameter m overrides Verilog-A parameter m	4	4
3	4	2	Verilog-A parameter m is treated as a local parameter	2	4

Method 3: setenv PVA_MULTIPLIER 1

1. Disable the simulator multiplicity factor.
2. Enable the Verilog-A multiplicity factor, if m is used in contribution.
3. Netlist instance parameter m overrides Verilog-A parameter m.
4. pVA outputs the result if `setenv PVA_MULTIPLIER 1`.
5. `$mfactor` is always set to 1.
6. Uppercase parameter M is not treated as the same parameter m in Method 3 in pVA.

Case #	Netlist Instance m=	Verilog-A Parameter m=	Parameter m in Verilog-A	Results in Verilog-A PrimeSim Continuum	
				m	\$mfactor
2	4	(*inherited_mfactor *) m=2	Netlist instance parameter m overrides Verilog-A parameter m	4	1
3	4	2	Netlist instance parameter m overrides Verilog-A parameter m	4	1

Illegal Format for \$strobe

In the following example, the declared integer must print the `%d` format instead of `%s`.

```
integer error_bin[9:0];
$strobe( "--- %m : CHECK:@%.3fns, where wrong [9:0]
%s%s_%s%s%s%s_%s%s%s%s",
$abstime/1e-09,
```

```
error_bin[9], error_bin[8], error_bin[7], error_bin[6],
error_bin[5], error_bin[4], error_bin[3], error_bin[2],
error_bin[1], error_bin[0]);
```

The `*pvaE*` format `%s` does not match the argument type (`$strobe`).

Illegal Format for \$fscanf

Format specification `%b` is defined for display only. For example, the following is illegal:

```
integer symb;
$fscanf (fd, "%b", symb);
```

The following is a legal example (see [Case 6: Using \\$fscanf With the String Format to Read a Bit File](#)).

```
string symb;
$fscanf (fd, "%s", symb);
```

Encrypting Verilog-A Files

Verilog-A module files are supported for encryption only when using 8-byte key and Triple DES private or random key. The following information provides detailed examples of using encryption with Verilog-A.

The following keyword letters are used to represent different case scenarios in [Table 9](#):

- **d**: With the `-d` option (`metaencrypt` command option)
- **p**: With a `.prot` and `.unprot` block in Verilog-A file
- **no**: No option `-d` and/or `.prot` and `.unprot` block in Verilog-A file
- **dp**: With the `-d` option and with a `.prot` block the in the Verilog-A file
- **nodp**: Without the `-d` option and with `.prot` block in the Verilog-A file
- **dnop**: With the `-d` option and without a `.prot` block in the Verilog-A file
- **nodnop**: Without `-d` option and without a `.prot` block in the Verilog-A file

Table 9 *Encrypting Verilog-A Files*

Case	Encryption Method	PrimeSim Continuum / pVA
1	dp/8byte_key	Supported
2	dp/traditional_freelib	Unsupported
3	dp/triple_des_priv	Supported

Table 9 *Encrypting Verilog-A Files (Continued)*

Case	Encryption Method	PrimeSim Continuum / pVA
4	dp/triple_des_rand	Supported
5	nodp/8byte_key	Supported
6	nodp/traditional_freelib	Unsupported
7	nodp/triple_des_priv	Supported
8	nodp/triple_des_rand	Supported
9	dnop/8byte_key	Unsupported
10	dnop/traditional_freelib	Unsupported
11	dnop/triple_des_priv	Unsupported
12	dnop/triple_des_rand	Unsupported
13	nodnop/8byte_key	Supported
14	nodnop/traditional_freelib	Unsupported
15	nodnop/triple_des_priv	Supported
16	nodnop/triple_des_rand	Supported

Note:

Here are some important notes:

1. The PrimeSim HSPICE-compatible commands `.PROT` and `.UNPROT` are case-insensitive in the Verilog-A file.
2. Unlike the PrimeSim Continuum solution, which always requires the netlist to begin in column 1, the Verilog-A file can be free form (beginning in any column).
3. You can comment out any `.PROT` and `.UNPROT` block by using the double forward slash (`// .PROT`).
4. You can mix Verilog-A encrypted and non-encrypted input in a PrimeSim HSPICE-compatible netlist.

Encryption Examples

This section provides encryption examples for some of the cases described in [Table 9](#). See also [Library and Data Encryption](#).

Case 1: dp/8byte_key

```
metaencrypt -i simple.va -o simple.inc -t AbCd12Eg -d VA
hspice testbench.sp -o testv
testbench.sp:
```

```
* .hdl simple.va  $-- original file
.hdl simple.inc   $-- encryption file
.....
.....
.end
```

simple.va:

```
`include "disciplines.vams"
`include "constants.vams"
module res(plus,minus);
    inout plus, minus;
    electrical plus,minus;
    .PROT
    parameter integer vvdd = 1;
    analog begin
        V(plus,minus) <+ 1*I(plus,minus);
        $strobe("=CasE= %d => simple.va\n", vvdd);
    end
    .UNPROT
endmodule
```

simple.inc:

```
`include "disciplines.vams"
`include "constants.vams"
module res(plus,minus);
    inout plus, minus;
    electrical plus,minus;
    .inc "VA/simple.va.enc1"
endmodule
```

VA/simple.va.enc1:

```
.PROT v200102
^@<88><88><88><88><82>Ã<82>Â¿RKf<9d>^@^@^@r{0Â¢Â³Ã^Vt^@^@^@Â¿RKf<9d>Â¿Ã$<
8c>Â¿x<83>Ã@^U^@^A^Dh9>W^?^WÃ XÃ
```

In the Case 1 example:

- metaencrypt -i simple.va -o simple.inc -t AbCd12Eg -d VA is the command-line invocation.
- simple.va is the original file, without encryption.
- simple.inc is the encrypted file.

In the Case 7 example:

- `metaencrypt -i simple.va -o simple.inc -t privkey 0123456789ABCDEF9876543210FEDCBA1357924680ACEBDF` is the command-line invocation for `triple_des_priv` using the private key.
- `hspice testbench.sp -o testv` launches the PrimeSim Continuum simulation with the encrypted Verilog-A file, `simple.inc`.
- `simple.va` is the original file, without encryption.
- `simple.inc` is the encrypted file.
- `simple.va` is totally replaced by `simple.inc`.

Case 16: nodnop/triple_des_rand

```
metaencrypt -i simple.va -o simple.inc -t randkey
hspice testbench.sp -o test
```

testbench.sp:

```
***.hdl simple.va
.hdl simple.inc
v1 1 0 10
x1 1 0 res
.tran 10n 100n
.print tran i(v1)
.end
```

simple.va:

```
`include "disciplines.vams"
`include "constants.vams"
module res(plus,minus);
inout plus, minus;
electrical plus, minus;
parameter integer vvdd = 1;
analog begin
    V(plus,minus) <+ 1*I(plus,minus);
    $strobe("=CasE= %d => simple.va\n", vvdd);
end
endmodule
```

simple.inc:

```
.PROT RANDKEY
1ü S Å%aî![úöð8_Üs5öæpÓqJT '...¿&#VA\ýì*íO*% Z"lq"÷`£"ô8Ü¥mçÈ&,¹\ - # @REA_1
Åñi (³*p±æcp. ábz^Óž™ø÷$ 4V`aÂMò "6pnoª WPiÜ[ÿ© y]²p.¹Ežù¢47ð-# "
ô'½Ñ'b~E,áŮWââžik'...2- @7 XÑe pR ôÄfÚª`1î
```

In the Case 16 example:

- `metaencrypt -i simple.va -o simple.inc -t randkey` is the command line invocation for `triple_des_rand` using the random key.
- `hspice testbench.sp -o testv` launches the PrimeSim Continuum simulation with the encrypted Verilog-A file, `simple.inc`.
- `simple.va` is the original file, without encryption and without a `.PROT/.UNPROT` block.
- `simple.inc` is the encrypted file.
- `simple.va` is totally replaced by `simple.inc`.

Compiling Verilog-A Files

Verilog-A modules used in PrimeSim Continuum simulations are automatically compiled and cached in the current working directory by the simulator. You can compile files manually if you want (to check syntax only of Verilog-A).

General pVA Command-Line Syntax

```
pva VA-file(s) [-o outlib] [-Ddefine=val] [-Iinclude-path]
[-hdlpath pathname]
```

Where:

- `pva`: Indicate scripts to run on 64-bit platforms.
- `VA-file(s)`: `*.va`, `*.vams`, or `1.va 2.va` files.
- `-o out`:
 - pVA creates the `out.pvadir` directory in which the intermediate files reside.
 - The default directory is `pvadir` if there is no `-o` option input.
- `-hdlpath pathname`
- `-Ddefine=val` applies `define` name to all input VA preprocessors.
- `-Iinclude-path` adds a search path of the header files for input Verilog-A files; otherwise the default include paths are applied (`$HSP_HOME/include`).

Supported LRM 2.4 Syntax and Features

Note:

The PrimeSim Continuum solution supports LRM 2.4 features on a case-by-case basis. If you require additional LRM 2.4 feature support, contact the Synopsys Technical Support Center.

See [Customer Support](#) in this manual.

The PrimeSim Continuum solution currently supports the following LRM features (Version 2.4, June 1, 2014):

- [How pVA Handles Strings](#)
- [String Data Type](#)
- [Module Port With Discipline](#)
- [\\$simparam\\$str Parameter System Function](#)
- [File Input-Output System Tasks and Functions](#)
- [cross, above, and timer With Support for enable](#)
- [Parameter Arrays](#)
- [Parameter Value Range](#)
- [Passing Array to Analog Function](#)
- [Analog Initial](#)
- [\\$arandom](#)
- [\\$simprobe](#)
- [\\$finish_current_analysis](#)
- [Supporting Semicolon \(;\) Symbols at the End of Nature and Discipline Identifiers](#)
- [Supporting All Types of Integer Constants as Defined by the VAMS LRM 2.4](#)

How pVA Handles Strings

The following cases describe different ways that pVA handles strings:

- [Case 1: Parameter String](#)
- [Case 2: String Variables](#)

- [Case 3: Concatenating String Parameters](#)
- [Case 4: Length of String—string.len\(\)](#)
- [Case 5: String functions \\${pattern}.{len\(\)}substr\(, \),atoi\(\)}](#)
- [Case 6: Using \\$fscanf With the String Format to Read a Bit File](#)

Case 1: Parameter String

For example, in the first line, pVA treats the `patha` type as a string.

```
parameter string patha="./xyz";  
parameter pathb="./xyz";
```

In the second line, pVA treats `pathb` as a string because it is enclosed within double quotation marks, even if it is not declared as a string.

Case 2: String Variables

```
string tbl_fn;  
tbl_fn="tbl.tbl";
```

For example:

```
module myva(oo, ii);  
output oo;  
input ii;  
electrical ii;  
electrical oo;  
string tbl_fn;  
analog begin  
    @(initial_step) begin  
        tbl_fn = "tbl.tbl";  
    end  
    V(oo) <+ $table_model(V(ii), tbl_fn, "1CC");  
end  
endmodule
```

Case 3: Concatenating String Parameters

The following is a SPICE netlist:

```
x0 a b eva_test datapath=str("data1") parameter datapath="";
```

pVA automatically decides the string type based on the double quotation marks ("") if there is not a string declared.

```
parameter string datapath="";  
parameter string dataname="data.tbl";string filename;  
analog begin  
    filename={datapath,daname};
```

```
    $display("file=%s ", filename);  
end
```

The resulting file name of the concatenated file is:

```
data1data.tbl
```

Case 4: Length of String—string.len()

```
Parameter string bit_string="00110011100000111010";  
J=(J+1)%string.len(bit_string);
```

Verilog-AMS LRM 2.4 states “Verilog-AMS includes the string data type from IEEE std 1800-2005 SystemVerilog.” The implementation of string length query function, `string.len()`, is derived from the IEEE standard and satisfies the requirement of LRM 2.4.

Case 5: String functions `${pattern}.${len( )|substr( , ),atoi( )}`

```
module bitstream (dout, doutb, cm);  
    ...  
    ...  
    parameter string bits = "00111110101100000101";  
    string strtemp;  
    integer length, position, bit;  
    analog begin  
        @(initial_step) begin  
            length=bits.len( );  
            ...  
            position=0;  
        end  
        ...  
        ...  
        begin  
            strtemp= bits.substr(position, position+1-1);  
            bit = strtemp.atoi( );  
            ...  
        end  
    end  
endmodule
```

Case 6: Using `$fscanf` With the String Format to Read a Bit File

The following example uses a `10bit.dat` file which contains:

```
1010110011
```

```
1110111101 ...
```

```
// This model loads 256 10-bit symbols from 10bit.dat file  
module test (dout, doutb, clk);  
    input clk;  
    output dout, doutb;
```

```
electrical clk, dout, doutb;
parameter string fpat = "10bit.dat";
parameter integer sym_totnum = 252 from [1:256];
parameter integer lsbfirst = 1 from [0:1];
integer fd;
integer pat[0:2559];
integer sym_num, pbit_num, sbit_num;
string symb;
analog begin
    @(initial_step) begin
        fd = $fopen(fpat, "r");
        for (sym_num = 0; sym_num < sym_totnum;
            sym_num = sym_num + 1) begin
            $fscanf(fd, "%s", symb);
            $strobe("[test] symbol[%d] = \t[%s]", sym_num, symb);
            for (sbit_num = 0; sbit_num < 10;
                sbit_num = sbit_num + 1) begin
                pbit_num = sym_num * 10 + lsbfirst
                    * sbit_num + (1 - lsbfirst) * (9-sbit_num);
                case (symb[sbit_num])
                    "0" : pat[pbit_num] = 0;
                    "1" : pat[pbit_num] = 1;
                endcase
            end
        end
        $fclose(fd);
    end
end //analog
endmodule
```

String Data Type

pVA supports the string data type. See Section 3.3 in the Accellera *Verilog-AMS Language Reference Manual, Analog & Mixed-Signal Extensions to Verilog HDL*, Version 2.3.1, June 1, 2009.

```
string s1 = "hello world";
```

Module Port With Discipline

pVA supports the module port with discipline. See Section 6.6.2 in the Accellera *Verilog-AMS Language Reference Manual, Analog & Mixed-Signal Extensions to Verilog HDL*, Version 2.3.1, June 1, 2009.

```
module nlres (inout electrical a, inout electrical b);
```

\$simparam\$str Parameter System Function

pVA supports the \$simparam\$str parameter, which is similar to \$simparam. It is used for returning string-valued simulation parameters. The following table provides the string parameter names supported by \$simparam\$str.

See Section 9.15 in the Accellera *Verilog-AMS Language Reference Manual, Analog & Mixed-Signal Extensions to Verilog HDL*, Version 2.3.1, June 1, 2009.

String	Description
analysis_name	The name of the current analysis, for example, TRlastTP
analysis_type	The type of the current analysis, for example, dc, tran, and ac
cwd	The current working directory in which the simulator was started.
module	The name of the module from which \$simparam\$str is called.
instance	The hierarchical name of the instance from which \$simparam\$str is called
path	The hierarchical path to the \$simparam\$str function

Example

```

module testbench;
    dut dut1;
endmodule

module dut;
    analog begin
        begin : BLOCK
            $strobe ("%s\n%s\n%s\n%s\n%s\n%s\n%s",
                    $simparam$str("analysis_name"),
                    $simparam$str("analysis_type"),
                    $simparam$str("cwd"),
                    $simparam$str("module"),
                    $simparam$str("instance"),
                    $simparam$str("path"));
        end
    end
endmodule

```

The preceding example produces the following results:

```
TRlastTP
TRAN
/users/test/LRM231/simparam_str
dut
x1.dut1
x1.dut1.BLOCK
```

File Input-Output System Tasks and Functions

pVA supports the `$fwrite`, `$sformat`, `$fgets`, `$sscan`, `$fseek`, `$ftell`, `$rewind`, and `$feof` input-output system tasks and functions.

cross, above, and timer With Support for enable

```
timer (start_time [,period [,time_tol [,enable]]])
```

See Section 5.10.3.1 for `cross`, Section 5.10.3.2 for `above`, and Section 5.10.3.3 for `timer` in the Accellera *Verilog-AMS Language Reference Manual, Analog & Mixed-Signal Extensions to Verilog HDL*, Version 2.3.1, June 1, 2009.

Parameter Arrays

The following is an example of a parameter array:

```
parameter real poles[0:3] = '{ 1.0, 3.198, 4.554, 2.00 };
```

See Section 3.4.4 in the Accellera *Verilog-AMS Language Reference Manual, Analog & Mixed-Signal Extensions to Verilog HDL*, Version 2.3.1, June 1, 2009.

Parameter Value Range

pVA supports the colon (:) character only for the parameter integer and real type in value range. However the comma (,) character can be used for the string type in value range. See Section 3.4.2 “Value Range Specification” in the Accellera *Verilog-AMS Language Reference Manual, Analog & Mixed-Signal Extensions to Verilog HDL*, Version 2.3.1, June 1, 2009.

Example

```
parameter integer ipar = 1 from [1:255];
parameter real rpar = 1.0 from (0:255);
parameter string ttype = "NMOS" from `{ "NMOS", "PMOS" };

param_assignment ::=
parameter_identifier = constant_mintypmax_expression { value_range }
| parameter_identifier range = constant_arrayinit { value_range }
```

```
value_range ::=
value_range_type ( value_range_expression : value_range_expression )
| value_range_type ( value_range_expression : value_range_expression ]
| value_range_type [ value_range_expression : value_range_expression )
| value_range_type [ value_range_expression : value_range_expression ]
| value_range_type '{ string { , string } }
| exclude_constant_expression
```

Passing Array to Analog Function

```
analog function integer array_mult;
    inout q[0:`ARRAY_SIZE];
    integer q[0:`ARRAY_SIZE];
    integer i;
    begin
        for (i = 0; i <= `ARRAY_SIZE; i = i + 1) begin
            q[i] = q[i] * 2;
        end
        array_mult = i;
    end
endfunction
integer x[0:`ARRAY_SIZE];
integer i, retval;
analog begin
    for (i = 0; i <= `ARRAY_SIZE; i = i + 1) begin
        x[i] = i;
    end
    retval = array_mult(x);
    for (i = 0; i <= `ARRAY_SIZE; i = i + 1) begin
        $strobe("x[%d] = %d\n", i, x[i]);
    end
    I(p,n) <+ V(p,n) / r;
end
```

See Section 4.7.1 in the Accellera *Verilog-AMS Language Reference Manual, Analog & Mixed-Signal Extensions to Verilog HDL*, Version 2.3.1, August 4, 2009.

Analog Initial

```
analog initial begin
for (i = 0; i < NUM_BITS; i = i + 1) begin
    if (((sel_val >> i)%2) == 1) begin
        v_tmp[i] = vhigh;
    end else begin
        v_tmp[i] = vlow;
    end //else
end //for loop
end //initial_step
```

See Section 5.2.1 in the Accellera *Verilog-AMS Language Reference Manual, Analog & Mixed-Signal Extensions to Verilog HDL*, Version 2.3.1, August 4, 2009.

\$arandom

See Section 9.13.1 in the Accellera *Verilog-AMS Language Reference Manual, Analog & Mixed-Signal Extensions to Verilog HDL*, Version 2.3.1, August 4, 2009.

\$simprobe

pVA supports the `$simprobe` system task function for the probing of values during the simulation.

See Section 9.16 in the Accellera *Verilog-AMS Language Reference Manual, Analog & Mixed-Signal Extensions to Verilog HDL*, Version 2.3.1, August 4, 2009.

\$finish_current_analysis

```
finish_current_analysis_task ::= $finish_current_analysis [ ( n ) ] ;
```

This causes the simulator to stop the current transient analysis without exiting the simulation and continue running the remaining analysis.

See Section 9.7.1 for the `n` parameter meaning in the Accellera *Verilog-AMS Language Reference Manual, Analog & Mixed-Signal Extensions to Verilog HDL*, Version 2.3.1, August 4, 2009.

Note:

The `$finish_current_analysis` parameter can be used to stop only the transient analysis and not any other analysis type.

Supporting Semicolon (;) Symbols at the End of Nature and Discipline Identifiers

```
discipline electrical;  
  domain continuous;  
  potential Voltage;  
  flow Current;  
enddiscipline
```

```
discipline discrete;  
  domain discrete;  
enddiscipline;
```

See Pages 37, 327, and Sections A.1.6 Nature Declaration, A.1.7 Discipline Declaration in the Accellera *Verilog-AMS Language Reference Manual, Analog & Mixed-Signal Extensions to Verilog HDL*, Version 2.3.1, August 4, 2009.

Supporting All Types of Integer Constants as Defined by the VAMS LRM 2.4

Examples:

```
integer dummy;

dummy = 5; // Standard decimal number
dummy = 'b110; // Unsigned binary number
dummy = 'b 110; // Unsigned binary number
dummy = 4'b1001; // 4-bit binary number
dummy = 4 'b1001; // 4-bit binary number
dummy = 16'b0011_0101_0001; // hexadecimal number
dummy = 32 'h 12ab_f001; // hexadecimal number
dummy = 16 'h ca85; // hexadecimal number
dummy = 'habc; // hexadecimal number
dummy = 'h abc; // hexadecimal number
dummy = 4'shf ; // = ( -4'h 1 ) interpreted as 2's complement => -1
dummy = -4 'sd15 ; // = +1 [ = - ( -4'h 1 ) ]
```

Sharing of Precompiled Verilog-A Files by Multiple Users

Use the following procedure to allow precompiled Verilog-A files to be shared by multiple users.

The environment variable `PVA_GENSO` generates the compiled Verilog-A code that can be reused by different users, when you set the value of the environment variable to 1.

That is, to share precompiled Verilog-A files, use the following command:

```
% setenv PVA_GENSO 1
```

After the simulation is complete, pVA keeps all the intermediate files in the `design_name.pvadir` directory. Do not delete this directory.

To share or reuse precompiled Verilog-A files, you should also set the environment variable `PVA_USESO` to the full path to the compiled Verilog-A files, using the following command:

```
% setenv PVA_USESO /directory_path/design_name.pvadir
```

Note:

The user sharing/reusing the precompiled Verilog-A files must use the same version of PrimeSim HSPICE, PrimeSim XA, or PrimeSim that was used to create the files.

Performance Considerations With the PrimeSim Continuum Solution

The following sections discuss these topics:

- [Environment Variables for Forcing Compilation of Verilog-A Modules](#)
- [Reusing Precompiled Verilog-A Files in Simultaneous Multiple Simulations](#)
- [pVA Compiles ALL Modules in an .hdl Statement](#)
- [pVA Time Derivative Function Performance Tuning Tip](#)

Environment Variables for Forcing Compilation of Verilog-A Modules

The following environment variables can be used to force pVA to compile Verilog-A modules to generate the runtime library `pvaRTL_platform.so`, when the internal default is not sufficient for long simulations.

Environment Variable	Value	Comment
PVA_GCC_FLAGS_G	1	Very fast, no optimization (for example, cell characterization)
PVA_GCC_FLAGS_O0	1	Less fast, some optimization
PVA_GCC_FLAGS_O1	1	Incrementally slower, more optimization
PVA_GCC_FLAGS_O2	1	Slower still, even more optimization
PVA_GCC_FLAGS_O3	1	Slow compilation, maximum optimization (for example, long transient simulations)

Reusing Precompiled Verilog-A Files in Simultaneous Multiple Simulations

Use the following procedure to reuse precompiled Verilog-A files in simultaneous multiple simulations. The environment variable `PVA_USESO` points to a shared runtime library path to reuse precompiled VA code in the existing runtime library.

1. Begin by creating a `pvadir` directory. You must run a PrimeSim Continuum simulator to create a shared `pvadir` directory. The PrimeSim Continuum simulator automatically creates the directory when you run the PrimeSim Continuum solution with Verilog-A instances.

Example of a simple PrimeSim Continuum placeholder netlist `test.sp` with a Verilog instance:

```
*Dummy netlist test.sp
.hdl res.va
.end
```

Run a PrimeSim Continuum simulator (such as the PrimeSim HSPICE tool) as follows:

```
setenv PVA_GENSO 1
hspice test.sp -o HSPICE_results/test
```

The `pvadir` directory is created: `HSPICE_results/test.pvadir`

2. Using precompiled Verilog-A files: After the `pvadir` directory has been created, you can use it for future PrimeSim Continuum solution simulations that use the same Verilog-A files in the `.hdl` command. It is recommended that you copy the `pvadir` to a central location. Use the `PVA_USESO` variable to point to that `pvadir` directory:

```
setenv PVA_USESO $absolute|$relative_path
```

Note:

The `$absolute|$relative_path` can be written with or without `.pvadir` because `PVA` appends the `.pvadir` if there is no `.pvadir`.

Example of `PVA_GENSO` and `PVA_USESO` Usage in C-Shell Commands

Step 1: Setting Shared Runtime Library—`setenv PVA_GENSO 1`

```
setenv PVA_GENSO 1
hspice test.sp -o HSPICE_results/test
if ( $status != 0 ) exit 999
if ( -d ./shared-location.pvadir )
rm -r -f ./shared-location.pvadir
cp -r -p HSPICE_results/test.pvadir ./shared-location.pvadir
```

Step 2: Running Simultaneous Jobs—`setenv PVA_USESO $absolute|$relative_path`

```
setenv PVA_USESO ./shared-location.pvadir
hspice test1.sp -o hsp/final_results1
hspice test2.sp -o hsp/final_results2
hspice test4.sp -o hsp/final_results4
hspice test5.sp -o hsp/final_results5
.....
```

or

```
setenv PVA_USESO ./shared-location.pvadir
bsub hspice test1.sp -o hsp/final_results1
bsub hspice test2.sp -o hsp/final_results2
bsub hspice test4.sp -o hsp/final_results4
bsub hspice test5.sp -o hsp/final_results5
bsub .....
```

Note:

The following are important considerations.

1. pVA looks for `XYZ.pvadir` if you set `XYZ` or `XYZ.pvadir`, when `PVA_USESO` is set.
2. pVA can accept a parameterized array or port sizes in a shared runtime library if the parameter value is passed from the PrimeSim HSPICE-compatible netlist.
3. If you execute Step 1 and Step 2 at the same time, you do not need to be concerned that the PrimeSim Continuum simulator version changed or a Verilog-A file model changed.
4. The user sharing/reusing the precompiled Verilog-A files must use the same version of whichever PrimeSim Continuum simulator that was used to create the files.

pVA Compiles ALL Modules in an .hdl Statement

pVA does not support Compile on Demand, as it harms performance if there are unused modules in the `.hdl` statements in an PrimeSim HSPICE-compatible netlist. pVA compiles all of the modules in `.hdl` statements; but in a PrimeSim HSPICE-compatible netlist you can add:

```
.hdl "use_va pfet_core nfet_core"
```

Then, pVA selects only the two specified Verilog-A modules (`pfet_core` and `nfet_core`) to create the runtime library `pvaRTL_platform.so`.

To identify these two modules, you can `grep` the string `Creating Verilog-A` from the `<-o>.valog` file. And then do a `sort` and `uniq` to get the output of the two modules. For example:

```
grep "Creating Verilog-A" <-o>.valog |sort| uniq | \
sed -e 's/.*module //'
```

Case-Sensitivity

Use double quotation marks to keep the case as written, otherwise the PrimeSim Continuum solution converts the text to lowercase.

For example, in the following command:

```
.hdl use_va Pfet_core Nfet_core
```

The PrimeSim Continuum simulator converts `Pfet_core` and `Nfet_core` to `pfet_core` and `nfet_core` if there are no double quotation marks.

Automatic Hierarchy

pVA automatically finds the child modules of the parent module.

For example, the runtime library `pvaRTL_platform.so` includes the child modules of the `pfet_core` and the `nfet_core` if these two modules are hierarchical Verilog-A.

pVA Time Derivative Function Performance Tuning Tip

When there are `ddt` (time derivative function) usages in a charge model such as a capacitor, it is often beneficial to include the constant capacitance inside the `ddt` expression, if the capacitance does not change during the simulation. Unified Verilog-A (pVA) does this automatically, but may miss some cases. For example:

```
C = param1 * param2;  
I(p,n) <+ C * ddt(V(p,n));
```

Because the `C` variable only depends on parameters, it must remain the same during the simulation. Rewriting the statement as follows may benefit the performance of the simulation:

```
I(p,n) <+ ddt(C * V(p,n));
```

Known Limitations

Known limitations for Verilog-A with the PrimeSim Continuum solution include the following:

- [C Language Keywords Limitation](#)
- [GCC Extension Keywords Prohibited](#)
- [analysis\(\) Function Behavior](#)
- [Unsupported Language Features](#)
- [Array/Vector Parameter Support](#)

C Language Keywords Limitation

Unified Verilog-A translates the Verilog-A file to C language. Therefore, to avoid conflicts, the following Standard ANSI C keywords cannot be used in the Verilog-A file:

auto	default	float	register	struct	volatile
break	do	for	return	switch	while
case	double	goto	short	typedef	
char	else	if	signed	union	
const	enum	int	sizeof	unsigned	
continue	extern	long	static	void	

If you use C (or GCC extension) keywords in port names, the compilation fails with the message:

```
*pvaE* Bad C code detected
```

Further down in your listing file, you also see that the model was not found:

```
Definition of model/subckt "shift" is not found for the element "x1"
```

GCC Extension Keywords Prohibited

asm, typeof, inline

analysis() Function Behavior

The `analysis()` function definition assumes that the operating point (OP) analysis associated with any user-specified analysis is unique to that user-specified analysis. For example, if you specify the following function, it must return 1 for AC analysis and 1 for its underlying operation point (OP) analysis.

```
analysis("ac")
```

Similarly, `analysis("tran")` must return 1 for transient analysis and 1 for its underlying OP analysis. In the PrimeSim Continuum solution, a single “common” OP analysis is performed in the setup that is outside the context of AC, transient, or other analyses. Because that OP is outside the context of the user-specified analysis, the `analysis()` function does not know the parent analysis type (during the OP analysis). The analysis (“ac”), analysis (“tran”), and so on, returns 0 during this “common” OP analysis. You can ensure that the analysis function returns true (1) during these analyses by adding “static” to the list of functions.

Example

```
if ( analysis("ac") )
begin
    // do something
end
```

The preceding example should be written as:

```
if( analysis("ac", "static") )
begin
    // do something
end
```

The same is true for the “tran” and “noise” analysis names.

Unsupported Language Features

The following Verilog-A LRM 2.3.1 language features are not supported.

- Unified Verilog-A (pVA) ignores input, output, and inout enforcement described in LRM 2.3.1, section 7.1.

```
module test(in,out);
    electrical in,out;
    input in;
    output out;
    real out_value;
    analog begin
        out_value = 1.0;
        V(in) <+ out_value;    // Input node used as output
                               // is not prevented, V(in) will be
                               // assigned to out_value
    end
endmodule
```

- `exclude` in parameter with an expression:

For example:

```
parameter real value=0;
parameter real par=0 from (-inf:inf) exclude 1.0*value+0;
```

- Out-of-module-references as described in the LRM 2.3.1, section 7.

In this example, the reference to `example2.net` inside the `example1` module is not supported.

```
module example1;
    electrical example2.net; // Feature not supported
endmodule
module example2;
```

```
    electrical net;  
endmodule
```

- Time tolerances on `transition()` functions, as described in LRM 2.3.1, Section 4.4.9.1, respectively.

```
transition(expr[,td [,rise_time [,fall_time [,time_tol ] ] ] ])
```

- reg-strings as described in Section 2.6 of LRM 2.3.1.
- Output variables and string parameters on paramsets.
- `$monitor`
- The following are limitations in PrimeSim Continuum solution advanced analog Verilog-A only:
 - `$simparam` simulation parameter
 - 0 port module
 - Delays (`absdelay()`), event-controlled constructs, memory states (variables that hold their value between time steps), and explicit time-dependent functions are not supported in PrimeSim Continuum solution advanced analog analyses.

Array/Vector Parameter Support

The PrimeSim Continuum solution does not support array/vector parameters, so a Verilog-A array/vector parameter cannot be passed through a SPICE netlist. Use the following workaround:

Create a wrapper module, instead:

```
module wrapper...  
parameter p1,p2,p3...  
    c1 (.p({p1,p2,p3,...}) child(...))
```

Then instantiate the wrapper:

```
x1 1 2... wrapper p1=1 p2=10...
```

Verilog-A (pVA) Messages

When compiling a Verilog-A module using the pVA compiler, you may see `pvaI`, `pvaW`, `pvaE`, or `pvaNIY` messages.

These messages have the following meanings:

- `pvaI`: Information message that has no effect on compilation and simulation results.
- `pvaW`: Warning message that possibly could affect compilation and simulation results.
- `pvaE`: Error detected by pVA. The compilation or simulation is terminated.
- `pvaNIY`: The Verilog-A function is “Not Implemented Yet” by pVA.

These messages give useful information and help you in debugging the Verilog-A module. For example:

```
*pvaI* ##### Total 131 line-size(s), 29 expr(s), 2 contr(s), 4 init(s), 4
  behav(s), 2 port(s)
*pvaW* macro `P_Q redefined at (constants.vams:34)
```

Troubleshooting Error Messages

If the PrimeSim Continuum simulator ends unexpectedly and issues the following error:

```
**error** call to epvaHDLinit failed
```

This error can occur when your system configuration is not sourcing the `meta.cshrc` file. You also see the following error in the `*.lis` file:

```
*pvaE* Please invoke hspice script instead of binary
```

To avoid these errors, you must set the `HSP_HOME` environment variable and source the `meta.cshrc` file:

```
% setenv HSP_HOME /root/hspice_installation/hspice
% source $HSP_HOME/bin/cshrc.meta
```

If the error persists, check the compiler log file, `*.pvadir/pvaRTL.log`, for errors. If an error is found, then the Verilog-A (pVA) compiler has an issue that requires further debugging. Contact the support center for further assistance.

Environment Variables and Command Options

[Table 10](#) summarizes Verilog-A (pVA) environment variables and equivalent options on the PrimeSim Continuum simulator's command line, PrimeSim Continuum command and control options, and `.hdl` command options.

Table 10 Environment Variables and Command Options

Environment Variable (setenv)		PrimeSim Continuum command-line option	PrimeSim Continuum command	.hdl command
Command	Value	-option	.OPTION	option
PVA_RMRTL (See Note 1 below)	1 0			
PVA_SKIP_LRM (See Note 2 below)	1 0			
PVA_GCC_FLAGS_G	1			
PVA_GCC_FLAGS_O0	1			
PVA_GCC_FLAGS_O1	1			
PVA_GCC_FLAGS_O2	1			
PVA_GCC_FLAGS_O3	1			
HSP_HDL_PATH (See Verilog-A File Search Path)	<i>path/dir_name</i>	-hdlpath (See Verilog-A File Search Path)		-I (See Verilog-A File Loading Considerations)
				-D (See Verilog-A File Loading Considerations)
		-hdl (See Loading Verilog-A Modules)	.hdl (See Loading Verilog-A Modules)	
		-vamodel (See Overriding Subcircuits With Verilog-A Modules)	vamodel (See Overriding Subcircuits With Verilog-A Modules)	
			macmod=1 (See Overriding Subcircuits With Verilog-A Modules)	
				use_va modnm1 modnm2 ... (See Performance Considerations With the PrimeSim Continuum Solution)
		-hpp	bypass_va=1	

Table 10 Environment Variables and Command Options (Continued)

Environment Variable (<i>setenv</i>)		PrimeSim Continuum command-line option	PrimeSim Continuum command	.hdl command
Command	Value	-option	.OPTION	option
PVA_GENSO	1 0			
PVA_USESO	<i>path/dir_name</i>			
PVA_AUTO_DECLARE (See Note 3 below)	1 0			
PVA_KEEP_VARS (See Note 4 below)	1 0			

Note:

Here are some important notes:

1. PVA_RMRTL – When you have a series of .ALTER commands in a PrimeSim HSPICE-compatible netlist, pVA clears all intermediate files in the .pvadir before it executes the next .ALTER command.
2. PVA_SKIP_LRM – Some Verilog-A syntax is illegal according to the LRM 2.3.1, but it is acceptable if you specify *setenv PVA_SKIP_LRM*. However, PrimeSim Continuum/pVA does not guarantee the correctness of the result.
3. PVA_AUTO_DECLARE – If a variable is used without a declaration, the variable is set to a proper type automatically with a warning message (default 1). Otherwise, produce an error message, when 0.
4. PVA_KEEP_VARS – If a variable is declared or assigned a value without using it anywhere else in RHS, the variable is removed for the probing table with a warning message (default 0). Otherwise, keep all variables, when 1.

Example: Verilog-A Frequency Divider Model

You can use Verilog-A to model circuit behavior before committing the design to hardware. For example, you can use Verilog-A to model the building blocks of a PLL (phase detector, vco, frequency divider, and so on), then verify and fine-tune their functionality using a PrimeSim Continuum simulator.

Because Verilog-A is a standard language managed by Accellera, it is portable to any SPICE-based simulator.

The following is an example of a programmable Verilog-A frequency divider model that can be used in a PLL circuit instead of full transistor-level implementation.

```
`include "constants.vams";
`include "disciplines.vams";
`include "compact.vams";
//
// Based on the OVI Verilog-A Language Reference Manual, version 2.3.1
//
//
module freq_divider(vtrig, vout_q);
input vtrig;
output vout_q;
electrical vtrig, vout_q;
//electrical vtrig, vout_q, vout_qbar;
parameter real vlogic_high = 3;
parameter real vlogic_low = 0;
parameter real vtrans = 1.5;
parameter real tdel = 0.1n from [0:inf);
parameter real trise = 1n from (0:inf);
parameter real tfall = 1n from (0:inf);
parameter integer div_ratio = 2 from (0:inf);
    integer q;
    integer out;
    integer x;
    real count;
    analog begin
// Initializing variables.
        @ ( initial_step ) begin
            x = 1;
            count = 1.1;
        end
// Count triggering at rising edge.
        @ (cross( V(vtrig) - vtrans))
        if ( count <= div_ratio) begin
            count = count +1;
            out = vlogic_high * x;
        end
// Frequency dividing.
        else begin
            x = abs(x - 1);
            count = 1.1;
        end
// Output.
        V(vout_q) <+ transition( out, tdel, trise, tfall );
    end
endmodule
```

Note that all of the variables declared in the parameter section (vlogic_high, vlogic_low, tdel, div_ratio, and so on) can be instantiated at the instance statement in the top-level SPICE netlist or testbench.

Chapter 5: Using Verilog-A

Example: Verilog-A Frequency Divider Model

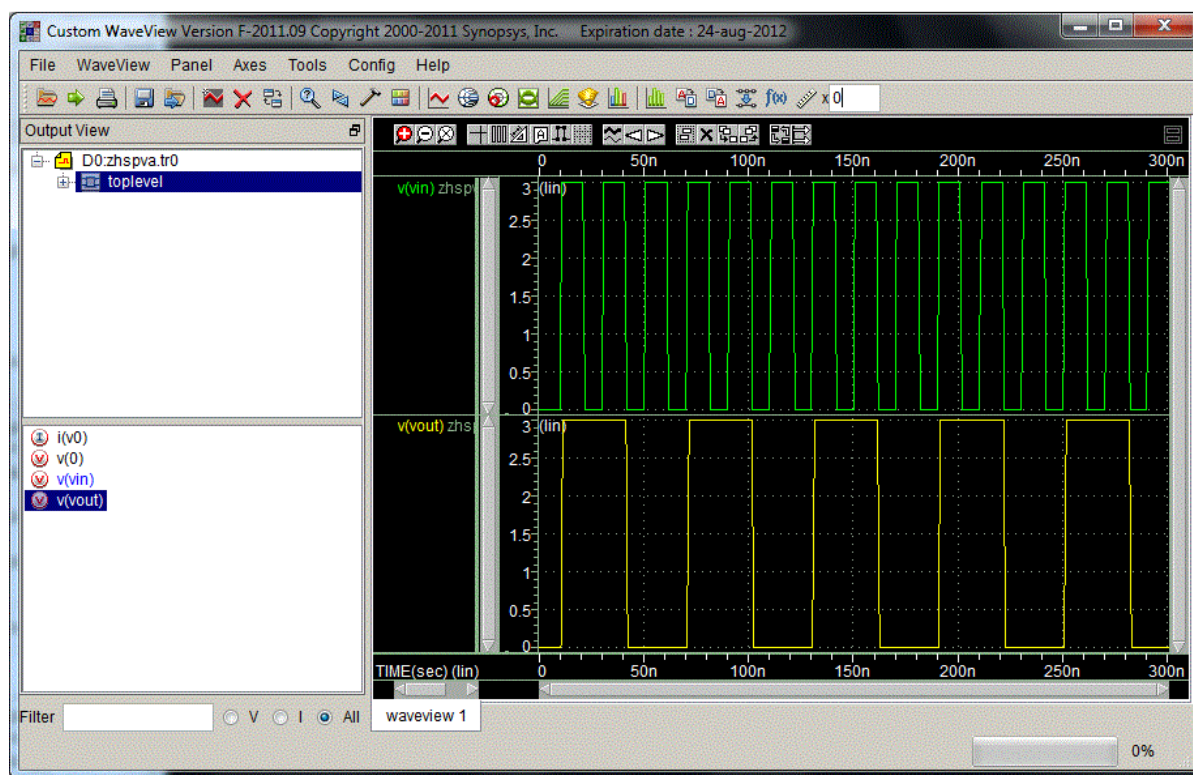
The key part of the divider function in the model is the $x = \text{abs}(x-1)$; statement. The value of x toggles between 0 and 1 at each iteration. For example if `div_ratio` is 2, it takes two iterations of positive going edges of the input signal to complete one clock cycle at the output.

Here is an example testbench to test the frequency divider with the divide ratio set to 3:

```
* Frequency divider.
X1 vin vout freq_divider div_ratio=3
Rout vout 0 1k
V0 vin 0 pulse 0 3 10n 1n 1n 10n 20n
.tran .1n 300n
.probe v(*)
.option post
.hdl veriloga.va.hspice
.end
```

The following is the output waveform of the frequency divider testbench.

Figure 8 Output waveform of the frequency divider testbench



6

Multi-Technology Simulation of 3D Integrated Circuit

Describes the PrimeSim Continuum solution to simulate a modularized IC chip inside a 3D integrated circuit.

PrimeSim Continuum simulators provide multi-die interconnect analysis capability. A three-dimensional integrated circuit (3DIC) is a single chip that integrates two or more layers of active electronic components both vertically and horizontally into a single circuit. All components on the layers communicate using on-chip signaling, whether vertically or horizontally.

The following sections discuss these topics:

- [Overview of PrimeSim HSPICE-Compatible 3DIC Simulation Netlist](#)
- [3DIC Netlist Construct and Usage](#)
- [Transient Analysis and Alters Simulation Features](#)
- [Full Circuit Example](#)
- [3DIC Hierarchical IC Module Support](#)
- [3DIC Support for TMI, Custom CMI, and MOSRA](#)

Overview of PrimeSim HSPICE-Compatible 3DIC Simulation Netlist

The following are the general rules for analysis statements and simulation control statements:

- You must define all analysis statements and simulation control statements with the full 3DIC instance hierarchy reference.
- In any analysis and simulation control statements that reference subcircuit names, you must identify the unique IC module using the `.module` construct label.

See [Full Circuit Example](#) for an example that references include and library files.

The PrimeSim Continuum solution simulates a 3DIC with existing methodology to simulate the modularized IC chip inside the full 3DIC (single die) and uses enhanced commands and simulation controls specific to the process.

The PrimeSim Continuum solution keeps the single IC module netlist intact. It also allows customizing of the circuit properties for different instantiations of the same IC module, such as `.TEMP`, `.OPTION TNOM`, `.OPTION SCALE`, `.OPTION GEOSHRINK`, and netlist parameters. This approach enables netlist interpretation and model interpretation controlling statements, such as `.IVTH`. This methodology maintains the instance full hierarchy to match the layout hierarchy for back-annotation.

The following sections discuss these topics:

- [PrimeSim HSPICE-Compatible Netlist Definitions](#)
- [IC Module Usage](#)

PrimeSim HSPICE-Compatible Netlist Definitions

The netlist scope includes:

- A device model that you can define by either the `.MODEL` command or the `.SUBCKT` (macro-model) constructs.
- Device model referencing depends on the netlist static scope (the netlist definition inside the `.SUBCKT` construct).
- Your reference netlist parameters based on dynamic scope (the full instance hierarchical path).

IC Module Usage

The PrimeSim Continuum solution allows you to define unique circuit properties as the default for each IC module in the following items:

- Circuit property definitions using PrimeSim HSPICE-compatible commands and control options such as: `.OPTION SCALE`, `.OPTION GEOSHRINK`, `.TEMP` and so on.
- Netlist parameter references using the `.PARAM` command
- Model card definitions for both model card and macro-model forms
- Verilog-A module definitions
- Subcircuit block definitions and device models sharing

- Confined netlist name definitions being the same but unique inside each of the IC modules
- Instance-based changes to the IC module default properties

3DIC Netlist Construct and Usage

The 3DIC construct employs two commands, `.MODULE` and `.ENDMODULE`, to create a 3DIC-specific netlist block. These commands enable you to define the unique IC module entities without name labels or circuit properties and to avoid collision between different IC modules. You can define the model reference static scope unique for the given IC module and define the unique IC module default entities and circuit properties.

For definitions and details about all legal commands and options listed in the `.MODULE` and `.ENDMODULE` construct, see the *PrimeSim™ Continuum Reference Manual: Commands and Control Options*.

Syntax

```
.MODULE label [BASE=base_module_label]
...
.ENDMODULE [label]
```

where: *label* consists of legal netlist commands and constructs, including circuit-topology definition statements and circuit-property definition statements such as:

- File inclusion commands `.LIB` and `.INCLUDE`
- `.SUBCKT` constructs that contain legal netlist commands
- `.HDL` (Verilog-A) commands
- `.PARAM` commands
- `.MODEL` commands
- `.OPTION SCALE` and `.OPTION GEOSHRINK` commands: These scaling control options define the device scaling factor for each IC module such that all instances below the subcircuit carry these properties.
- `.TEMP` and `.OPTION TNOM` commands: These temperature control commands define the simulation temperature for each IC module such that all instances below the subcircuit carry these properties.
- `.GLOBAL` command: This command defines the global node for each IC module. Thus, all nodes residing below the subcircuits that carry this node definition can connect to this node within the IC module. For example, if `.GLOBAL` defines a node within the `.MODULE` construct, only the instances inside the subcircuits (defined within the same

.MODULE construct) and subsequent nodes below the subcircuits can connect to the defined node without connecting through subcircuit ports.

Note:

Even though PrimeSim Continuum simulators define the .GLOBAL nodes for each IC module, each simulator limits its reference to the IC module only. The nodes can be referenced from the top level through the following syntax:

```
instance_name.global_node_label
```

- .IVTH command: If you declare this command inside the .MODULE block, it applies to the model card defined within the same .MODULE construct only.

and where: *base_module_label* allows you to define and inherit all of the content of the base module in the derived IC module without any IC module label. The derived IC module content can overwrite the base IC module content.

You can connect the module-based global nodes explicitly at the top level such that, all instances instantiated with the IC module top subcircuit could have different top level connection. For more information on accessing the global node inside a module from the top level, see the .MODULE and .CONNECT commands in the *PrimeSim™ Continuum Reference Manual: Commands and Control Options*.

Illegal Netlist Commands, Controls, and Constructs

The following may not be part of the contents of a .MODULE construct:

- Analysis feature statements
- Output probing statements such as .PRINT, .PROBE, and .MEASURE
- Simulation feature statements, such as .ALTER, .TRAN, and Monte Carlo constructs
- Simulation controlling options other than those explicitly stated in the [Syntax](#) section
- Conditional/logical statements such as .IF and .ELSE

The following sections discuss these topics:

- [Scope Reference Rules](#)
- [Using Verilog-A Modules Within the .MODULE Scope](#)
- [Top-Level IC Module Reference](#)
- [Intrinsic Model Card Reference From a Top-Level Instance](#)
- [Over-writing Circuit Properties](#)

- [Parameter Direct Access From Top-Level Netlist](#)
- [Using Parameters Defined in Top-Level Netlist](#)

Scope Reference Rules

All entities and circuit properties can be applicable only to the netlist entities inside the same `.MODULE` and `.ENDMODULE` construct. In other words, the entities defined inside the `.MODULE` construct take higher precedence over those with the same name labels that are defined at the top level outside a `.MODULE` construct.

Examples

Example 1: This is a legal netlist in which the simulation nominal temperature for the `xtop1` block is 40. The nominal temperature for the rest of the circuit is 25. The device length for the `xtop1.m1` is `3.6e-008`. The device length for the `xtop2.m1` is `5.6e-008`.

```
.temp 25
.param ptop=5.6e-008
xtop1 ... top-module::top1
xtop2 ... top2
.module top-module
    .temp 40
    .param ptop=3.6e-008

    .subckt top1 ...
        m1 ... nmod l="ptop" w=2.7e-006 ...
    .ends top1
.endmodule top-module
.subckt top2 ...
    m1 ... nmod l="ptop" w=2.7e-006 ...
.ends top2
```

Example 2: Legal netlist with device references to the models defined within the same `.module` construct. The `xtop1.m1` references the model card with `vth0=0.38` while the `xtop2.m1` references the one with `vth0=0.41`.

```
.model nmod nmos level=54 vth0=0.41 ...

xtop1 ... top::top1
xtop2 ... top2
.module top
    .model nmod nmos level=54 vth0=0.38
    ...
    .subckt top1
        m1 ... nmod l=3.6e-008 w=2.7e-006 ...
    .ends top1
.endmodule top
.subckt top2
    m1 ... nmod l=3.6e-008 w=2.7e-006 ...
.ends top2
```

Example 3: This *illegal* netlist shows how you must not reference undefined model cards, either in the same `.module` construct or at the top-level, even though the referenced model name is uniquely defined inside another `.module` construct.

```
xtop1 ... tmod1::top1
xtop2 ... tmod2::top2
.module tmod1
    .model nmod1 nmos level=54 vth0=0.45 ...
    .subckt top1
    m1 ... nmod2 l=3.6e-008 w=2.7e-006 ...
    .ends top1
.endmodule tmod1
.module tmod2
    .model nmod2 nmos level=54 vth0=0.41 ...
    .subckt top2
    m1 ... nmod1 l=3.6e-008 w=2.7e-006 ...
    .ends top2
.endmodule tmod2
```

Example 4: This legal netlist shows the device reference to the models defined without the `.module` construct only because there is not a model defined inside the `.module` construct with the same name label. The `xtop1.m1` references the model card with `vth0=0.45` at the top level static scope. The `xtop1.m2` references the model card with `vth0=0.41`. The `xtop2.m1` references the model card with `vth0=0.45` (same as the `xtop1.m1`).

```
.model nmod1 nmos level=54 vth0=0.45 ...
xtop1 ... top::top1
xtop2 ... top2
.module top
    .model nmod2 nmos level=54 vth0=0.41
    ...
    .subckt top1
    m1 ... nmod1 l=3.6e-008 w=2.7e-006
    ...
    m2 ... nmod2 l=3.6e-008 w=2.7e-006
    ...
    .ends top1
.endmodule top
    .subckt top2
        m1 ... nmod1 l=3.6e-008 w=2.7e-006 ...
    .ends top2
```

Using Verilog-A Modules Within the .MODULE Scope

You can use a Verilog-A (VA) module to uniquely define each IC module scope by using the `.HDL` statement inside the `.MODULE` construct. See ".HDL" in the *PrimeSim™ Continuum Reference Manual: Commands and Control Options*, and [Using Verilog-A](#) in this document.

The following rules apply for VA module references:

- Instances inside an IC module can reference the VA modules defined inside the same IC module and the VA modules defined without the IC-module scope (top level).
- Instances inside an IC module cannot cross-reference any VA module defined in a different IC module scope.
- The interposer level (top level) instance can reference the VA modules defined without the IC-module scope (top level) and VA modules defined inside any IC module with additional IC-module label references.

The VA module reference precedence is as follows:

1. VA module within the same IC module scope
2. VA module without IC-module scope (top level)

The following sections discuss these topics:

- [VA Reference Usage Rules](#)
- [VA Reference Usage Examples](#)

VA Reference Usage Rules

Use the following PrimeSim HSPICE-compatible subcircuit instance syntax when you want to have a top-level instance reference the VA module inside the IC module:

```
Xinstance_name interconnect VA_module_name parameters
```

Use the following Subcircuit Instance Syntax Extension:

```
Xinstance_name interconnect A_module_name[module_label::] VA_module_name
```

VA Reference Usage Examples

Legal Netlist Example 1: VA module referenced with multiple VA modules with the same name in different scopes. The `xtop1.x1` references the VA module, `my_inv`, defined inside the `top` IC module while the `xtop2.x1` references the one defined without IC module (top-level).

```
.HDL "my_va.va"          * "my_inv" VA module is defined.
xtop1 ... top::top1
xtop2 ... top2
.module top
  .HDL "my_va2.va"      * "my_inv" VA module is defined.
  .subckt top1
    x1 ... my_inv ...
  .ends top1
.endmodule top
.subckt top2
```

```
x1 ... my_inv ...
.ends top2
```

Legal Netlist Example 2: VA module defined within the IC module and referenced from the top level instance. The `xr2` references the VA module, `resistor`, defined inside the `mod1` IC module while the `xr3` references to the one defined inside the `mod2` IC module.

```
// res.va
`include "discipline.h"
module resistor (a,b);
  inout a,b;
  electrical a,b;
  branch(a,b) res;
  parameter real R=1;
  analog begin
    I(res) <+ V(res)/R;
  end
endmodule

// res_cap.va
`include "discipline.h"
module resistor (a,b);
  inout a,b;
  electrical a,b;
  branch(a,b) res;
  parameter real R=1;
  parameter real C=1e-15;
  analog begin
    I(res) <+ V(res)/R + C*ddt(V(res));
  end
endmodule

*** Top level netlist
.module mod1
  .hdl './res.va'
  ...
.endmodule

.module mod2
  .hdl './res_cap.va'
  ...
.endmodule

vsource 1 0 pw1 0n 0 10n 3
r1 1 2 10K
xr2 2 3 mod1::resistor R=10K
xr3 3 0 mod2::resistor R=10K C=1p
...
c1 2 0 1uF
.tran 1n 1u
.end
```

Illegal Netlist Example 3: This example contains a VA module cross-referenced to the VA module defined in a different IC module scope. It is illegal to have the `xtop.x1` in the IC module `top1` cross-referenced to the VA module, `inv`, defined inside the `top2` IC module.

```
xtop ... top1::top1
.module top1
  .subckt top1
    x1 ... inv ...
  .ends top1
.endmodule top
.module top2
  .HDL "my_va.va"      * "inv" VA module is defined.
.endmodule top
```

Top-Level IC Module Reference

Reference Rules for top-level IVC module references:

A top-level instance instantiates the IC module top subcircuit block that is defined inside the `.MODULE` construct and requires a reference to the module label.

PrimeSim HSPICE-Compatible Subcircuit Instance Syntax:

`Xinstance_name interconnect subcircuit_name parameters`

PrimeSim HSPICE-Compatible Subcircuit Instance Syntax Extension:

`Xinstance_name interconnect [module_label::]subcircuit_name parameters`

Examples of Top-Level References

Example 1: A legal netlist in which the `xtop1` instance instantiates the `top` subcircuit defined inside the `tmod` module (with the `m1` instance) while the `xtop2` instance instantiates the `top` subcircuit defined without the `.module` construct (with the `m2` instance).

```
xtop1 ... tmod::top
xtop2 ... top
.module tmod
  .subckt top
    m1 ...
  .ends top
.endmodule tmod
.subckt top
m2 ...
.ends top
```

Example 2: An *illegal* netlist because you may not reference a subcircuit definition label with an incorrect `.module` label even though the subcircuit definition name is unique throughout the netlist.

```
xtop1 ... tmod1::top2
xtop2 ... tmod2::top1
xtop3 ... top1
xtop4 ... top2
.module tmod1
  .subckt top1
  m1 ...
  .ends top1
.endmodule tmod1
.module tmod2
  .subckt top2
  m2 ...
  .ends top2
.endmodule tmod2
```

Intrinsic Model Card Reference From a Top-Level Instance

Interpretation rules: When the intrinsic model card is referenced from the top-level instance, all interpretation references are based on the top-level rules, such as parameter passing and scaling.

PrimeSim HSPICE-Compatible Transistor Instance Syntax

Minstance_name interconnect model_card_name parameters

PrimeSim HSPICE-Compatible Transistor Instance Syntax Extension:

Minstance_name interconnect [module_label::]model_card_name parameters

Example

```
m1 ... tmod::nmos ...
.module tmod
  .model nmos nmos ...
.endmodule tmod
```

Over-writing Circuit Properties

The top-level IC module instance can overwrite any circuit properties with a predefined `.modulevar` construct label.

Syntax

PrimeSim HSPICE-Compatible Subcircuit Instance Syntax:

Xinstance_name interconnect subcircuit_name parameters

PrimeSim HSPICE-Compatible Subcircuit Instance Syntax Extension:

Xinstance_name interconnect [module_label::] subcircuit_name
[modulevar=modulevar_label] parameters

Examples: Instance-Specific Properties Reference

Example 1: Legal netlist specifying the properties as follows:

Instance	Nominal Temperature	Device Length
xtop1.m1	25	5e-008
xtop2.m1	40	3e-008
xtop3.m1	25	5e-008
xtop4.m1	10	1e-008

```
xtop1 ... tmod::top modulevar="top-inst"
xtop2 ... tmod::top
xtop3 ... top modulevar="top-inst"
xtop4 ... top

.temp 10
.param ptop=1e-008
.module tmod
    .temp 40
    .param ptop=3e-008

    .subckt top ...
    m1 ... nmod l="ptop" w=2.7e-006 ...
    .ends top
.endmodule tmod

.modulevar top-inst
    .temp 25
    .param ptop=5e-008
.endmodulevar top-inst

.subckt top ...
m1 ... nmod l="ptop" w=3.7e-006 ...
.ends top
```

Example 2: This netlist shows top-down parameter passing of the following properties (by way of `.option parhier=global`):

Instance	Nominal Temperature	Device Length
xtop1.m1	25	5e-008
xtop2.m1	25	5e-008
xtop3.m1	40	3e-008
xtop4.m1	40	3e-008
xtop5.m1	25	5e-008
xtop6.m1	25	5e-008
xtop7.m1	10	1e-008
xtop8.m1	10	1e-008

And the bottom-up parameter passing of the following properties using `.OPTION PARHIER=local`:

Instance	Nominal Temperature	Device Length
xtop1.m1	25	8e-008
xtop2.m1	25	4e-008
xtop3.m1	40	8e-008
xtop4.m1	40	6e-008
xtop5.m1	25	7e-008
xtop6.m1	25	2e-008
xtop7.m1	10	7e-008
xtop8.m1	10	9e-008

```

xtop1 ... tmod::top modulevar="top-inst"
xtop2 ... tmod::top modulevar="top-inst"
+ ptop=4e-008
xtop3 ... tmod::top
xtop4 ... tmod::top ptop=6e-008

xtop5 ... top modulevar="top-inst"
xtop6 ... top modulevar="top-inst"
+ ptop=2e-008
xtop7 ... top

```

```
xtop8 ... top ptop=9e-008

.temp 10
.param ptop=1e-008

.module tmod
    .temp 40
    .param ptop=3e-008

    .subckt top ...
    .param ptop=8e-008
    m1 ... nmod l="ptop" w=2.7e-006 ...
    .ends top
.endmodule tmod

.modulevar top-inst
    .temp 25
    .param ptop=5e-008
.endmodulevar top-inst

.subckt top ...
.param ptop=7e-008
m1 ... nmod l="ptop" w=3.7e-006 ...
.ends top
```

Example 3: *Illegal* netlist due to referencing undefined `.modulevar` construct label.

```
xtop1 ... tmod::top modulevar="top-inst"
.temp 10
.param ptop=1e-008
.module tmod
    .temp 40
    .param ptop=3e-008

    .subckt top ...
    m1 ... nmod l="ptop" w=2.7e-006 ...
    .ends top
.endmodule tmod
```

Parameter Direct Access From Top-Level Netlist

The following syntax extension allows you to access the parameters defined inside the `.module` or `.modulevar` construct.

Syntax

```
.param param_label="module_label::param_label"
.param param_label="modulevar_label::param_label"
```

Example

```
.module top
  .param top_p=5
.endmodule
.modulevar top_inst
  .param top_inst_p=8
.endmodulevar
.param p1="top::top_p"          * p1 = 5
.param p2="top_inst::top_inst_p" * p2 = 8
```

Using Parameters Defined in Top-Level Netlist

The following option allows you to use the parameters defined in top-level Netlist instead of those defined inside the `.module` or `.modulevar` construct.

.OPTION USE_TOP_PARAM_3DIC

Specifies the list of parameters that is defined in top and sets higher priority than those defined in the module.

Syntax

```
.option use_top_param_3dic='param1'          $ one parameter
.option use_top_param_3dic='param2,param3'    $ use ',' as delimiter
.option use_top_param_3dic='param4 param5'    $ use ' ' as delimiter
```

Default NULL

Description

Use this option in a 3DIC netlist and put out of `.module`. The option value may be one parameter or multiple parameters. If multiple parameters are specified in one option, use a comma (,) or an empty space symbol as a delimiter. If multiple `use_top_param_3dic` options are set, all the option values get accumulated.

Examples

```
.option use_top_param_3dic='dagetime'      $ override 'dagetime' defined in
module
.param dagetime=3

.module tmod
  .param dagetime=0  $ final value: dagetime=3
  .....
.endmodule
```

Transient Analysis and Alters Simulation Features

The current 3DIC solution applies to transient analysis only.

The following sections cover these topics:

- [Sweeps for Temperature and Parameters](#)
- [.ALTER Features](#)

Sweeps for Temperature and Parameters

You can simulate 3DIC modules using temperature and parameter sweeps in a transient analysis:

- [Temperature Sweep in a Transient Simulation](#)
- [Parameter Sweep in a Transient Simulation](#)

Temperature Sweep in a Transient Simulation

This reference extension allows you to reference the `.temp` command or `.option tnom` for a temperature `SWEEP` in a transient simulation.

Keyword temp Syntax Extension

```
module_label::temp  
modulevar_label::temp
```

Example 1: Single IC module temperature `SWEEP`

```
.module top  
  .temp 25  
.endmodule  
* Sweep "top1::temp" from 40 to 100 with each step increase by 10.  
* Total simulation: 6  
.tran 0ns 10ns SWEEP top::temp 40 100 10
```

Example 2: Multiple IC module temperature `SWEEP` with "data"

```
.module top1  
  .temp 25  
.endmodule  
.module top2  
  .temp 40  
.endmodule  
.tran 0ns 1ns SWEEP data=tempdata  
* Sweep top1::temp from 25 to 35 and top2::temp from 40 to 45  
* Total simulation: 6  
.data tempdata top1::temp top2::temp  
25 40  
30 40  
35 40  
25 45  
30 45
```

```
35 45
.enddata
```

Parameter Sweep in a Transient Simulation

This reference extension allows you to sweep through the given parameters defined in either the `.module` or `.modulevar` constructs.

Reference Syntax

```
module_label::param_label
modulevar_label::param_label
```

Example1: Single IC module parameter SWEEP

```
.module top
.param p1= 25
.endmodule
* Sweep "top::p1" from 25 to 35 with each step increase by 5.
* Total simulation: 3
.tran 0ns 10ns SWEEP top::p1 25 35 5
```

Example 2: Multiple IC module parameter SWEEP with "data"

```
.module top1
.param p1= 25
.endmodule
.module top2
.param p2= 40
.endmodule
.tran 0ns 1ns SWEEP data=paramdata
* Sweep top1::p1 from 25 to 30 and top2::p2 from 40 to 45
* Total simulation: 4
.data paramdata top1:p1 top2::p2
25 40
30 40
25 45
30 45
.enddata
```

.ALTER Features

The 3DIC usage for the PrimeSim HSPICE-compatible `.ALTER` instance-statement replacement scheme is the same as the existing replacement scheme for a standard device simulation. Two PrimeSim HSPICE-compatible commands aid in this process: `.DEL MODULE` and `.DEL MODELEVAR`.

You modify or add to the existing `.module` or `.modulevar` constructs by redefining the `.module` or `.modulevar` constructs with predefined `label` names.

The following modification rule applies:

- The content defined with the new constructs for the existing label constructs only modifies the content to the same `label` construct content. If the statements exist in the original construct, the new content replaces the content of the original construct. If the statements are undefined in the original construct, the content is added to the `label` construct. If any reference content becomes unavailable for instance reference, you need to modify the instance statements by redefining them as replacements to the existing top-level instance statements.

See the following topics:

- [.DEL MODULE Command for .ALTER Blocks](#)
- [.DEL MODULEVAR Command for .ALTER Blocks](#)

.DEL MODULE Command for .ALTER Blocks

The `.DEL MODULE` command undefines the previously defined `.module` construct and prepares it for redefinition. You can only define the `.DEL MODULE` construct inside `.ALTER` blocks and all the contents previously defined with the specified `.MODULE` label are no longer referenced.

Syntax

```
.del module existing_module_label
```

Example: Redefining the `top` label

```
.module top
    .subckt inv
        m1...
        m2...
    .ends inv
.endmodule
xtop ... top::inv
.alter sl
    .del module top      * Undefine the "top" IC module.
    .module top          * Redefine the "top" IC module
        .subckt inv
            xm1 ... nch
            xm2 ... pch
        .ends inv
        .subckt nch
            ...
        .ends
        .subckt pch
            ...
        .ends
    .endmodule
.end
```

.DEL MODULEVAR Command for .ALTER Blocks

The `.DEL MODULEVAR` command undefines the previously defined `.MODULEVAR` construct and prepares it for redefinition. You can only define the `.DEL MODULEVAR` construct inside `.ALTER` blocks. With the `.DEL MODULEVAR` statement defined, all the contents that are previously defined with the specified `.MODULEVAR` label can no longer be referenced.

Syntax

```
.del modulevar existing_modulevar_label

.module top
    .subckt inv
        m1... w=p l=0.02u
    .ends inv
.endmodule

.modulevar ic1
    .param p=0.05u
.endmodulevar
.param p=0.06u
xtop ... top::inv modulevar="ic1"

.alter s1
    .del modulevar ic1    * "xtop.m1" will have "0.06u" as width.

.alter s2
    .del modulevar ic1
    .modulevar ic1
        .param p=0.07u    * "xtop.m1" will have "0.07u" as width.
    .endmodulevar
.end
```

Full Circuit Example

The following full circuit example includes these characteristics:

- This 3DIC consists of multiple single dies with the same design and technology node.
- This 3DIC has die-based simulation corners or circuit properties.

The example shows how you can use a single IC module netlist for a single IC set of memory simulation properties from the file `memory.lib`.

```
.lib TT
    .param ... * parameters for the simulation corner TT.
    ...
.endl TT
.lib FF
    .param ... * parameters for the simulation corner FF
    ...
```

```
.endl FF
.lib SS
    .param ... * parameters for the simulation corner SS
    ...
endl SS
.lib models
    .models ... * model cards for the memory IC.
    .subckt nch_mac ... * macro-models for the memory IC.
    ...
    .ends nch_mac
endl models
```

The netlist then draws on single IC memory circuit definitions from the file `memory.sp`:

```
* Top level circuit from single memory IC module
.subckt 1G_mem_top ...
    .temp 100
    xbank1 ... bank
    ...
.ends 1G_mem_top
* Other subcircuit definitions.
.subckt bank ...
    ...
.ends bank
```

The 3DIC Memory netlist draws from the `3D_mem.sp` file:

```
* global control and parameters for 3D IC simulation.
.temp -40
* 1st IC memory module (fast corner)
xmem1 .... 1GMem::1G_mem_top
* 2nd IC memory module (slow corner)
xmem2 .... 1GMem::1G_mem_top
* 3rd IC memory module (typical - default)
xmem3 .... 1GMem::1G_mem_top
* 4th IC memory module
xmem4 .... 1G_mem_top
    * top level control logic block.
x5 .... memory_control
.subckt memory_control
    ...
.ends memory_control
* Netlist definitions from the original single IC circuit.
.include "memory.sp"

.module 1GMem * Default control and parameters
    .lib "memory.lib" TT
    * Default single IC memory properties
    .temp 25
    ...
    * Models for the circuit elaborations in the memory circuit.
    .lib "memory.lib" models
    * Netlist definitions from the original single IC circuit.
```

```
.include "memory.sp  "  
.endmodule 1GMem
```

3DIC Hierarchical IC Module Support

The PrimeSim Continuum solution provides the ability of modularized IC chips inside the full 3DIC. The IC module instances can be instantiated hierarchically with reference to IC module subcircuit definition.

The following topics discuss the detail rules of hierarchical IC module netlisting configuration:

- [Netlist Construct for 3DIC Hierarchical IC Module Support](#)
- [Scope Reference Rules for 3DIC Hierarchical IC Module Support](#)
- [Output Syntax for 3DIC Hierarchical IC Module Support](#)

Netlist Construct for 3DIC Hierarchical IC Module Support

The PrimeSim Continuum simulators support hierarchical `.module` instances by defining the `.module` instances within a `.subckt` definition inside another `.module` definition.

Syntax:

```
.module modulename1  
  .subckt subcktname1  
  ...  
  Xmoduleinstance ... [modulename2::] subcktname2  
  .ends subcktname1  
.endmodule modulename1
```

Example netlist:

```
xtop1 ... tmod1::top1 ...  
.module tmod1  
  .subckt top1  
  ...  
  xtop2 ... tmod2::top2  
  .ends top1  
  ...  
.endmodule tmod1  
.module tmod2  
  .subckt top2 ...  
  ...  
  .ends top2  
  ...  
.endmodule tmod2
```

Description:

The `xtop1` instance instantiates the `top1` subcircuit that is defined inside the `tmod1` module. The `xtop2` instance inside `top1` subcircuit instantiates the `top2` subcircuit that is defined inside the `tmod2` module. The hierarchical path becomes `xtop1` for IC module `tmod1` and `xtop1.xtop2` for IC module `tmod2`.

Scope Reference Rules for 3DIC Hierarchical IC Module Support

With hierarchical IC module netlisting configuration, the IC module specific option only applies inside the IC module netlist interpretation, but does not influence the lower or upper IC module netlist interpretation.

The following sections discuss these topics:

- [Parameter-passing Rules for 3DIC Hierarchical Instantiation](#)
- [Options `scale`, `geoshrink` Inside module and `optparhier`](#)
- [Module-based `.temp` and `.option tnom`](#)
- [Module-based `.ivth` and `.option ivth`](#)
- [Access Model Card and Parameter Inside Module](#)
- [Hierarchical Instantiation With Different IC Modules and Global Node Reference](#)
- [.modulevar Usage Under Hierarchical 3DIC Netlist Configuration](#)

Parameter-passing Rules for 3DIC Hierarchical Instantiation

With hierarchical IC module netlisting configuration, the parameter passing depends on the value of `.option parhier`. The following rules apply:

- For `.option parhier = global`
 1. Value inside `.modulevar` block
 2. Value inside `.module` block
 3. Value in top level
 4. Value defined in X-element instance line
 5. Value defined in `.subckt` block
- For `.option parhier = local`
 1. Value defined in X-element instance line
 2. Value defined in `.subckt` block

3. Value inside `.modulevar` block
4. Value inside `.module` block
5. Value in top level

Options `scale`, `geoshrink` Inside module and `optparhier`

The scaling option takes effect within the IC module only, even when the IC modules are instantiated hierarchically. Thus, each IC module retains its independent properties.

- For `.option optparhier = global`
 1. Value inside `.modulevar` block
 2. Value inside `.module` block
 3. Value in top level
 4. Value defined in `.subckt` block
- For `.option optparhier = local`
 1. Value defined in `.subckt` block
 2. Value inside `.modulevar` block
 3. Value inside `.module` block
 4. Value in top level

Module-based `.temp` and `.option tnom`

The simulation temperature `.temp` takes effect within the IC module only, even when the IC modules are instantiated hierarchically. Thus, each IC module retains its independent properties.

Module-based `.ivth` and `.option ivth`

If this option is declared inside the `.module` construct, it applies to the model card defined within the same `.module` construct only.

Access Model Card and Parameter Inside Module

Model card cross-module reference is illegal under the hierarchical 3DIC netlist configuration. The PrimeSim Continuum simulators check for such references and issue an error message.

Example Illegal Netlist:

```
xtop1 ... tmod1::top  
xtop2 ... tmod2::top
```

```
.module tmod1
  .subckt top
    m1 ... tmod2::nmos1 ...
  .ends top
.endmodule tmod1

* Hierarchical 3DIC netlist configuration
.module tmod2
  .model nmos1 nmos ...
  .subckt top
    x2 ... tmod3::top
    m1 ... tmod3::nmos3 ...
  .ends top
.endmodule tmod2
.module tmod3
  .model nmos3 nmos ...
  .subckt top
    ...
  .ends top
.endmodule tmod3
```

Description:

The `x2top.m1` reference to `nmos1` inside the IC module `tmod2` is illegal as a cross-module reference. The `x2top2.m1` references to the `nmos3` inside the IC module `tmod3` is also illegal under the hierarchical 3DIC netlisting configuration.

Hierarchical Instantiation With Different IC Modules and Global Node Reference

Example Netlist:

```
x2top1 ... tmod1::top1 ...
* Cross hierarchical global node reference.
r1 top_net x2top1.x2top2.vdd r=50
.module tmod1
  .global vdd
  .subckt top1 ...

  * Single level global node reference
  .connect vdd x2top2.vdd
  x2top2 ... tmod2::top2 ...
  ...
  .ends top
.endmodule tmod1
.module tmod2
  .global vdd
  .subckt top2 ...
  ...
  .ends top2
.endmodule tmod2
```

Description:

By default, the `vdd` in `xtop1` is not connected to `vdd` in `xtop1.xtop2`. It requires explicit definitions to connect the nodes, if it is the intention. In this example, the `vdd` in the IC module `tmod1` for `xtop1` and the one in the IC module `tmod2` for `xtop1.xtop2` are connected together.

.modulevar Usage Under Hierarchical 3DIC Netlist Configuration

It is illegal to reference an undefined `.modulevar` label.

Example netlist:

```
xtop1 ... tmod1::top1
.module tmod1
  .temp 40
  .param ptop=3e-008
  .subckt top1 ...
  xtop2 ... tmod2::top2 modulevar="mid"
  m1 ... nmod l="ptop" w=2.7e-006 ...
  .ends top
.endmodule tmod1
.module tmod2
  .temp 45
  .param ptop=8e-008
  .subckt top2 ...
  xtop3 ... tmod3::top3
  m1 ... nmod l="ptop" w=2.7e-006 ...
  .ends top
.endmodule tmod2

.module tmod3
  .temp 30
  .param ptop=6e-008
  .subckt top3 ...
  m1 ... nmod l="ptop" w=2.7e-006 ...
  .ends top
.endmodule tmod3
.modulevar mid
  .temp 55
  .param ptop=5e-008
.endmodulevar mid
```

Instance	Nominal Temperature	Device Length
xtop1.m1	40	3e-008
xtop1.xtop2.m1	55	5e-008
xtop1.xtop2.xtop3.m1	30	6e-008

Output Syntax for 3DIC Hierarchical IC Module Support

The PrimeSim Continuum solution provides IC module label referencing capability for output syntax with the `.print`, `.probe`, and `.measure` statements.

Nodal Voltage Syntax:

```
.print V([ic_module_label::]node_pattern
```

Current Syntax:

```
.print I([ic_module_label::]instance_pattern)
```

Description:

- `[ic_module_label::]` is the optional IC module scope reference.
- `node_pattern` is the wildcard pattern for node matching.
- `instance_pattern` is the wildcard pattern for instance matching.

Hierarchical 3DIC Output Examples

Example netlist:

```
.module ic1
  .subckt tc1 ...
  xchip2 ... ic2::tc2
  ...
  .ends
.endmodule ic1
.module ic2
  .subckt tc2 ...
  xchip3 ... ic3::tc3
  ...
  .ends
.endmodule ic2
.module ic3
  .subckt tc3 ...
  ...
  .ends
.endmodule ic3
xchip1 ... ic1::tc1
xchip4 ... ic2::tc2
```

Matching Specifications

1. All instances/nodes under the given instance path

Example syntax:

```
.print V(xchip1.*)
```

Description: This specification works the same way as 2D circuit reference. It matches all primary nodes with `xchip1` as the prefix hierarchy.

2. IC module `ic1` instances/nodes only

Example syntax:

```
.print V(ic1::*)
```

OR

```
.print V(ic1::xchip1.*)
```

Description: This specification limits the scope matching to the primary nodes under the `xchip1` hierarchy, except all the nodes under the `xchip1.xchip2` domain. There is only one `ic1::tc1` instantiated, therefore the first specification would work as expected without possible ambiguity.

3. IC module `ic2` instances/nodes only

Example syntax:

```
.print V(ic2::*)
```

Description: This specification limited the scope matching to the primary nodes under the `xchip1.xchip2` and `xchip4` hierarchy, except the nodes under the `xchip1.xchip2.xchip3` and `xchip2.xchip3` domains.

4. IC module `ic2` instance/nodes under the `xchip1.xchip2` only

Example syntax:

```
.print V(ic2::xchip1.xchip2*)
```

OR

```
.print V(ic2::xchip1.*)
```

Description: This specification limits the scope matching to the primary nodes under the `xchip1.xchip2` hierarchy only, except the nodes under the `xchip1.xchip2.xchip3` domain. The second specification already determines the matching scope to be under `xchip1`. Thus, the result is the same as the first specification.

5. IC module `ic3` instances/nodes only

Example syntax:

```
.print V(ic3::*)
```

Description: This specification limits the scope matching to the primary nodes under the `xchip1.xchip2.chip3` and `xchip4.xchip3` hierarchies.

6. IC module `ic3` instances/nodes under `xchip1.xchip2.xchip3` only

Example syntax:

```
.print V(ic3::xchip1.xchip2.xchip3*)
```

OR

```
.print V(ic3::xchip1.xchip2 *)
```

OR

```
.print V(ic3::xchip1.*)
```

Description: This specification limits the scope matching to the primary nodes under the `xchip1.xchip2.xchip3` hierarchy only. The second and third specifications already determine the matching scope prefix as `xchip1`. Thus, the result is the same as the first specification.

3DIC Support for TMI, Custom CMI, and MOSRA

The PrimeSim Continuum solution supports the TSMC Model Interface (TMI), Custom Common Model Interface (CMI), and MOSFET Reliability Analysis (MOSRA) models in a 3DIC netlist.

Multiple technology TMI, Custom CMI, or MOSRA model libraries can be packaged to the same netlist through the `.module` command.

The `TMIPath`, `CMIPath`, and `MOSRAPath` options can be set inside the `.module` command and used to specify the paths to the `.so` files for the TMI, CMI, and MOSRA model libraries, respectively.